

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
— o0o —



BÁO CÁO CUỐI KỲ
HỌC MÁY

PHÂN TÍCH VÀ DỰ ĐOÁN NGUY CƠ MẮC BỆNH ALZHEIMER

Nhóm thực hiện: Nhóm 5
Hồ Huyền Trang - 23001565
Trần Thị Mai Anh - 23001497
Đỗ Thị Như Quỳnh - 23001556

Lớp: K68A5 Khoa học dữ liệu

Hà Nội - 2025

Mục lục

Mục lục	i
Danh sách hình vẽ	v
Danh sách bảng	vii
LỜI MỞ ĐẦU	1
2. THÔNG TIN NHÓM	2
2.1 Thành viên nhóm	2
2.2 Phân chia công việc	2
3. TỔNG QUAN	3
3.1 Giới thiệu bài toán	3
3.2 Mục tiêu	3
4. CƠ SỞ LÝ THUYẾT	5
4.1 K-nearest neighbor	5
4.2 Hồi quy Softmax (Softmax Regression)	9
4.3 K-means Clustering	13
4.4 Chỉ số đánh giá phân cụm [2]	18
4.4.1 Silhouette Score	18
4.4.2 Davies–Bouldin Index (DBI)	18
4.4.3 Calinski–Harabasz Index (CH)	18
4.5 DBSCAN Clustering [3]	19
4.6 Chỉ số đánh giá phân cụm [2]	22
4.6.1 Silhouette Score	22
4.6.2 Davies–Bouldin Index (DBI)	23
4.6.3 Calinski–Harabasz Index (CH)	23

4.7	Support Vector Machine (SVM)	24
4.7.1	Hard Margin SVM	24
4.7.2	Soft Margin SVM	27
4.7.3	Multiclass Support Vector Machines	30
5.	TIỀN XỬ LÝ DỮ LIỆU VÀ PHÂN TÍCH DỮ LIỆU	33
5.1	Giới thiệu	33
5.2	Đọc hiểu dữ liệu (Data Understanding)	33
5.3	Phân tích thành phần và miền giá trị (Feature Domain Analysis)	33
5.4	Làm sạch dữ liệu (Data Cleaning)	34
5.4.1	Xử lý giá trị khuyết (Missing Values)	34
5.4.2	Xử lý trùng lặp (Duplicates)	34
5.4.3	Chuẩn hóa và mã hóa dữ liệu	34
5.5	Phát hiện và xử lý ngoại lệ (Outlier Detection)	34
5.6	Trực quan hóa dữ liệu (Data Visualization)	35
5.7	Phân tích quan hệ giữa đặc trưng và mục tiêu (Feature–Target Relationship)	35
5.8	Phân tích tương quan (Correlation Analysis)	35
5.9	Chuẩn hóa và Biến đổi Dữ liệu (Data Transformation)	35
5.9.1	Scaling	35
5.9.2	Biến đổi đặc trưng (Feature Engineering)	36
5.9.3	Giảm chiều dữ liệu	36
5.10	Phân tích Thống kê Mô tả (Descriptive Statistical Analysis)	36
5.11	Đánh giá Chất lượng Dữ liệu (Data Quality Assessment)	36
5.11.1	Phát hiện dữ liệu không hợp lệ	36
5.11.2	Phát hiện inconsistency	36
5.11.3	Gắn cờ dữ liệu nghi ngờ	36
5.12	Phân tích Mọi quan hệ Phức tạp (Complex Relationship Analysis)	37
5.13	Chuẩn bị Dữ liệu cho Mô hình (Model Readiness)	37
5.13.1	Xử lý dữ liệu mất cân bằng	37
5.13.2	Chia dữ liệu	37
5.13.3	Kiểm tra rò rỉ dữ liệu (Data Leakage)	37
5.14	Trực quan hóa Nâng cao (Advanced Visualization)	37
5.15	Kết luận và Đề xuất	38
5.16	Thực hiện trên bộ dữ liệu Alzheimer	38
5.16.1	Thông tin về bộ dữ liệu	38
5.16.2	Tiền xử lý và khám phá dữ liệu	40

6. GIẢM CHIỀU VÀ TRỰC QUAN	46
6.1 Phân tích Thành phần Chính (PCA)	46
6.1.1 Mục tiêu của PCA	46
6.1.2 Cơ sở toán học của PCA	46
6.1.3 Diễn giải kết quả PCA	47
6.2 Phân tích Phân biệt Tuyến tính (LDA)	47
6.2.1 Mục tiêu của LDA	47
6.2.2 Cơ sở toán học của LDA	47
6.2.3 So sánh PCA và LDA	48
6.2.4 Kết luận	48
6.3 Giảm chiều bằng UMAP (Uniform Manifold Approximation and Projection)	48
6.3.1 Giới thiệu	48
6.3.2 Nguyên lý hoạt động	48
6.3.3 Các bước chính	49
6.3.4 Hàm mất mát của UMAP	49
6.3.5 Đặc điểm nổi bật của UMAP	49
6.3.6 So sánh PCA, LDA và UMAP	50
6.4 Thực hành trên tập Alzheimer	50
6.4.1 Chuẩn hóa dữ liệu	50
6.4.2 PCA	50
6.4.3 LDA	55
6.4.4 UMAP	56
6.4.5 Tổng kết	58
7. KẾT QUẢ THỰC NGHIỆM	61
7.1 K-nearest neighbor	61
7.1.1 Trên tập dữ liệu gốc	61
7.1.2 Trên tập dữ liệu đã giảm chiều PCA	63
7.1.3 Trên tập dữ liệu đã giảm chiều LDA	65
7.1.4 Tổng kết trên tập dữ liệu gốc và dữ liệu giảm chiều	66
7.2 Softmax regression	67
7.2.1 Trên tập dữ liệu gốc	67
7.2.2 Trên tập dữ liệu đã giảm chiều PCA	68
7.2.3 Trên tập dữ liệu đã giảm chiều LDA	69
7.2.4 Tổng kết trên tập dữ liệu gốc và dữ liệu giảm chiều	71
7.3 So sánh hai mô hình KNN và Softmax	71
7.4 Phân cụm Kmeans	73

7.4.1	Kmeans với PCA	73
7.4.2	Kmean với Umap	77
7.4.3	Kết luận	80
7.5	DBScan Clustering	81
7.5.1	DBScan với PCA	81
7.5.2	DBScan với Umap	81
7.5.3	Kết luận	83
7.6	So sánh K-means và DBSCAN	83
7.7	Kết quả phân loại với SVM	84
7.7.1	SVC trên dữ liệu gốc (không giảm chiều)	84
7.7.2	SVC trên dữ liệu giảm chiều bằng PCA	85
7.7.3	SVC trên dữ liệu giảm chiều bằng LDA	85
7.7.4	So sánh ba chiến lược biểu diễn dữ liệu	86
7.7.5	Kết luận	86
7.8	So sánh hiệu năng giữa các mô hình SVM, KNN và Softmax Regression	87
7.8.1	So sánh trên dữ liệu gốc	87
7.8.2	So sánh trên dữ liệu giảm chiều bằng PCA	88
7.8.3	So sánh trên dữ liệu giảm chiều bằng LDA	88
7.8.4	Kết luận	89
7.9	Chuyển bài toán về dạng hồi quy với Softmax	89
7.9.1	Lý thuyết	89
7.9.2	Các bước triển khai mô hình	91
7.9.3	Dữ liệu gốc	92
7.9.4	Dữ liệu giảm chiều bằng PCA	93
7.10	Chuyển bài toán về dạng hồi quy với SVM	94
7.10.1	Lý thuyết	94
7.10.2	Các bước triển khai mô hình	95
7.10.3	Dữ liệu gốc	96
7.10.4	Dữ liệu giảm chiều bằng PCA	96
7.10.5	So sánh hiệu suất hai mô hình sử dụng trong bài toán hồi quy dựa trên Softmax Regression và SVM	97
8.	TỔNG KẾT	100
	Tài liệu tham khảo	102

Danh sách hình vẽ

4.1	Cách hoạt động của mô hình K-nearest neighbor	6
4.2	Ví dụ về dữ liệu được phân vào ba cụm. Mỗi cụm có một điểm tâm cụm đại diện màu vàng, và nhóm của mỗi điểm được xác định qua việc nó gần với tâm cụm nào nhất trong ba tâm cụm.[1]	14
4.3	Hình minh họa cách xác định ba loại điểm bao gồm: <i>điểm lõi</i> (<i>core</i>) chấm vuông màu xanh, <i>điểm biên</i> (<i>border</i>) chấm tròn màu đen, và <i>điểm nhiễu</i> (<i>noise</i>) chấm tròn màu trắng trong thuật toán DBSCAN. Các hình tròn đường viền nét đứt bán kính ϵ thể hiện vùng lân cận epsilon tương ứng với các <i>điểm lõi</i> nhằm xác định nhân cho từng điểm. $\text{minPts}=3$ là số lượng tối thiểu để một <i>điểm lõi</i> rơi vào vùng có mật độ cao nếu xung quanh chúng có số lượng điểm tối thiểu là 3. [3]	20
4.4	Minh họa Hard Margin SVM	27
4.5	Kernel biến đổi dữ liệu ở không gian ban đầu sang 1 không gian mới	32
5.1	Phân bố của biến mục tiêu.	43
5.2	Ma trận tương quan giữa các biến số.	45
6.1	Scree plot.	52
6.2	Plot dữ liệu theo PC1, PC2.	52
6.3	Biplot.	53
6.4	Plot từng cặp PCs.	54
6.5	LDA	55
6.6	Unsupervised UMAP 2D	57
6.7	Unsupervised UMAP 3D	57
6.8	Supervised UMAP 2D	58
7.1	Hình ảnh trực quan Dữ liệu giảm chiều bằng PCA 2D [7]	73
7.2	Hình ảnh chỉ số đánh giá với K chạy từ 2 đến 15 Kmeans+PCA [7]	74
7.3	Dữ liệu sau khi phân cụm KMeans+PCA với K=4 [7]	75
7.4	Hình ảnh Dữ liệu giảm chiều bằng Umap [7]	77
7.5	Hình ảnh chỉ số đánh giá với K chạy từ 2 đến 15 Kmeans+Umap [7]	78

7.6	Dữ liệu sau khi phân cụm KMeans+Umap với K=4 [7]	79
7.7	Số cụm và điểm nhiễu của DBSCAN+PCA [7]	82
7.8	Số cụm và điểm nhiễu của DBSCAN+Umap [7]	82

Danh sách bảng

6.1	Tỷ lệ phương sai giải thích của các thành phần chính (PCA) . . .	51
7.1	Bảng kết quả phân loại cho phân chia 4:1	62
7.2	Bảng kết quả phân loại cho phân chia 7:3	62
7.3	Bảng kết quả phân loại cho phân chia 6:4	62
7.4	Bảng kết quả phân loại cho phân chia 4:1	63
7.5	Bảng kết quả phân loại cho phân chia 7:3	63
7.6	Bảng kết quả phân loại cho phân chia 6:4	64
7.7	Bảng kết quả phân loại cho phân chia 4:1	65
7.8	Bảng kết quả phân loại cho phân chia 7:3	65
7.9	Bảng kết quả phân loại cho phân chia 6:4	65
7.10	Bảng kết quả phân loại cho phân chia 4:1	67
7.11	Bảng kết quả phân loại cho phân chia 7:3	67
7.12	Bảng kết quả phân loại cho phân chia 6:4	68
7.13	Bảng kết quả phân loại cho phân chia 4:1	68
7.14	Bảng kết quả phân loại cho phân chia 7:3	68
7.15	Bảng kết quả phân loại cho phân chia 6:4	69
7.16	Bảng kết quả phân loại cho phân chia 4:1	70
7.17	Bảng kết quả phân loại cho phân chia 7:3	70
7.18	Bảng kết quả phân loại cho phân chia 6:4	70
7.19	Các chỉ số đánh giá Kmeans+PCA theo số cụm K [7]	74
7.20	Ma trận tần suất giữa nhãn gốc (<i>Output</i>) và nhãn do K-means gán (<i>Cluster</i>) [7]	76
7.21	Các chỉ số đánh giá Kmeans+Umap theo số cụm K [7]	78
7.22	Ma trận tần suất giữa nhãn gốc (<i>Output</i>) và nhãn do K-means gán (<i>Cluster</i>) [7]	80
7.23	Chỉ số đánh giá phân cụm của bốn thực nghiệm [7]	83
7.24	Kết quả trên dữ liệu gốc với các tỉ lệ chia khác nhau	92
7.25	Kết quả trên dữ liệu PCA với các tỉ lệ chia khác nhau	93
7.26	Kết quả trên dữ liệu gốc với các tỉ lệ chia khác nhau	96

7.27	Kết quả trên dữ liệu PCA với các tỉ lệ chia khác nhau	96
7.28	So sánh hiệu suất giữa Softmax Regression và SVM (RBF) trên dữ liệu gốc và dữ liệu PCA	98

LỜI MỞ ĐẦU

Trong bối cảnh ngành y sinh đang phát triển mạnh mẽ theo xu hướng ứng dụng công nghệ vào quá trình chuẩn đoán bệnh trong thời đại 4.0. Dựa vào đó, chúng em đi phân tích và dự đoán bệnh Alzheimer nguyên nhân hàng đầu gây sa sút trí tuệ, ảnh hưởng nghiêm trọng đến sức khỏe cộng đồng và gánh nặng kinh tế. Việc chẩn đoán sớm có ý nghĩa sống còn để can thiệp kịp thời, làm chậm tiến trình bệnh và cải thiện chất lượng sống cho bệnh nhân.

Báo cáo này được thực hiện trong khuôn khổ môn học Machine Learning, với mục tiêu áp dụng kiến thức lý thuyết vào việc xây dựng mô hình phân loại để dự đoán khả năng thu thập dữ liệu y tế toàn diện từ những thông số lâm sàng, lối sống đến kết quả xét nghiệm. Học máy trở thành công cụ đắc lực giúp phát hiện các mẫu phức tạp và mối quan hệ giữa các yếu tố nguy cơ mà con người khó nhận ra được. Từ đó xây dựng các mô hình dự đoán không xâm lấn mang lại hiệu quả và độ chính xác cao.

Nhóm nghiên cứu đã thực hiện các bước phân tích dữ liệu và áp dụng các thuật toán học máy để xây dựng mô hình phân loại, từ đó tìm ra các mối quan hệ giữa các yếu tố giúp làm chậm tiến trình bệnh. Chúng em kỳ vọng báo cáo này sẽ cung cấp những thông tin hữu ích về khả năng ứng dụng học máy trong y sinh, đồng thời nâng cao khả năng phân tích dữ liệu và phát triển các mô hình dự đoán hiệu quả.

Chúng em xin gửi lời cảm ơn chân thành đến Thầy Cao Văn Chung, người đã hỗ trợ và tạo điều kiện để chúng em thực hiện báo cáo này. Hy vọng rằng báo cáo sẽ là tài liệu tham khảo hữu ích cho việc phát triển các mô hình dự đoán trong ngành y sinh, đồng thời đóng góp vào nghiên cứu và ứng dụng học máy trong các lĩnh vực khác.

Xin chân thành cảm ơn!

Hà Nội, ngày 18 tháng 01 năm 2025

Nhóm sinh viên thực hiện

Hồ Huyền Trang
Trần Thị Mai Anh
Đỗ Thị Như Quỳnh

THÔNG TIN NHÓM

2.1 Thành viên nhóm

Nhóm gồm ba thành viên:

- Hồ Huyền Trang - 23001565
- Trần Thị Mai Anh - 23001497
- Đỗ Thị Như Quỳnh - 23001556

2.2 Phân chia công việc

Với mong muốn tạo ra một môi trường làm việc thật chuyên nghiệp, công bằng và mang lại kết quả tốt nhất, nhóm đã phân chia đều các công việc cho các thành viên, mỗi thành viên sẽ phụ trách một mảng chính của đề tài. Trong quá trình làm việc nếu gặp khó khăn, cả nhóm sẽ thảo luận và đưa ra các phương án để giải quyết. Cụ thể như sau:

- Hồ Huyền Trang: Phân tích dữ liệu và áp dụng giảm chiều PCA, LDA, UMap, lý thuyết và mô hình SVM.
- Trần Thị Mai Anh: Lý thuyết và mô hình K-Nearest Neighbors, phân cụm bằng Kmeans, DBSCAN.
- Đỗ Thị Như Quỳnh: Lý thuyết và mô hình Softmax Regression, đưa bài toán phân loại Softmax Regression, SVM về bài toán hồi quy.

Ngoài ra các thành viên trong nhóm cùng nhau chia sẻ, trao đổi đưa ra dự đoán về nguy cơ mắc bệnh Alzheimer, viết báo cáo và làm slide.

TỔNG QUAN

3.1 Giới thiệu bài toán

Bệnh Alzheimer là nguyên nhân hàng đầu gây sa sút trí tuệ, ảnh hưởng nghiêm trọng đến sức khỏe cộng đồng và gánh nặng kinh tế. Việc chẩn đoán sớm có ý nghĩa sống còn để can thiệp kịp thời, làm chậm tiến trình bệnh và cải thiện chất lượng sống cho bệnh nhân.

Với sự phát triển của công nghệ và khả năng thu thập dữ liệu y tế toàn diện từ những thông số lâm sàng, lối sống đến kết quả xét nghiệm. Học máy trở thành công cụ đắc lực giúp phát hiện các mẫu phức tạp và mối quan hệ giữa các yếu tố nguy cơ mà con người khó nhận ra được. Từ đó xây dựng các mô hình dự đoán không xâm lấn mang lại hiệu quả và độ chính xác cao.

Trong bài báo cáo này, chúng em tập trung vào việc phân tích, chuẩn đoán sớm để can thiệp kịp thời, làm chậm tiến trình bệnh, sử dụng bộ dữ liệu "alzheimers_disease_data" thu thập từ Kaggle. Trong bộ dữ liệu gồm các nhóm tính năng về nhân khẩu học và lối sống, lịch sử bệnh và nguy cơ, đánh giá chức năng và nhận thức, chỉ số sinh học.

Qua đó giúp chúng em xây dựng mô hình chuẩn đoán với 2 mức độ (Không mắc bệnh - mắc bệnh ở mức độ nhẹ: "not sick - mildly sick", Mắc bệnh: "sick"). Mô hình này không chỉ mang lại cái nhìn sâu sắc và khách quan về các yếu tố cốt lõi ảnh hưởng đến việc chuẩn đoán bệnh của bệnh nhân, mà còn đóng góp vào việc làm chậm tiến trình bệnh và cải thiện chất lượng sống cho bệnh nhân.

3.2 Mục tiêu

Mục tiêu chính của báo cáo này là xây dựng và triển khai mô hình phân loại dựa trên bộ dữ liệu "alzheimers_disease_data" để chuẩn đoán bệnh. Cụ thể, chúng em đặt ra các mục tiêu sau:

- **Phân tích dữ liệu:** Khám phá và phân tích các yếu tố ảnh hưởng đến dự đoán mắc bệnh của bệnh nhân, bao gồm nhân khẩu học và lối sống, lịch sử bệnh và nguy cơ, đánh giá chức năng và nhận thức, chỉ số sinh học.
- **Tiền xử lý dữ liệu:** Áp dụng các kỹ thuật tiền xử lý dữ liệu để xử lý các giá trị thiếu, giảm chiều dữ liệu, chuẩn hóa và mã hóa các thuộc tính cần

thiết, giúp tối ưu hóa quá trình huấn luyện mô hình học máy.

- **Xây dựng mô hình dự đoán:** Xây dựng mô hình phân loại sử dụng các thuật toán học máy như Softmax Regression, K-Nearest Neighbors để dự đoán mắc bệnh của bệnh nhân. Đánh giá và lựa chọn mô hình có hiệu quả dự đoán tốt nhất dựa trên các chỉ số như accuracy, precision, recall và F1-score.
- **Phân tích yếu tố quyết định:** Phân tích và xác định các yếu tố quan trọng nhất ảnh hưởng đến việc bệnh nhân mắc bệnh thông qua việc đánh giá các trọng số của các thuộc tính trong mô hình dự đoán.

Chúng em kỳ vọng báo cáo này sẽ cung cấp những hiểu biết sâu sắc về cách thức ứng dụng học máy trong ngành y sinh, đồng thời đóng góp vào việc nâng cao chất lượng, để can thiệp kịp thời, làm chậm tiến trình bệnh và cải thiện chất lượng sống cho bệnh nhân.

CƠ SỞ LÝ THUYẾT

4.1 K-nearest neighbor

K-nearest neighbor (KNN) là một trong những thuật toán thuộc nhóm học giám sát, nổi tiếng với sự đơn giản. Trong giai đoạn huấn luyện, KNN không thực hiện bất kỳ việc học nào từ dữ liệu huấn luyện mà nhớ lại một cách máy móc toàn bộ dữ liệu đó; toàn bộ quá trình tính toán chỉ diễn ra khi cần dự đoán đầu ra cho dữ liệu mới. Thuật toán này có thể được áp dụng cho cả hai loại bài toán phân loại và hồi quy. KNN còn được gọi là một thuật toán dựa theo mẫu hay thuật toán dựa theo trí nhớ.

Trong KNN, đối với bài toán phân loại, nhãn của một dữ liệu mới được xác định dựa trên K điểm dữ liệu gần nhất trong tập huấn luyện. Nhãn đó có thể được quyết định thông qua phương pháp bầu chọn đa số (major voting) trong số K điểm gần nhất, hoặc nó có thể được suy ra bằng cách đánh trọng số khác nhau cho mỗi trong các điểm gần nhất đó rồi suy ra kết quả.

Đối với bài toán hồi quy, kết quả đầu ra của một điểm dữ liệu mới sẽ được xác định bằng giá trị của điểm dữ liệu gần nhất trong tập huấn luyện (khi $K=1$), hoặc được tính dựa trên giá trị trung bình có trọng số của các điểm lân cận, hoặc thông qua một mối quan hệ phụ thuộc vào khoảng cách tới các điểm gần nhất.

Cũng đáng lưu ý rằng thuật toán KNN cũng là một phần của họ các mô hình "học lười biếng", nghĩa là nó chỉ lưu trữ một tập dữ liệu đào tạo so với việc trải qua một giai đoạn đào tạo. Điều này cũng có nghĩa là tất cả các phép tính diễn ra khi phân loại hoặc dự đoán đang được thực hiện. Vì nó phụ thuộc rất nhiều vào bộ nhớ để lưu trữ tất cả dữ liệu đào tạo của nó, nên nó cũng được gọi là phương pháp học dựa trên thể hiện hoặc dựa trên bộ nhớ.

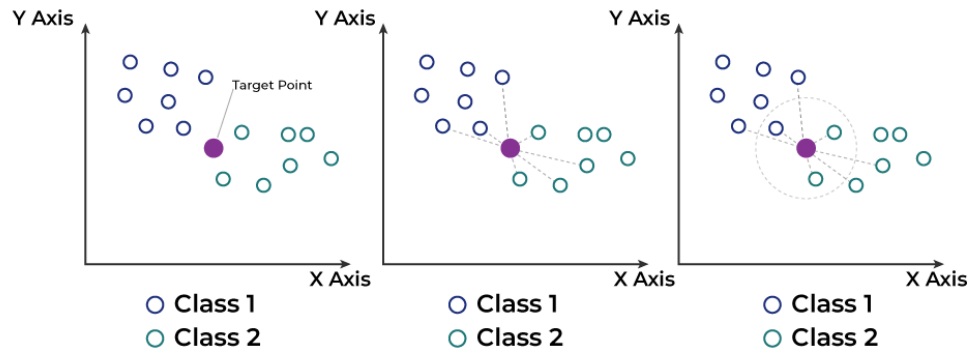
Tóm lại, KNN là thuật toán đi tìm đầu ra cho một điểm dữ liệu mới bằng cách dựa trên thông tin từ K điểm gần nhất trong tập huấn luyện, mà không xét đến việc một số điểm trong đó có thể là nhiễu.

Mặc dù không còn phổ biến như trước đây, nhưng đây vẫn là một trong những thuật toán đầu tiên mà người ta học trong khoa học dữ liệu do tính đơn giản và chính xác của nó. Tuy nhiên, khi tập dữ liệu phát triển, KNN ngày càng kém

hiệu quả, làm giảm hiệu suất chung của mô hình. Nó thường được sử dụng cho các hệ thống đề xuất đơn giản, nhận dạng mẫu, khai thác dữ liệu, dự đoán thị trường tài chính, phát hiện xâm nhập, v.v.

* Cách KNN hoạt động

Thuật toán K-Nearest Neighbors (KNN) hoạt động dựa trên một nguyên tắc đơn giản: dự đoán nhãn hoặc giá trị của một điểm dữ liệu mới bằng cách tham khảo nhãn hoặc giá trị của K điểm lân cận gần nhất trong tập huấn luyện.



Hình 4.1: Cách hoạt động của mô hình K-nearest neighbor

- **Bước 1:** Lựa chọn giá trị thích hợp cho K

K là số lượng điểm lân cận gần nhất được sử dụng để đưa ra dự đoán kết quả. Ví dụ, nếu $K = 3$, thuật toán sẽ tìm 3 điểm gần nhất để quyết định nhãn hoặc giá trị cho điểm dữ liệu mới.

- **Bước 2:** Tính toán khoảng cách

Để xác định điểm dữ liệu nào gần nhất với điểm truy vấn nhất định, khoảng cách giữa điểm truy vấn và các điểm dữ liệu khác sẽ cần được tính toán. Các số liệu khoảng cách này giúp hình thành ranh giới quyết định, phân chia các điểm truy vấn thành các vùng khác nhau, các khoảng cách phổ biến được sử dụng bao gồm:

Khoảng cách Euclidean ($p=2$)

Khoảng cách Euclid, thường được coi là số liệu trực quan nhất, tính toán khoảng cách đường thẳng giữa hai điểm dữ liệu trong không gian đa chiều.

Công thức của nó, bắt nguồn từ định lý Pythagore, rất đơn giản:

$$d_{\text{Euclidean}}(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

trong đó:

- x và y là hai điểm dữ liệu.
- x_i và y_i là các thành phần tại chiều thứ i .
- n là số chiều của không gian dữ liệu.

Khoảng cách Manhattan (p=1)

Khoảng cách Manhattan, còn được gọi là khoảng cách taxi, đo khoảng cách giữa hai điểm bằng cách cộng các chênh lệch tuyệt đối của tọa độ của chúng theo từng chiều. Trong hai chiều, hãy hình dung nó như việc di chuyển trong một khối thành phố dạng lưới - bạn chỉ có thể di chuyển theo chiều ngang hoặc chiều dọc.

Khoảng cách Manhattan trở nên đặc biệt hữu ích trong các không gian chiều cao thừa thớt dân cư như được mô tả bởi Aggarwal và cộng sự (“Về hành vi đáng ngạc nhiên của các phép đo khoảng cách trong không gian chiều cao,” 2001), hoặc nếu các đặc điểm có tỷ lệ khác biệt đáng kể, trong đó việc bình phương các điểm khác biệt có xu hướng khuếch đại khoảng cách do các số lớn hơn gây ra (hãy tưởng tượng một đặc điểm có giá trị từ 0 đến 10, so với đặc điểm có giá trị từ 0 đến 1).

$$d_{\text{Manhattan}}(x, y) = \sum_{i=1}^n |x_i - y_i|$$

Khoảng cách Minkowski

Đo khoảng cách này là dạng tổng quát của các số liệu khoảng cách Euclidean và Manhattan. Tham số p trong công thức bên dưới cho phép tạo ra các số liệu khoảng cách khác.

$$d_{\text{Minkowski}}(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}}$$

- Khi $p = 1$, khoảng cách Minkowski trở thành khoảng cách Manhattan.
- Khi $p = 2$, khoảng cách Minkowski trở thành khoảng cách Euclidean.
- p là một tham số tự do.

- **Bước 3:** Xác định K hàng xóm gần nhất

K điểm dữ liệu có khoảng cách nhỏ nhất đến điểm mục tiêu được chọn làm các hàng xóm gần nhất. Nhìn chung, giá trị K nhỏ có thể dẫn đến kết quả quá nhạy cảm với nhiễu trong dữ liệu, trong khi giá trị K lớn có thể không nắm bắt được những hàng xóm có gần nhất.

Một số kỹ thuật có thể giúp xác định giá trị K tối ưu cho các ứng dụng tìm kiếm vectơ, bao gồm:

- **Cross-Validation:** Trong bối cảnh này, cross-validation liên quan đến việc phân vùng tập dữ liệu thành nhiều tập con. Hiệu suất của thuật toán tìm kiếm được đánh giá trên các tập con này để xác định giá trị K hàng đầu cân bằng giữa độ chính xác và khả năng thu hồi.
- **Phương pháp Elbow:** Phương pháp này vẽ biểu đồ tỷ lệ lỗi tìm kiếm hoặc số liệu hiệu suất có liên quan so với nhiều giá trị K khác nhau. 'Điểm Elbow' trên biểu đồ, nơi tỷ lệ cải thiện bắt đầu chậm lại, thường chỉ ra lựa chọn tốt cho K . Điểm này biểu thị sự cân bằng giữa việc có quá ít và quá nhiều hàng xóm.
- **GridSearchCV:** Grid search hoạt động có hệ thống thông qua nhiều tổ hợp giá trị tham số, trong trường hợp này là các giá trị K khác nhau, để tìm ra thiết lập tham số tối ưu. Kỹ thuật này đòi hỏi nhiều tính toán nhưng có thể rất hiệu quả. Giá trị hiệu suất tốt nhất có thể được chọn bằng cách đánh giá hiệu suất của thuật toán tìm kiếm trên một phạm vi giá trị K .
- **RandomizedSearchCV:** RandomizedSearchCV tìm kiếm siêu tham số bằng cách thử nghiệm ngẫu nhiên trong không gian tham số, thay vì thử tất cả các tổ hợp như GridSearchCV. Phương pháp này giúp tiết kiệm thời gian tính toán và có thể hiệu quả hơn khi số lượng tham số lớn, đồng thời tìm ra giá trị siêu tham số tối ưu cho mô hình.

Việc sử dụng các kỹ thuật này có thể xác định giá trị K cao nhất giúp cân bằng tối ưu độ chính xác tìm kiếm và hiệu quả tính toán, do đó cải thiện hiệu suất và độ mạnh mẽ của tìm kiếm vectơ.

- **Bước 4:** Dự đoán kết quả dựa trên Phân loại hoặc Hồi quy

- Đối với bài toán **Phân loại**, nhãn của điểm mới được xác định thông qua **bỏ phiếu đa số** từ K điểm lân cận. Nhãn xuất hiện nhiều nhất sẽ là nhãn dự đoán.
- Đối với bài toán **Hồi quy**, giá trị dự đoán được tính bằng **trung bình có trọng số** của các giá trị từ K điểm lân cận gần nhất.

Thuật toán KNN, giống như bất kỳ công cụ nào khác, đều có những ưu điểm và nhược điểm riêng:

Những lợi ích:

- **Dễ hiểu và triển khai:** Bản chất trực quan của KNN giúp nhiều người dùng, từ người mới bắt đầu đến người có kinh nghiệm, đều có thể sử dụng được. Do tính đơn giản và chính xác của thuật toán, đây là một trong những bộ phân loại đầu tiên mà một nhà khoa học dữ liệu mới vào nghề sẽ học.
- **Dễ thích ứng:** Khi các mẫu đào tạo mới được thêm vào, thuật toán sẽ điều chỉnh để tính đến bất kỳ dữ liệu mới nào vì tất cả dữ liệu đào tạo đều được lưu trữ trong bộ nhớ.
- **Ít siêu tham số:** KNN chỉ yêu cầu giá trị k và số liệu khoảng cách p, thấp khi so sánh với các thuật toán học máy khác, giúp huấn luyện đơn giản hơn.
- **Tính linh hoạt trong việc xử lý nhiều loại dữ liệu khác nhau:** KNN hỗ trợ nhiều loại dữ liệu khác nhau, phù hợp với nhiều ứng dụng khác nhau.

Mặt hạn chế:

- **Tốn kém về mặt tính toán:** Việc tính toán khoảng cách giữa tất cả các điểm dữ liệu, đặc biệt là trong các tập dữ liệu lớn, có thể tốn nhiều công sức tính toán, ảnh hưởng đến tốc độ.
- **Độ nhạy với giá trị ngoại lệ:** Hiệu suất của KNN có thể bị ảnh hưởng đáng kể bởi các giá trị ngoại lệ trong dữ liệu.
- **Dễ bị quá khớp:** KNN gặp phải "lời nguyền của đa chiều", khi phải xử lý dữ liệu với quá nhiều chiều, làm giảm khả năng phân loại chính xác. Điều này cũng khiến thuật toán dễ bị quá khớp, và để khắc phục, các kỹ thuật giảm chiều và lựa chọn đặc điểm thường được áp dụng.

4.2 Hồi quy Softmax (Softmax Regression)

Bài toán Phân loại Đa lớp (Multiclass classification) là một trong những ứng dụng của học sâu/học máy, trong đó mô hình nhận đầu vào và đưa ra đầu ra

phân loại tương ứng với một trong các nhãn trong tập các nhãn đầu ra đã có. Để thực hiện điều này, khi mô hình phải xuất ra nhiều giá trị cho mỗi lớp, hàm logistic đơn giản (hay còn gọi là hàm sigmoid) không thể được sử dụng. Thay vào đó, một hàm kích hoạt khác gọi là hàm Softmax được sử dụng cùng với hàm mất mát cross-entropy.

Công thức của hàm Softmax

Chúng ta cần một mô hình xác suất để với mỗi đầu vào $\mathbf{x}$, giá trị a_i biểu diễn xác suất đầu vào thuộc về lớp i . Điều kiện cần thiết là các giá trị a_i , phải dương và tổng của chúng bằng 1. Để đảm bảo các điều kiện này, chúng ta cần quan sát giá trị z_i ; và xác định mối liên hệ giữa z_i và a_i để tính a_i .

Bên cạnh đó, ngoài việc a_i phải dương và tổng a_i bằng 1, cần thêm một điều kiện khác: a_i tăng theo z_i . Nghĩa là, nếu z_i càng lớn thì xác suất a_i càng cao hay xác suất lớp i được dự đoán cao hơn, điều này có thể đạt được bằng một hàm số đơn điệu tăng.

Với z_i có thể là bất kỳ giá trị thực nào (âm hoặc dương), một cách tự nhiên để chuyển nó thành giá trị dương, đồng thời đảm bảo tính đơn điệu, là sử dụng hàm e^{z_i} . Khi đó, để đảm bảo tổng các a_i bằng 1, ta định nghĩa:

$$a_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}, \quad \forall i = 1, 2, \dots, C$$

Hàm này, gọi là **Hàm Softmax (Softmax Function)**, tính toán tất cả các giá trị a_i từ các giá trị z_i , thỏa mãn: $a_i > 0$, tổng các $a_i = 1$, và thứ tự của z_i được bảo toàn. Lưu ý rằng, không có a_i nào tuyệt đối bằng 0 hoặc 1, trừ khi z_i rất nhỏ hoặc rất lớn so với các z_j , $j \neq i$.

Khi đó, xác suất để điểm dữ liệu $\mathbf{x}$ thuộc về lớp i được viết là:

$$P(y = i \mid \mathbf{x}; \mathbf{W}) = a_i$$

Trong đó, $P(y = i \mid \mathbf{x}; \mathbf{W})$ là xác suất mà mô hình (với tham số $\mathbf{W}$) dự đoán điểm $\mathbf{x}$ thuộc lớp i .

One-hot coding

Trong các mô hình mạng nơ-ron, đầu ra thường không phải là một giá trị đơn lẻ đại diện cho mỗi lớp, mà là một vector gọi là *one-hot vector*. Vector này có duy nhất một phần tử bằng 1, các phần tử còn lại bằng 0. Phần tử bằng 1 nằm ở vị trí tương ứng với lớp của mẫu đầu vào, biểu diễn rằng mẫu này chắc chắn thuộc

về lớp đó (xác suất 1). Đây chính là phương pháp mã hóa **one-hot coding**. Ví dụ, với 3 lớp, lớp thứ 2 sẽ có one-hot vector là $[0, 1, 0]$.

Khi áp dụng mô hình Softmax Regression, với mỗi đầu vào $\mathbf{x}$, ta tính **đầu ra dự đoán** bằng công thức:

$$\mathbf{a} = \text{softmax}(\mathbf{W}^\top \mathbf{x})$$

Trong đó, $\mathbf{a}$ là xác suất dự đoán cho từng lớp. Ngược lại, đầu ra thực sự (ground truth label) $\mathbf{y}$ được biểu diễn dưới dạng one-hot vector, với giá trị đúng là 1 tại vị trí của lớp đúng và 0 ở các vị trí khác.

Hàm mất mát (loss function) được dùng để đo lường mức độ khác biệt giữa đầu ra dự đoán $\mathbf{a}_i$ và nhãn thực sự $\mathbf{y}_i$. Một lựa chọn đơn giản là sử dụng khoảng cách bình phương (MSE):

$$J(\mathbf{W}) = \sum_{i=1}^N \|\mathbf{a}_i - \mathbf{y}_i\|_2^2$$

*Tuy nhiên, khi làm việc với phân phối xác suất (softmax), hàm MSE không phải lựa chọn phù hợp, do nó không phản ánh rõ mức độ khác biệt giữa hai phân phối. Để cải thiện, ta sử dụng một đại lượng hiệu quả hơn gọi là **cross-entropy**.*

Cross-Entropy

Cross-entropy là một hàm mất mát phổ biến trong bài toán phân loại nhiều lớp, đo lường khoảng cách giữa phân phối xác suất thực tế và phân phối dự đoán. Khoảng cách giữa hai phân phối $\mathbf{p}$ và $\mathbf{q}$ được định nghĩa là:

$$H(\mathbf{p}, \mathbf{q}) = \mathbb{E}_{\mathbf{p}}[-\log \mathbf{q}]$$

Trong trường hợp phân phối rời rạc (như vector $\mathbf{y}$ và $\mathbf{a}$), công thức trên có thể viết lại dưới dạng:

$$H(\mathbf{p}, \mathbf{q}) = - \sum_{i=1}^C p_i \log q_i$$

Trong bài toán phân loại, p_i thường là nhãn thực sự (dạng one-hot) và q_i là xác suất dự đoán.

Cross-entropy giúp đo lường hiệu quả sự khác biệt giữa hai phân phối, đặc biệt hữu ích trong các bài toán phân loại. Vì vậy, cross-entropy thường được sử dụng làm hàm mất mát chính trong huấn luyện mô hình softmax regression hoặc

mạng nơ-ron phân loại.

Chú ý: Hàm Cross-Entropy không có tính đối xứng, nghĩa là $H(\mathbf{p}, \mathbf{q}) \neq H(\mathbf{q}, \mathbf{p})$. Điều này có thể được giải thích qua công thức của hàm. Trong đó, các thành phần của phân phối $\mathbf{q}$ bằng 0 sẽ dẫn tới giá trị $\log(0)$, vốn không xác định. Vì vậy, khi sử dụng Cross-Entropy trong bài toán học có giám sát (supervised learning), giá trị đầu ra thực sự $\mathbf{y}$ được biểu diễn dưới dạng one-hot vector: một phần tử có giá trị 1 (ứng với lớp đúng), còn lại bằng 0. Trong khi đó, đầu ra dự đoán $\mathbf{q}$ là phân phối xác suất dự đoán của mô hình, giá trị này không bao giờ tuyệt đối bằng 0 hay 1.

Trong **Logistic Regression**, đầu ra dự đoán là một phân phối đơn giản:

- **Đầu ra thực sự** của điểm dữ liệu đầu vào $\mathbf{x}_i$ có phân phối xác suất là $[y_i; 1 - y_i]$, với y_i là xác suất để đầu vào rơi vào lớp thứ nhất (bằng 1 nếu đúng, bằng 0 nếu sai).
- **Đầu ra dự đoán** là phân phối xác suất dự đoán: $\hat{y}_i = \text{sigmoid}(\mathbf{w}^\top \mathbf{x}_i)$, biểu diễn xác suất dự đoán rơi vào lớp thứ nhất. Do đó, xác suất dự đoán rơi vào lớp thứ hai là $1 - \hat{y}_i$.

Với cách định nghĩa này, hàm mất mát của Logistic Regression được viết dưới dạng:

$$J(\mathbf{w}) = - \sum_{i=1}^N \left(y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right)$$

Hàm mất mát này là một trường hợp đặc biệt của cross-entropy khi số lớp $C = 2$, trong đó N là số lượng điểm dữ liệu trong tập huấn luyện.

Khi số lớp $C > 2$, mô hình logistic được mở rộng thành Softmax Regression, trong đó mỗi điểm đầu ra là một vector xác suất có tổng bằng 1. Hàm mất mát giữa đầu ra dự đoán $\mathbf{a}_i$ và đầu ra thực sự $\mathbf{y}_i$ của một điểm dữ liệu được định nghĩa như sau:

$$J(\mathbf{W}; \mathbf{x}_i, \mathbf{y}_i) = - \sum_{j=1}^C y_{ij} \log(a_{ij})$$

Trong đó:

- y_{ij} là thành phần thứ j trong vector $\mathbf{y}_i$ (xác suất thực sự),
- a_{ij} là thành phần thứ j trong vector $\mathbf{a}_i$ (xác suất dự đoán),
- $\mathbf{W}$ là ma trận trọng số của mô hình. Nếu nhãn thực sự là one-hot, chỉ phần tử ứng với lớp đúng là $y_{ij} = 1$, do đó chỉ có một phần tử đóng góp vào hàm mất mát.

Hàm mất mát tổng quát cho Softmax Regression

Với tập dữ liệu gồm N cặp đầu vào $(\mathbf{x}_i, \mathbf{y}_i)$ với $i = 1, 2, \dots, N$, hàm mất mát tổng quát cho Softmax Regression được tính như sau:

$$J(\mathbf{W}; \mathbf{X}, \mathbf{Y}) = - \sum_{i=1}^N \sum_{j=1}^C y_{ij} \log(a_{ij})$$

Khi thay $\mathbf{a}_i$ bằng biểu thức Softmax, ta có:

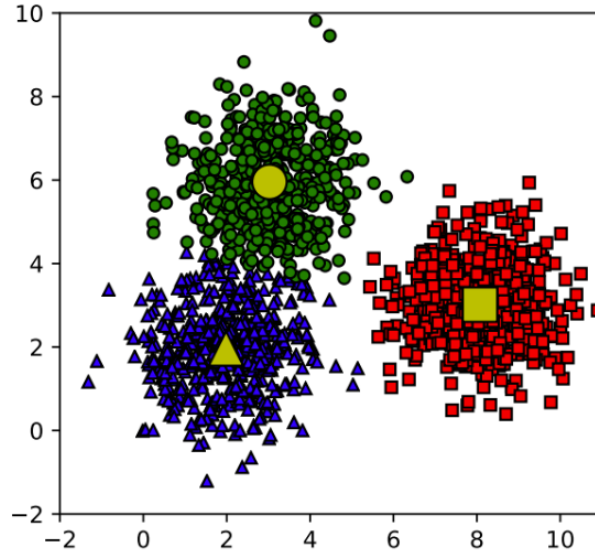
$$J(\mathbf{W}; \mathbf{X}, \mathbf{Y}) = - \sum_{i=1}^N \sum_{j=1}^C y_{ij} \log \left(\frac{e^{\mathbf{w}_j^\top \mathbf{x}_i}}{\sum_{k=1}^C e^{\mathbf{w}_k^\top \mathbf{x}_i}} \right)$$

4.3 K-means Clustering

Trong bài toán này, chúng ta sẽ sử dụng thuật toán phân cụm K-means để phân cụm dữ liệu. Vậy K-means là gì? Đây là một thuật toán học máy không giám sát, được thiết kế để phân chia các dữ liệu không có nhãn thành nhiều cụm khác nhau dựa trên sự tương đồng của chúng. Mục đích là làm thế nào để phân dữ liệu thành các cụm (cluster) khác nhau sao cho dữ liệu trong cùng một cụm có những tính chất giống nhau.

Một định nghĩa đơn giản của nhóm/cụm (cluster) là tập hợp các điểm có các vector đặc trưng gần nhau. Việc đo khoảng cách giữa các vector thường được thực hiện dựa trên norm, trong đó khoảng cách Euclid, tức l2 norm, được sử dụng phổ biến hơn cả.

Thuật toán K-means clustering hoạt động dựa trên việc phân chia các điểm dữ liệu vào một trong K cụm, dựa trên khoảng cách của chúng đến tâm cụm. Ban đầu, các tâm cụm được chọn ngẫu nhiên trong không gian. Mỗi điểm dữ liệu sau đó sẽ được gán vào cụm có tâm gần nhất. Khi tất cả các điểm đã được gán cụm, các tâm cụm sẽ được tính toán lại dựa trên vị trí trung bình của các



Hình 4.2: Ví dụ về dữ liệu được phân vào ba cụm. Mỗi cụm có một điểm tâm cụm đại diện màu vàng, và nhóm của mỗi điểm được xác định qua việc nó gần với tâm cụm nào nhất trong ba tâm cụm.[1]

điểm dữ liệu trong cụm đó. Quá trình này tiếp tục lặp lại cho đến khi các cụm không còn thay đổi hoặc đạt được trạng thái ổn định. Trong quá trình thực hiện, số lượng cụm K được giả định là đã biết trước, với mục tiêu chính là phân loại các điểm dữ liệu vào các nhóm phù hợp.

Trong một số trường hợp, số lượng cụm K không được xác định trước, do đó, cần phải tìm giá trị K tối ưu. K -means hoạt động hiệu quả nhất khi dữ liệu có sự phân tách rõ ràng. Tuy nhiên, nếu các điểm dữ liệu bị chồng lớp, thuật toán này sẽ mất hiệu quả. So với các phương pháp phân cụm khác, K -means có ưu điểm về tốc độ xử lý nhanh và khả năng kết nối tốt giữa các điểm dữ liệu trong cùng một cụm. Mặc dù vậy, thuật toán này cũng có một số hạn chế. Nó không cung cấp đánh giá rõ ràng về chất lượng của các cụm được tạo ra, và kết quả phân cụm có thể thay đổi tùy thuộc vào cách khởi tạo ngẫu nhiên các tâm cụm ban đầu. Ngoài ra, K -means rất nhạy cảm với nhiễu và dễ bị mắc kẹt tại các cực trị cục bộ.

Phân tích toán học Thuật toán k -means[2]

- Cho tập dữ liệu $\mathbf{X} = \{x_n\}_{n=1}^N$ với $x_n \in \mathbb{R}^d$ và số cụm cần phân chia $1 < k < N$.

- Đặt biến ẩn $y_{n,j} \in \{0, 1\}$, trong đó $y_{n,j} = 1$ nếu và chỉ nếu x_n thuộc cụm thứ j . Ngược lại $y_{n,j} = 0$.

- Với mỗi x_n , ta có: $\sum_{j=1}^K y_{n,j} = 1$.

- Giả sử đã biết K tâm cụm (centroids) là $M = \{m_k\}_{k=1}^K$.

- Giả sử các điểm x_n được gán vào cụm thứ k . Tổng phương sai nội bộ (*within-class variance*) của cụm thứ k , cho mỗi x_n , sẽ là

$$\sum_{x_n \in \text{Cluster}_k} \|x_n - m_k\|_2^2 = \sum_{y_{n,k}=1} \|x_n - m_k\|_2^2.$$

- Tổng phương sai nội bộ trên tất cả các cụm:

$$\text{Loss} = L(\mathbf{Y}, \mathbf{M}) = \sum_{n=1}^N \sum_{j=1}^K y_{n,j} \|x_n - m_j\|_2^2.$$

- Tìm tham số tối ưu:

$$(\mathbf{Y}^*, \mathbf{M}^*) = \arg \min_{\mathbf{Y}, \mathbf{M}} L(\mathbf{Y}, \mathbf{M}) = \arg \min_{\mathbf{Y}, \mathbf{M}} \sum_{n=1}^N \sum_{j=1}^K y_{n,j} \|x_n - m_j\|_2^2,$$

$$\text{với ràng buộc: } y_{n,j} \in \{0, 1\}, \quad \sum_{j=1}^K y_{n,j} = 1, \quad \forall n = 1, \dots, N.$$

- Giả sử đã có các tâm cụm hiện tại $\{m_c\}_{c=1}^K$.

- Do $y_{n,j} \in \{0, 1\}$ và $\sum_{j=1}^K y_{n,j} = 1$,

$$\sum_{n=1}^N \sum_{j=1}^K y_{n,j} \|x_n - m_j\|_2^2 \longrightarrow \min_{\mathbf{Y}, \mathbf{M}}$$

nếu và chỉ nếu

$$\begin{cases} y_{n,j} = 1 & \text{khi và chỉ khi } \|x_n - m_j\|_2^2 = \min_c \|x_n - m_c\|_2^2, \\ y_{n,j} = 0 & \text{tất cả trường hợp khác.} \end{cases}$$

- Với một bộ $\{y_{n,j}\}_{n=1,j=1}^{N,K}$ đã cho, ta có:

$$\sum_{n=1}^N \sum_{j=1}^K y_{n,j} \|x_n - m_j\|_2^2 =: J(Y, M) \longrightarrow \min_M$$

nếu và chỉ nếu

$$\nabla_M J(Y, M) = \nabla_M \sum_{n=1}^N \sum_{j=1}^K y_{n,j} \|x_n - m_j\|_2^2 = \sum_{n=1}^N \sum_{j=1}^K y_{n,j} \nabla_M \|x_n - m_j\|_2^2 = 0.$$

Tức là $\frac{\partial J}{\partial m_j} = 0$ với mọi $j = 1, 2, \dots, K$

$$\sum_{n=1}^N y_{n,j} 2(x_n - m_j) = 2 \sum_{y_{n,j}=1} (x_n - m_j) = 2 \sum_{x_n \in \text{Cluster}_j} (x_n - m_j) = 0$$

Đặt $N_j = |\text{Cluster}_j|$, đẳng thức cuối suy ra

$$\sum_{x_n \in \text{Cluster}_j} x_n = N_j m_j \iff m_j = \frac{1}{N_j} \sum_{x_n \in \text{Cluster}_j} x_n$$

m_j là trung bình cộng của các điểm trong cluster j.

Cụ thể thuật toán k-means hoạt động như sau[2]:

- Cho tập dữ liệu $\mathbf{X} = \{x_n\}_{n=1}^N$ và số cụm cần phân chia $0 < k < N$.
- **Khởi tạo:** Lấy k điểm dữ liệu khác nhau $c_1, c_2, \dots, c_k$ từ tập $\mathbf{X}$ làm các tâm cụm ban đầu. Tại bước lặp thứ t :
 - **Gán điểm dữ liệu vào cụm:**
 - * Với k tâm cụm từ bước trước, với mỗi điểm dữ liệu x_n tính các khoảng cách $d_{n,j} = \text{dist}(x_n, c_j)$ từ x_n đến tâm cụm hiện tại c_j .
 - * Giả sử I là chỉ số cụm có $d_{n,I} = \text{dist}(x_n, c_I)$ nhỏ nhất.
 - * Gán điểm x_n vào cụm thứ I , tức cụm có tâm hiện tại là c_I .
 - * Sau thao tác gán, dữ liệu $\mathbf{X}$ được chia vào các cụm $S_1, S_2, \dots, S_k$.
 - **Xác định lại tâm cụm:** Tâm cụm c_j được cập nhật bằng trung bình cộng các điểm trong cụm S_j :

$$c_j^{\text{new}} := \frac{\sum_{x_n \in S_j} x_n}{|S_j|}.$$

Gán $t \leftarrow t + 1$ và lặp lại.

- Thuật toán dừng khi các tâm cụm tìm được ở bước $t + 1$ hoàn toàn trùng với các tâm cụm tìm được ở bước t .

Một số lưu ý khi thực hiện thuật toán[2]

- **Số cụm k phù hợp nhất:** Tùy theo số điểm dữ liệu N . Số cụm k tối ưu thường được chọn bằng cách xét một dãy các giá trị của k và tìm giá trị k phù hợp nhất.
- **Chọn tâm cụm ở bước khởi tạo**
 - Không chọn các điểm khởi tạo quá gần nhau.
 - Chọn điểm c_1 bất kỳ.
 - Chọn c_2 ở xa c_1 nhất có thể (theo khoảng cách Euclide).
 - Chọn c_3 sao cho $\text{dist}(c_1, c_3) + \text{dist}(c_2, c_3)$ lớn nhất có thể.
 - Tiếp tục như vậy cho đến khi chọn đủ tâm cụm chỉ số k .

Tóm tắt Thuật toán K -means clustering[1]

Đầu vào: Ma trận dữ liệu $\mathbf{X} \in \mathbb{R}^{d \times N}$ và số lượng cụm cần tìm $K < N$.

Đầu ra: Ma trận centroid $\mathbf{M} \in \mathbb{R}^{d \times k}$ và ma trận nhãn $\mathbf{Y} \in \mathbb{R}^{N \times k}$.

1. Chọn K điểm bất kỳ trong *training set* làm các centroid ban đầu.
2. Phân mỗi điểm dữ liệu vào cluster có centroid gần nó nhất.
3. Nếu việc phân cụm ở Bước 2 không thay đổi so với vòng lặp trước, dừng thuật toán.
4. Cập nhật centroid cho từng cluster bằng cách lấy trung bình cộng của tất cả các điểm dữ liệu đã được gán vào cluster đó sau Bước 2.
5. Quay lại Bước 2.

Thuật toán K -means có ưu điểm là dễ triển khai, tốc độ xử lý nhanh và phù hợp với dữ liệu phân tách rõ ràng. Tuy nhiên, nó cũng có nhược điểm như phụ thuộc vào giá trị k , nhạy cảm với nhiễu, và có thể bị mắc kẹt ở cực trị cục bộ nếu khởi tạo tâm cụm không tốt.

4.4 Chỉ số đánh giá phân cụm [2]

4.4.1 Silhouette Score

- **Ý nghĩa:** Đo lường sự tương đồng của một điểm với cụm của chính nó so với cụm gần nhất khác. Giá trị nằm trong khoảng $[-1, 1]$. Giá trị càng cao nghĩa là phân cụm càng tốt.
- **Công thức:** Với mỗi điểm dữ liệu i :

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (4.1)$$

- $a(i)$: khoảng cách trung bình từ điểm i đến các điểm khác trong cùng cụm.
- $b(i)$: khoảng cách trung bình từ điểm i đến các điểm trong cụm gần nhất khác.

4.4.2 Davies–Bouldin Index (DBI)

- **Ý nghĩa:** Đo lường độ tương đồng giữa các cụm. Chỉ số càng nhỏ càng tốt.
- **Công thức:** Với số cụm C đã cho:

$$DBI = \frac{1}{C} \sum_{i=1}^C \max_{j \neq i} \left(\frac{\sigma_i + \sigma_j}{d_{ij}} \right) \quad (4.2)$$

- σ_i : độ rộng cụm i (trung bình khoảng cách từ điểm trong cụm đến tâm cụm).
- d_{ij} : khoảng cách giữa tâm cụm i và j .

4.4.3 Calinski–Harabasz Index (CH)

- **Ý nghĩa:** Tỷ lệ giữa phương sai giữa các cụm và trong cụm. Giá trị càng lớn càng tốt.
- **Công thức:** Với số cụm K và số điểm dữ liệu N :

$$CH = \frac{\text{Tr}(B_k)}{\text{Tr}(W_k)} \cdot \frac{N - K}{K - 1} \quad (4.3)$$

- $\text{Tr}(B_k)$: trace (tổng phần tử trên đường chéo) của ma trận phương sai giữa các cụm.
- $\text{Tr}(W_k)$: trace của ma trận phương sai trong cụm.

4.5 DBSCAN Clustering [3]

Khi biểu diễn các điểm dữ liệu trong không gian, ta thường thấy rằng các vùng có mật độ cao thường được xen kẽ bởi các vùng có mật độ thấp. Nếu sử dụng mật độ làm tiêu chí để phân chia, các tâm cụm có xu hướng tập trung tại các vùng mật độ cao, trong khi các vùng biên sẽ thuộc về những khu vực có mật độ thấp. Trong các mô hình phân cụm của học không giám sát, có một kỹ thuật được gọi là phân cụm dựa trên mật độ (Density-Based Spatial Clustering of Applications with Noise Clustering). Kỹ thuật này tập trung vào việc xác định các cụm riêng biệt trong phân phối dữ liệu, dựa trên quan niệm rằng một cụm trong không gian dữ liệu là một vùng có mật độ điểm cao, được phân cách với các cụm khác bằng các vùng liền kề có mật độ điểm thấp.

DBSCAN là thuật toán phân cụm dựa trên mật độ điểm, rất phổ biến trong học máy và khai phá dữ liệu. Nó có khả năng nhận diện các cụm với hình dạng và kích thước khác nhau từ tập dữ liệu lớn, ngay cả khi tập dữ liệu chứa nhiễu.

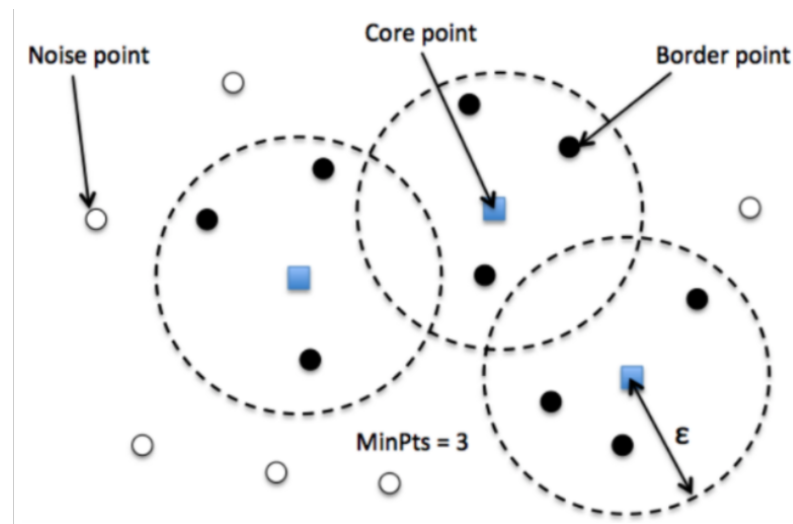
Căn cứ vào vị trí của các điểm dữ liệu so với cụm, chúng ta có thể chia chúng thành ba loại: Đối với các điểm nằm sâu bên trong cụm, chúng ta xem chúng là *điểm lõi*. Các *điểm biên* nằm ở phần ngoài cùng của cụm và *điểm nhiễu* không thuộc bất kỳ một cụm nào. Bên dưới là hình vẽ mô phỏng thể hiện ba loại điểm tương ứng nêu trên.

Trong thuật toán DBSCAN sử dụng hai tham số chính đó là:

- **minPts**: Là một ngưỡng số điểm dữ liệu tối thiểu được nhóm lại với nhau nhằm xác định một vùng lân cận **epsilon** có mật độ cao. Số lượng minPts không bao gồm điểm ở tâm.
- **epsilon (kí hiệu ϵ)**: Một giá trị khoảng cách được sử dụng để xác định vùng lân cận **epsilon** của bất kỳ điểm dữ liệu nào.

Hai tham số trên sẽ được sử dụng để xác định vùng lân cận epsilon và khả năng tiếp cận giữa các điểm dữ liệu lẫn nhau. Từ đó giúp kết nối chuỗi dữ liệu vào chung một cụm.

Hai tham số trên giúp xác định ba loại điểm:



Hình 4.3: Hình minh họa cách xác định ba loại điểm bao gồm: *điểm lõi* (*core*) chấm vuông màu xanh, *điểm biên* (*border*) chấm tròn màu đen, và *điểm nhiễu* (*noise*) chấm tròn màu trắng trong thuật toán DBSCAN. Các hình tròn đường viền nét đứt bán kính ϵ thể hiện vùng lân cận epsilon tương ứng với các *điểm lõi* nhằm xác định nhân cho từng điểm. $\text{minPts}=3$ là số lượng tối thiểu để một *điểm lõi* rơi vào vùng có mật độ cao nếu xung quanh chúng có số lượng điểm tối thiểu là 3. [3]

- **Điểm lõi (core):** Đây là một điểm có ít nhất **minPts** điểm trong vùng lân cận *epsilon* của chính nó.
- **Điểm biên (border):** Đây là một điểm có ít nhất một điểm lõi nằm ở vùng lân cận *epsilon* nhưng mật độ không đủ minPts điểm.
- **Điểm nhiễu (noise):** Đây là điểm không phải là điểm lõi hay điểm biên.

Đối với một cặp điểm (P, Q) bất kì sẽ có ba khả năng:

- Cả P và Q đều có khả năng *kết nối mật độ* được với nhau. Khi đó P, Q đều thuộc về chung một cụm.
- P có khả năng *kết nối mật độ* được với Q nhưng Q không *kết nối mật độ* được với P . Khi đó P sẽ là *điểm lõi* của cụm còn Q là một *điểm biên*.
- P và Q đều không *kết nối mật độ* được với nhau. Trường hợp này P và Q sẽ rơi vào những cụm khác nhau hoặc một trong hai điểm là *điểm nhiễu*.

Xác định tham số

Xác định tham số là một bước quan trọng và ảnh hưởng trực tiếp tới kết quả của các thuật toán. Đối với thuật toán **DBSCAN** cũng không ngoại lệ. Chúng ta cần phải xác định chính xác tham số cho thuật toán **DBSCAN** một cách phù hợp với từng bộ dữ liệu cụ thể, tùy theo đặc điểm và tính chất của phân phối của bộ dữ liệu. Hai tham số cần lựa chọn trong **DBSCAN** đó chính là *minPts* và *epsilon*:

- **minPts**: Theo quy tắc chung, *minPts* tối thiểu có thể được tính theo số chiều D trong tập dữ liệu đó là $minPts \geq D + 1$. Một giá trị $minPts = 1$ không có ý nghĩa, vì khi đó mỗi điểm bản thân nó đều là một cụm. Với $minPts \leq 2$, kết quả sẽ giống như **phân cụm phân cấp (hierarchical clustering)** với **single linkage** với biểu đồ **dendrogram** được cắt ở độ cao $y = epsilon$. Do đó, *minPts* phải được chọn ít nhất là 3. Tuy nhiên, các giá trị lớn hơn thường tốt hơn cho các tập dữ liệu có nhiều và kết quả phân cụm thường hợp lý hơn. Theo quy tắc chung thì tăng giá trị $minPts = 2 \times dim$. Trong trường hợp dữ liệu có nhiều hoặc có nhiều quan sát lặp lại thì cần lựa chọn giá trị *minPts* lớn hơn nữa tương ứng với những bộ dữ liệu lớn.
- **epsilon**: Giá trị ϵ có thể được chọn bằng cách vẽ một biểu đồ **k-distance**. Đây là biểu đồ thể hiện giá trị khoảng cách trong thuật toán **k-Means clustering** đến $k = minPts - 1$ điểm lân cận gần nhất. Ứng với mỗi điểm chúng ta chỉ lựa chọn ra khoảng cách lớn nhất trong k khoảng cách. Những khoảng cách này sẽ được sắp xếp theo thứ tự tăng dần. Các giá trị tốt của ϵ là vị trí mà biểu đồ thể hiện sự thay đổi xuất hiện một điểm **khuyết tay (elbow point)**. Nếu ϵ được chọn quá nhỏ, một phần lớn dữ liệu sẽ không được phân cụm và được xem là *nhiều*, trong khi đối với giá trị ϵ quá cao, các cụm sẽ hợp nhất và phần lớn các điểm sẽ nằm trong cùng một cụm. Nói chung, các giá trị nhỏ của ϵ được ưu tiên hơn và theo quy tắc chung, chỉ một phần nhỏ các điểm nên nằm trong vùng lân cận ϵ .
- **Hàm khoảng cách**: Việc lựa chọn hàm khoảng cách có mối liên hệ chặt chẽ với lựa chọn ϵ và tạo ra ảnh hưởng lớn tới kết quả. Điểm quan trọng trước tiên đó là chúng ta cần xác định một thước đo hợp lý về độ khác biệt (*dissimilarity*) cho tập dữ liệu trước khi có thể chọn tham số ϵ . Khoảng cách được sử dụng phổ biến nhất là **euclidean distance**.

Các bước trong thuật toán DBSCAN

Các bước của thuật toán DBSCAN khá đơn giản. Thuật toán sẽ thực hiện lan truyền để mở rộng dần phạm vi của cụm cho tới khi chạm tới những điểm biên thì thuật toán sẽ chuyển sang một cụm mới và lặp lại tiếp quá trình trên.

Qui trình của thuật toán:

- **Bước 1:** Thuật toán lựa chọn một điểm dữ liệu bất kì. Sau đó tiến hành xác định các *điểm lõi* và *điểm biên* thông qua vùng lân cận *epsilon* bằng cách lan truyền theo liên kết chuỗi các điểm thuộc cùng một cụm.
- **Bước 2:** Cụm hoàn toàn được xác định khi không thể mở rộng được thêm. Khi đó lặp lại đệ qui toàn bộ quá trình với điểm khởi tạo trong số các điểm dữ liệu còn lại để xác định một cụm mới.

Kết luận: DBSCAN là một thuật toán đơn giản và hiệu quả. Nó hoạt động dựa trên cách tiếp cận mật độ phân phối của dữ liệu.

Ưu điểm của thuật toán đó là tự động xác định số lượng cụm (không cần chỉ định trước như K-Means/GMM), có thể tự động loại bỏ được các điểm dữ liệu nhiễu, hoạt động tốt đối với những dữ liệu có hình dạng phân phối đặc thù, phức tạp và có tốc độ tính toán nhanh.

Tuy nhiên, DBSCAN thường không tốt nếu dữ liệu có mật độ không đồng đều (các cụm có mật độ khác nhau). Ngoài ra, DBSCAN còn tính toán khoảng cách cho tất cả các cặp điểm (nên chi phí có thể cao với dữ liệu lớn). Khi huấn luyện DBSCAN thì các tham số của mô hình như khoảng cách epsilon, số lượng điểm lân cận tối thiểu minPts và hàm khoảng cách là những tham số có ảnh hưởng rất lớn đối với kết quả phân cụm. Thực tế cho thấy thuật toán khá nhạy với tham số **epsilon** và **minPts** nên chúng ta cần phải lựa chọn tham số cho mô hình trước khi tiến hành xây dựng mô hình.

4.6 Chỉ số đánh giá phân cụm [2]

4.6.1 Silhouette Score

- **Ý nghĩa:** Đo lường sự tương đồng của một điểm với cụm của chính nó so với cụm gần nhất khác. Giá trị nằm trong khoảng $[-1, 1]$. Giá trị càng cao nghĩa là phân cụm càng tốt.
- **Công thức:** Với mỗi điểm dữ liệu i :

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (4.4)$$

- $a(i)$: khoảng cách trung bình từ điểm i đến các điểm khác trong cùng cụm.
- $b(i)$: khoảng cách trung bình từ điểm i đến các điểm trong cụm gần nhất khác.

4.6.2 Davies–Bouldin Index (DBI)

- **Ý nghĩa:** Đo lường độ tương đồng giữa các cụm. Chỉ số càng nhỏ càng tốt.
- **Công thức:** Với số cụm C đã cho:

$$DBI = \frac{1}{C} \sum_{i=1}^C \max_{j \neq i} \left(\frac{\sigma_i + \sigma_j}{d_{ij}} \right) \quad (4.5)$$

- σ_i : độ rộng cụm i (trung bình khoảng cách từ điểm trong cụm đến tâm cụm).
- d_{ij} : khoảng cách giữa tâm cụm i và j .

4.6.3 Calinski–Harabasz Index (CH)

- **Ý nghĩa:** Tỷ lệ giữa phương sai giữa các cụm và trong cụm. Giá trị càng lớn càng tốt.
- **Công thức:** Với số cụm K và số điểm dữ liệu N :

$$CH = \frac{\text{Tr}(B_k)}{\text{Tr}(W_k)} \cdot \frac{N - K}{K - 1} \quad (4.6)$$

- $\text{Tr}(B_k)$: trace (tổng phần tử trên đường chéo) của ma trận phương sai giữa các cụm.
- $\text{Tr}(W_k)$: trace của ma trận phương sai trong cụm.

4.7 Support Vector Machine (SVM)

Support Vector Machine (SVM) là một mô hình học máy có giám sát ban đầu dùng cho bài toán phân loại nhị phân. Mục tiêu chính của SVM là tìm siêu phẳng phân chia tối ưu giữa hai lớp dữ liệu sao cho margin (khoảng cách từ siêu phẳng đến các điểm gần nhất của hai lớp) là lớn nhất. Những điểm nằm sát siêu phẳng này được gọi là support vectors, và chúng đóng vai trò quyết định đến nghiệm của mô hình.

4.7.1 Hard Margin SVM

Bài toán phân loại nhị phân với *hard margin* — thường được gọi là *Thuật toán Siêu phẳng Tối ưu* (Optimal Hyperplane Algorithm) — ban đầu được phát triển cho trường hợp dữ liệu huấn luyện có thể phân tách tuyến tính hoàn toàn, không có lỗi. Mục tiêu của phương pháp này là tìm ra một hàm quyết định tuyến tính duy nhất sao cho khoảng cách phân tách giữa hai lớp là lớn nhất, từ đó đảm bảo khả năng tổng quát hóa cao.

Định nghĩa và Mục tiêu (Biên tối đa)

Mục tiêu của Hard Margin SVM là xây dựng *siêu phẳng tối ưu*:

$$w_0 \cdot x + b_0 = 0$$

- **Biên tối đa (Maximal Margin):** Siêu phẳng tối ưu là siêu phẳng duy nhất phân tách dữ liệu huấn luyện với biên lớn nhất.
- **Điều kiện phân tách:** Với tập dữ liệu huấn luyện (y_i, x_i) , $y_i \in \{-1, 1\}$, cần tồn tại một nghiệm thỏa mãn:

$$y_i(w \cdot x_i + b) \geq 1, \quad \forall i$$

- **Bài toán tối ưu:** Biên phân tách giữa hai lớp được xác định bởi:

$$\rho(w, b) = \frac{2}{\|w\|}$$

Do đó, việc tối đa hóa biên tương đương với việc tối thiểu hóa:

$$\min \frac{1}{2} w \cdot w$$

dưới các ràng buộc phân loại đúng với biên tối thiểu bằng 1.

Mô hình toán học và Hàm Lagrange

Bài toán Hard Margin SVM là một bài toán tối ưu có ràng buộc và được giải bằng *quy hoạch bậc hai*. Để chuyển từ bài toán nguyên thủy (primal) sang bài toán đối ngẫu (dual), ta xây dựng hàm Lagrange:

$$L(w, b, A) = \frac{1}{2}w \cdot w - \sum_{i=1}^{\ell} \alpha_i y_i (x_i \cdot w + b) - 1$$

trong đó $A = (\alpha_1, \dots, \alpha_\ell)$ là các hệ số Lagrange không âm.

Nghiệm tối ưu của bài toán đạt được tại *điểm yên ngựa* (saddle point), nơi hàm Lagrange:

- được **tối thiểu hóa** theo các biến nguyên thủy w và b ,
- và được **tối đa hóa** theo các hệ số Lagrange A .

Loại bỏ biến nguyên thủy và dạng nghiệm

Việc lấy đạo hàm riêng của hàm Lagrange cho phép loại bỏ các biến nguyên thủy:

- Theo w :

$$\frac{\partial L}{\partial w} = 0 \quad \Rightarrow \quad w_0 = \sum_{i=1}^{\ell} \alpha_i y_i x_i$$

cho thấy vector trọng số tối ưu là tổ hợp tuyến tính của các mẫu huấn luyện.

- Theo b :

$$\frac{\partial L}{\partial b} = 0 \quad \Rightarrow \quad \sum_{i=1}^{\ell} \alpha_i y_i = 0$$

Như vậy, chỉ các mẫu huấn luyện có $\alpha_i > 0$ mới ảnh hưởng đến nghiệm. Các mẫu này được gọi là *vector hỗ trợ* (support vectors) và nằm đúng trên biên:

$$y_i(x_i \cdot w_0 + b_0) = 1$$

Bài toán đối ngẫu và cách giải

Thay các biểu thức của w_0 và điều kiện tổng vào hàm Lagrange, ta thu được bài toán đối ngẫu:

$$W(A) = A^T \mathbf{1} - \frac{1}{2} A^T D A$$

trong đó ma trận D được xác định bởi:

$$D_{ij} = y_i y_j (x_i \cdot x_j)$$

Bài toán đối ngẫu là một bài toán quy hoạch bậc hai với các ràng buộc:

- $\mathbf{A} \geq 0$ (các hệ số Lagrange không âm),
- $A^T y = 0$ (ràng buộc tuyến tính).

Giá trị cực đại của $W(A)$ liên hệ trực tiếp với biên tối đa:

$$W(A_0) = \frac{2}{\rho_0^2}$$

Giải bài toán trên tập dữ liệu lớn. Với các tập dữ liệu lớn, việc giải trực tiếp bài toán đối ngẫu là tốn kém. Do đó, có thể sử dụng chiến lược lặp gia tăng:

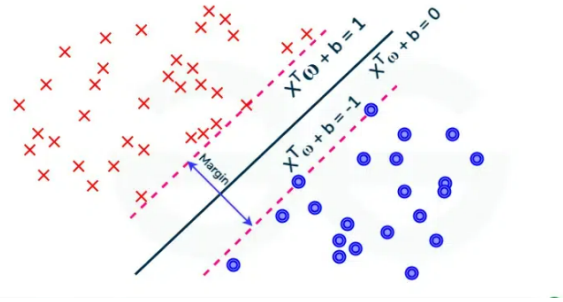
- Chia tập huấn luyện thành các phần nhỏ.
- Giải bài toán đối ngẫu trên phần đầu tiên.
- Xác định các vector hỗ trợ và kết hợp chúng với các mẫu vi phạm ràng buộc từ phần dữ liệu tiếp theo.
- Lặp lại quá trình cho đến khi $W(A)$ hội tụ và siêu phẳng tối ưu cho toàn bộ dữ liệu được tìm thấy.

Điều kiện Kuhn–Tucker và vector hỗ trợ. Theo điều kiện Kuhn–Tucker, một hệ số α_i chỉ có thể khác 0 khi:

$$y_i(x_i \cdot w_0 + b_0) - 1 = 0$$

Các điểm dữ liệu tương ứng chính là *vector hỗ trợ*, và chúng hoàn toàn quyết định biên phân tách tối ưu của Hard Margin SVM.

Nếu dữ liệu không thể phân tách tuyến tính, giá trị của $W(A)$ sẽ tăng không bị chặn, cho thấy không tồn tại nghiệm hard margin, và khi đó cần chuyển sang mô hình *Soft Margin SVM*.



Hình 4.4: Minh họa Hard Margin SVM

4.7.2 Soft Margin SVM

Trong thực tế, dữ liệu huấn luyện thường không thể phân tách tuyến tính hoàn hảo. *Soft Margin SVM* được giới thiệu để mở rộng Hard Margin SVM bằng cách cho phép một số vi phạm phân loại có kiểm soát, từ đó đạt được sự cân bằng tốt hơn giữa khả năng khớp dữ liệu và khả năng tổng quát hóa.

Biến slack và ràng buộc phân tách mềm

Để xử lý dữ liệu không phân tách tuyến tính, mỗi mẫu huấn luyện được gán một biến *slack* $\xi_i \geq 0$, dùng để đo mức độ vi phạm ràng buộc phân tách lý tưởng.

Ràng buộc hard margin:

$$y_i(w \cdot x_i + b) \geq 1$$

được nối lỏng thành ràng buộc soft margin:

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i$$

Ý nghĩa của biến slack:

- $0 < \xi_i \leq 1$: Mẫu được phân loại đúng nhưng nằm trong vùng biên.
- $\xi_i > 1$: Mẫu bị phân loại sai (nằm sai phía của siêu phẳng).

Nhờ các biến slack, mô hình cho phép một số lỗi huấn luyện nhưng vẫn duy trì cấu trúc biên phân tách hợp lý.

Hàm mục tiêu và vai trò của tham số C

Hàm mục tiêu của Soft Margin SVM được mở rộng thành:

$$\min_{w,b,\xi} \frac{1}{2}w \cdot w + C \sum_{i=1}^{\ell} \xi_i^{\sigma}$$

Trong đó:

- $\frac{1}{2}w \cdot w$: Tối đa hóa biên phân tách.
- $\sum \xi_i$: Tối thiểu hóa tổng mức độ vi phạm (lỗi huấn luyện).
- $C > 0$: Tham số đánh đổi giữa biên lớn và số lỗi nhỏ.

Vai trò của C :

- C lớn: Phạt mạnh lỗi huấn luyện, biên quyết định cứng hơn.
- C nhỏ: Cho phép nhiều vi phạm hơn để đạt biên rộng hơn.

Trong thực tế, lựa chọn phổ biến là $\sigma = 1$, giúp bài toán vẫn là quy hoạch bậc hai và có thể giải hiệu quả.

Bài toán đối ngẫu của Soft Margin SVM

Bài toán nguyên thủy được giải bằng cách xây dựng hàm Lagrange:

$$\begin{aligned} L(w, \xi, b, A, R) = & \frac{1}{2}w \cdot w + C \sum_{i=1}^{\ell} \xi_i - \sum_{i=1}^{\ell} \alpha_i [y_i(x_i \cdot w + b) - 1 + \xi_i] \\ & - \sum_{i=1}^{\ell} r_i \xi_i \end{aligned}$$

trong đó $\alpha_i \geq 0$ và $r_i \geq 0$ là các hệ số Lagrange.

Từ điều kiện tối ưu:

- $w_0 = \sum_{i=1}^{\ell} \alpha_i y_i x_i$
- $\sum_{i=1}^{\ell} \alpha_i y_i = 0$
- $C - \alpha_i - r_i = 0$

Ta thu được bài toán đối ngẫu:

$$W(A) = A^T \mathbf{1} - \frac{1}{2} A^T D A$$

với:

$$D_{ij} = y_i y_j (x_i \cdot x_j)$$

Ràng buộc hộp (Box Constraint). Khác với Hard Margin SVM, Soft Margin SVM có ràng buộc:

$$0 \leq \alpha_i \leq C$$

Giới hạn trên này đảm bảo rằng không có một điểm ngoại lai nào chi phối hoàn toàn nghiệm tối ưu.

Diễn giải các hệ số đối ngẫu

Các hệ số α_i cho biết vai trò của từng điểm dữ liệu:

- $\alpha_i = 0$: Điểm nằm ngoài biên, không ảnh hưởng đến siêu phẳng.
- $0 < \alpha_i < C$: Vector hỗ trợ nằm trên biên.
- $\alpha_i = C$: Điểm nằm trong biên hoặc bị phân loại sai.

Các vector hỗ trợ (support vectors) là những điểm duy nhất quyết định biên phân tách.

Hội tụ về Hard Margin khi dữ liệu phân tách tuyến tính

Khi dữ liệu huấn luyện phân tách tuyến tính và tham số C đủ lớn, nghiệm của Soft Margin SVM trùng hoàn toàn với nghiệm Hard Margin SVM.

Nguyên nhân:

- Với C rất lớn, mọi $\xi_i > 0$ đều gây ra chi phí cực lớn.
- Thuật toán buộc phải chọn nghiệm $\xi_i = 0 \forall i$.
- Khi đó, ràng buộc soft margin trở thành ràng buộc hard margin.

Hàm mục tiêu khi đó rút gọn thành:

$$\min \frac{1}{2} w \cdot w$$

dẫn đến cùng siêu phẳng tối ưu (w_0, b_0) như trong Hard Margin SVM.

Cơ chế này phản ánh nguyên lý *Occam–Razor*: khi không cần thiết phải cho phép lỗi, mô hình sẽ chọn lời giải đơn giản nhất với biên lớn nhất.

4.7.3 Multiclass Support Vector Machines

Support Vector Machines (SVM) ban đầu được thiết kế cho các bài toán phân loại nhị phân với hai lớp. Tuy nhiên, trong nhiều ứng dụng thực tế như nhận dạng chữ viết tay hay phân loại ảnh, dữ liệu thường bao gồm nhiều hơn hai lớp. Do đó, SVM đã được mở rộng để xử lý bài toán đa lớp thông qua nhiều chiến lược khác nhau. Các phương pháp này có thể được chia thành hai nhóm chính: *phân rã thành các bài toán nhị phân* và *mô hình tối ưu hóa đơn*.

Phân rã bài toán đa lớp thành các bài toán nhị phân

Cách tiếp cận phổ biến nhất để mở rộng SVM cho bài toán đa lớp là chia bài toán c lớp thành nhiều bài toán nhị phân độc lập.

One-versus-Rest (OvR). Chiến lược One-versus-Rest xây dựng c mô hình SVM nhị phân. Với mỗi mô hình thứ j , các mẫu thuộc lớp j được xem là lớp dương, trong khi tất cả các lớp còn lại được gộp thành lớp âm. Khi dự đoán, mẫu mới được gán vào lớp có giá trị hàm quyết định lớn nhất.

Ưu điểm của OvR là số lượng mô hình cần huấn luyện chỉ bằng số lớp. Tuy nhiên, nhược điểm chính là sự mất cân bằng dữ liệu nghiêm trọng trong mỗi bài toán con, do lớp âm thường lớn hơn rất nhiều so với lớp dương, điều này có thể làm giảm hiệu năng phân loại.

One-versus-One (OvO). Trong chiến lược One-versus-One, một mô hình SVM nhị phân được huấn luyện cho mỗi cặp lớp khác nhau, dẫn đến tổng cộng $c(c - 1)/2$ mô hình. Mỗi mô hình chỉ sử dụng dữ liệu của hai lớp tương ứng. Khi dự đoán, mỗi bộ phân loại bỏ phiếu cho một lớp, và lớp nhận được nhiều phiếu nhất sẽ là kết quả cuối cùng.

Phương pháp này thường cho độ chính xác cao do từng bài toán con cân bằng hơn, nhưng chi phí tính toán và bộ nhớ rất lớn khi số lớp tăng.

Các mô hình tối ưu hóa đơn cho đa lớp

Thay vì phân rã bài toán, một hướng tiếp cận khác là xây dựng trực tiếp mô hình SVM đa lớp trong một bài toán tối ưu hóa duy nhất.

Mô hình của Weston và Crammer. Các mô hình này mở rộng SVM bằng cách đưa ra các ràng buộc rủi ro và chuẩn hóa đặc biệt (ví dụ: chuẩn Frobenius) để tối ưu hóa đồng thời tất cả các lớp trong một hàm mục tiêu chung. Cách tiếp

cận này có nền tảng lý thuyết vững chắc nhưng việc triển khai và giải bài toán thường phức tạp.

Tối đa hóa biên đa lớp (Maximizing Multi-Class Margins – MMM). Phương pháp MMM xây dựng c vector trọng số $(w_1, \dots, w_c)$, mỗi vector đại diện cho một lớp. Bộ phân loại giữa hai lớp j và k được xác định bởi hiệu $w_j - w_k$. Mục tiêu là tối đa hóa biên phân tách giữa mọi cặp lớp, đồng thời tránh được sự mất cân bằng dữ liệu của OvR và số lượng mô hình lớn của OvO.

Quy tắc quyết định và kiểm tra Trong hầu hết các mô hình SVM đa lớp, việc phân loại một mẫu mới x_i dựa trên giá trị của các hàm quyết định. Với các phương pháp OvR và MMM, lớp dự đoán được xác định bởi:

$$\tilde{y}_i = \arg \max_j (w_j^T x_i + b_j)$$

Quy tắc này chỉ yêu cầu tính toán c hàm quyết định trong giai đoạn kiểm tra, hiệu quả hơn đáng kể so với chiến lược OvO, vốn có thể cần đến hàng trăm hoặc hàng nghìn phép so sánh khi số lớp lớn.

Trực quan hóa. Có thể hình dung các chiến lược đa lớp như một giải đấu thể thao:

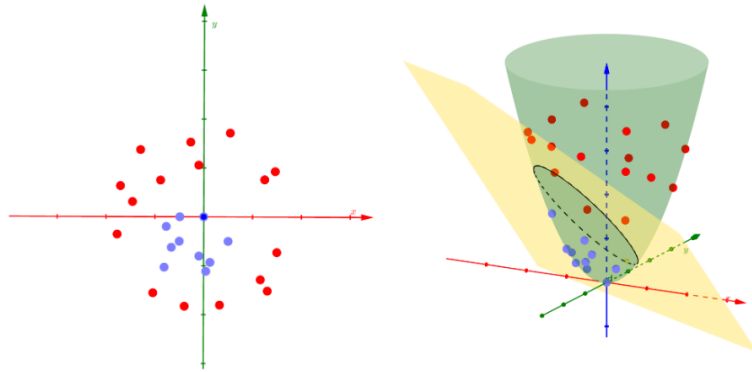
- *One-versus-Rest* giống như mỗi đội thi đấu với một “siêu đội” gồm tất cả các đội còn lại, thường không công bằng.
- *One-versus-One* giống như một giải đấu vòng tròn, nơi mọi đội đều gặp nhau; rất công bằng nhưng tốn thời gian.
- *Maximizing Multi-Class Margins* giống như việc gán cho mỗi đội một “chỉ số sức mạnh” và so sánh các chỉ số này để quyết định kết quả, vừa hiệu quả vừa phản ánh đúng năng lực tương đối của các đội.

Ưu và nhược điểm của SVM

- **Ưu điểm:**
 - Tối ưu margin, cho ranh giới phân chia rõ ràng và tốt.
 - Có nghiệm toàn cục (bài toán lồi).
 - Có thể mở rộng phi tuyến qua kernel.
 - Mô hình thưa, chỉ phụ thuộc vào support vectors.

- **Nhược điểm:**

- Không hiệu quả với dữ liệu rất lớn (nhiều mẫu).
- Nhạy cảm với dữ liệu nhiễu.
- Việc chọn kernel và tham số là rất quan trọng.
- Khó mở rộng trực tiếp sang phân loại đa lớp nếu không kết hợp thêm chiến lược như One-vs-One hoặc One-vs-Rest.



Hình 4.5: Kernel biến đổi dữ liệu ở không gian ban đầu sang 1 không gian mới

TIỀN XỬ LÝ DỮ LIỆU VÀ PHÂN TÍCH DỮ LIỆU

5.1 Giới thiệu

Tiền xử lý dữ liệu (*Data Preprocessing*) là giai đoạn đầu tiên và quan trọng nhất trong quy trình phân tích dữ liệu và học máy. Mục tiêu chính là:

- Hiểu rõ cấu trúc, ý nghĩa và chất lượng dữ liệu.
- Làm sạch, chuẩn hóa và biến đổi dữ liệu để phù hợp cho mô hình.
- Giảm nhiễu, xử lý dữ liệu thiếu, trùng lặp hoặc bất thường.

5.2 Đọc hiểu dữ liệu (Data Understanding)

- Kiểm tra kích thước dữ liệu: số dòng (samples), số cột (features).
- Xác định kiểu dữ liệu: số (*numeric*), chuỗi (*string*), danh mục (*categorical*).
- Thống kê mô tả cơ bản:

mean, std, min, max, percentile

- Kiểm tra phân phối và sự mất cân bằng của biến mục tiêu (target).

5.3 Phân tích thành phần và miền giá trị (Feature Domain Analysis)

- Xác định loại biến: định lượng, định tính, nhị phân, thứ tự, v.v.
- Kiểm tra miền giá trị hợp lệ (domain): giá trị nhỏ nhất, lớn nhất, phạm vi hợp lệ.
- Phát hiện giá trị hiếm, sai phạm, hoặc nằm ngoài phạm vi nghiệp vụ.

5.4 Làm sạch dữ liệu (Data Cleaning)

5.4.1 Xử lý giá trị khuyết (Missing Values)

- Tính tỷ lệ thiếu dữ liệu:

$$\text{Missing Rate} = \frac{\text{số giá trị thiếu}}{\text{tổng số quan sát}}$$

- Chiến lược xử lý:
 - Loại bỏ cột/dòng có quá nhiều giá trị khuyết.
 - Điền trung bình, trung vị, hoặc mode.
 - Dự đoán giá trị thiếu bằng mô hình thống kê.

5.4.2 Xử lý trùng lặp (Duplicates)

- Phát hiện dòng trùng lặp hoàn toàn hoặc gần giống.
- Giữ lại bản ghi duy nhất, loại bỏ bản sao.

5.4.3 Chuẩn hóa và mã hóa dữ liệu

- Chuẩn hóa dữ liệu số: z-score, min-max scaling, log-transform.
- Mã hóa dữ liệu phân loại:
 - One-hot encoding.
 - Label encoding.

5.5 Phát hiện và xử lý ngoại lệ (Outlier Detection)

- Quy tắc 3-sigma:

$$|x - \mu| > 3\sigma$$

- Quy tắc IQR:

$$\text{IQR} = Q_3 - Q_1, \quad x < Q_1 - 1.5 \times \text{IQR} \text{ hoặc } x > Q_3 + 1.5 \times \text{IQR}$$

- Phương pháp mô hình: Isolation Forest, LOF, DBSCAN.

5.6 Trực quan hóa dữ liệu (Data Visualization)

- Histogram: hiển thị phân phối giá trị.
- Boxplot: nhận diện ngoại lệ.
- Pairplot/Scatter matrix: quan sát mối quan hệ giữa các đặc trưng.
- Heatmap: hiển thị ma trận tương quan giữa các biến.

5.7 Phân tích quan hệ giữa đặc trưng và mục tiêu (Feature–Target Relationship)

- Hồi quy: dùng Pearson, Spearman hoặc kiểm định t.
- Phân loại: dùng ANOVA hoặc Chi-square để đo ảnh hưởng của đặc trưng đến target.
- Trực quan hóa bằng violin plot, boxplot hoặc bar chart.

5.8 Phân tích tương quan (Correlation Analysis)

- Ma trận tương quan:

$$r_{ij} = \frac{\text{Cov}(X_i, X_j)}{\sigma_i \sigma_j}$$

- $r_{ij} \in [-1, 1]$: giá trị gần 1 là tương quan mạnh.
- Loại bỏ các đặc trưng tương quan cao để tránh đa cộng tuyến.

5.9 Chuẩn hóa và Biến đổi Dữ liệu (Data Transformation)

5.9.1 Scaling

- Standardization:

$$x' = \frac{x - \mu}{\sigma}$$

- Min–Max Scaling:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

- Robust Scaling: dùng trung vị và IQR.

5.9.2 Biến đổi đặc trưng (Feature Engineering)

- Tạo đặc trưng tổng hợp, tỉ lệ, hoặc chênh lệch giữa các cột.
- Trích xuất thuộc tính thời gian (ngày, tháng, năm, mùa).
- Sinh biến nhị phân dựa theo điều kiện nghiệp vụ.

5.9.3 Giảm chiều dữ liệu

- PCA, t-SNE, UMAP cho trực quan hóa.
- Feature selection dựa trên thông tin tương hỗ (Mutual Information).

5.10 Phân tích Thống kê Mô tả (Descriptive Statistical Analysis)

- Kiểm tra độ lệch (skewness) và độ nhọn (kurtosis).
- Kiểm định phân phối chuẩn (Shapiro–Wilk, KS test).
- Thống kê trung bình, trung vị, tứ phân vị theo từng nhóm dữ liệu.

5.11 Đánh giá Chất lượng Dữ liệu (Data Quality Assessment)

5.11.1 Phát hiện dữ liệu không hợp lệ

- Kiểm tra logic (ví dụ: tuổi > 0).
- Đối chiếu domain nghiệp vụ.

5.11.2 Phát hiện inconsistency

- Chuẩn hóa định dạng chuỗi, xóa ký tự đặc biệt.
- Gộp các giá trị trùng nghĩa (“Hà Nội”, “Ha Noi”).

5.11.3 Gắn cờ dữ liệu nghi ngờ

- Tạo cột `is_suspect = 1` cho quan sát có vấn đề.

5.12 Phân tích Mối quan hệ Phức tạp (Complex Relationship Analysis)

- Sử dụng Spearman hoặc Kendall cho tương quan phi tuyến.
- Trực quan hóa scatter + đường hồi quy.
- Phân tích đa biến (multivariate): heatmap, cluster analysis, factor analysis.

5.13 Chuẩn bị Dữ liệu cho Mô hình (Model Readiness)

5.13.1 Xử lý dữ liệu mất cân bằng

- Oversampling (SMOTE), undersampling.
- Gán trọng số lớp (class weighting).

5.13.2 Chia dữ liệu

- Chia train/test theo tỷ lệ (70/30 hoặc 80/20).
- Dùng stratified sampling để bảo toàn phân phối lớp.
- Áp dụng cross-validation để đánh giá ổn định.

5.13.3 Kiểm tra rò rỉ dữ liệu (Data Leakage)

- Đảm bảo dữ liệu test không bị ảnh hưởng bởi tiền xử lý hoặc chọn đặc trưng.

5.14 Trực quan hóa Nâng cao (Advanced Visualization)

- Heatmap có hiển thị giá trị tương quan.
- Pairplot phân lớp theo target.
- Violinplot, swarmplot để so sánh phân phối.
- Biểu đồ phân phối so sánh giữa các nhóm dữ liệu.

5.15 Kết luận và Đề xuất

- Tổng hợp các vấn đề dữ liệu phát hiện được trong quá trình phân tích.
- Đề xuất hướng xử lý bổ sung: thu thập thêm dữ liệu, chọn đặc trưng, loại bỏ nhiễu.
- Cung cấp insight hỗ trợ xây dựng và huấn luyện mô hình học máy hiệu quả hơn.

5.16 Thực hiện trên bộ dữ liệu Alzheimer

5.16.1 Thông tin về bộ dữ liệu

Bộ dữ liệu được sử dụng trong dự án bao gồm thông tin về bệnh nhân, yếu tố lối sống, tiền sử y tế, các chỉ số lâm sàng, đánh giá nhận thức – chức năng và triệu chứng liên quan đến bệnh Alzheimer. Cụ thể như sau:

Thông tin bệnh nhân

- **PatientID:** Mã định danh duy nhất được gán cho từng bệnh nhân (từ 4751 đến 6900).

Thông tin nhân khẩu học

- **Tuổi (Age):** Độ tuổi của bệnh nhân, dao động từ 60 đến 90.
- **Giới tính (Gender):** 0 biểu thị Nam, 1 biểu thị Nữ.
- **Chủng tộc (Ethnicity):**
 - 0: Người da trắng (Caucasian)
 - 1: Người Mỹ gốc Phi (African American)
 - 2: Người châu Á (Asian)
 - 3: Khác (Other)
- **Trình độ học vấn (EducationLevel):**
 - 0: Không có
 - 1: Trung học phổ thông
 - 2: Cử nhân
 - 3: Sau đại học

Yếu tố lối sống

- **BMI:** Chỉ số khối cơ thể, từ 15 đến 40.
- **Hút thuốc (Smoking):** 0 là Không, 1 là Có.
- **Uống rượu (AlcoholConsumption):** Lượng tiêu thụ rượu hàng tuần (đơn vị), từ 0 đến 20.
- **Hoạt động thể chất (PhysicalActivity):** Số giờ hoạt động thể chất mỗi tuần, từ 0 đến 10.
- **Chất lượng chế độ ăn (DietQuality):** Điểm đánh giá chất lượng chế độ ăn, từ 0 đến 10.
- **Chất lượng giấc ngủ (SleepQuality):** Điểm đánh giá chất lượng giấc ngủ, từ 4 đến 10.

Tiền sử y tế

- **FamilyHistoryAlzheimers:** Tiền sử gia đình mắc bệnh Alzheimer (0: Không, 1: Có).
- **CardiovascularDisease:** Bệnh tim mạch (0: Không, 1: Có).
- **Diabetes:** Bệnh tiểu đường (0: Không, 1: Có).
- **Depression:** Trầm cảm (0: Không, 1: Có).
- **HeadInjury:** Tiền sử chấn thương đầu (0: Không, 1: Có).
- **Hypertension:** Tăng huyết áp (0: Không, 1: Có).

Chỉ số lâm sàng

- **SystolicBP:** Huyết áp tâm thu (90–180 mmHg).
- **DiastolicBP:** Huyết áp tâm trương (60–120 mmHg).
- **CholesterolTotal:** Tổng lượng cholesterol (150–300 mg/dL).
- **CholesterolLDL:** Mức cholesterol LDL (50–200 mg/dL).
- **CholesterolHDL:** Mức cholesterol HDL (20–100 mg/dL).
- **CholesterolTriglycerides:** Mức triglyceride (50–400 mg/dL).

Đánh giá nhận thức và chức năng

- **MMSE:** Điểm kiểm tra Mini-Mental State Examination (0–30). Điểm thấp hơn cho thấy suy giảm nhận thức.
- **FunctionalAssessment:** Điểm đánh giá chức năng (0–10). Điểm thấp hơn thể hiện suy giảm nghiêm trọng hơn.
- **MemoryComplaints:** Có phàn nàn về trí nhớ hay không (0: Không, 1: Có).
- **BehavioralProblems:** Có vấn đề hành vi hay không (0: Không, 1: Có).
- **ADL:** Điểm đánh giá hoạt động sinh hoạt hàng ngày (0–10). Điểm thấp hơn biểu thị suy giảm chức năng.

Triệu chứng

- **Confusion:** Có tình trạng lú lẫn (0: Không, 1: Có).
- **Disorientation:** Có tình trạng mất định hướng (0: Không, 1: Có).
- **PersonalityChanges:** Có thay đổi tính cách (0: Không, 1: Có).
- **DifficultyCompletingTasks:** Khó khăn khi hoàn thành công việc (0: Không, 1: Có).
- **Forgetfulness:** Hay quên (0: Không, 1: Có).

Thông tin chẩn đoán

- **Diagnosis:** Tình trạng chẩn đoán bệnh Alzheimer (0: Không, 1: Có).

Thông tin bảo mật

- **DoctorInCharge:** Thông tin bí mật về bác sĩ phụ trách, với giá trị mặc định là “XXXConfid” cho tất cả bệnh nhân.

5.16.2 Tiền xử lý và khám phá dữ liệu

Trong phần này, quá trình tiền xử lý và phân tích khám phá dữ liệu (EDA) được thực hiện nhằm hiểu rõ hơn về đặc điểm của bộ dữ liệu, phát hiện các giá trị bất thường, mối tương quan giữa các biến, và mối liên hệ với biến mục tiêu (chẩn đoán bệnh Alzheimer).

1. Tìm hiểu và làm sạch dữ liệu

Bộ dữ liệu được kiểm tra để phát hiện các giá trị bị thiếu, giá trị ngoại lai, và định dạng không hợp lệ.

- Các giá trị đều là số.
- **Giá trị thiếu/ lặp:** Không.

```
Number of duplicate rows: 0
```

```
Number of missing values per column:
```

Age	0
Gender	0
Ethnicity	0
EducationLevel	0
BMI	0
Smoking	0
AlcoholConsumption	0
PhysicalActivity	0
DietQuality	0
SleepQuality	0
FamilyHistoryAlzheimers	0
CardiovascularDisease	0
Diabetes	0
Depression	0
HeadInjury	0
Hypertension	0
SystolicBP	0
DiastolicBP	0
CholesterolTotal	0
CholesterolLDL	0
CholesterolHDL	0
CholesterolTriglycerides	0
MMSE	0
FunctionalAssessment	0
MemoryComplaints	0
BehavioralProblems	0
ADL	0
Confusion	0

```

Disorientation          0
PersonalityChanges      0
DifficultyCompletingTasks 0
Forgetfulness           0
Diagnosis               0
dtype: int64

```

- **Giá trị ngoại lai:** Không.

```

Number of outliers per numerical column (IQR method):
Age: 0
BMI: 0
AlcoholConsumption: 0
PhysicalActivity: 0
DietQuality: 0
SleepQuality: 0
SystolicBP: 0
DiastolicBP: 0
CholesterolTotal: 0
CholesterolLDL: 0
CholesterolHDL: 0
CholesterolTriglycerides: 0
MMSE: 0
FunctionalAssessment: 0
ADL: 0

```

- **Các biến phân loại:**

BehavioralProblems, CardiovascularDisease, Confusion, Depression, Diabetes, DifficultyCompletingTasks, Disorientation, EducationLevel, Ethnicity, FamilyHistoryAlzheimers, Forgetfulness, Gender, HeadInjury, Hypertension, MemoryComplaints, PersonalityChanges, Smoking.

Trong đó, ngoại trừ EducationLevel và Ethnicity, các biến đều là nhị phân.

- **Các biến liên tục:**

'Age', 'BMI', 'AlcoholConsumption', 'PhysicalActivity',

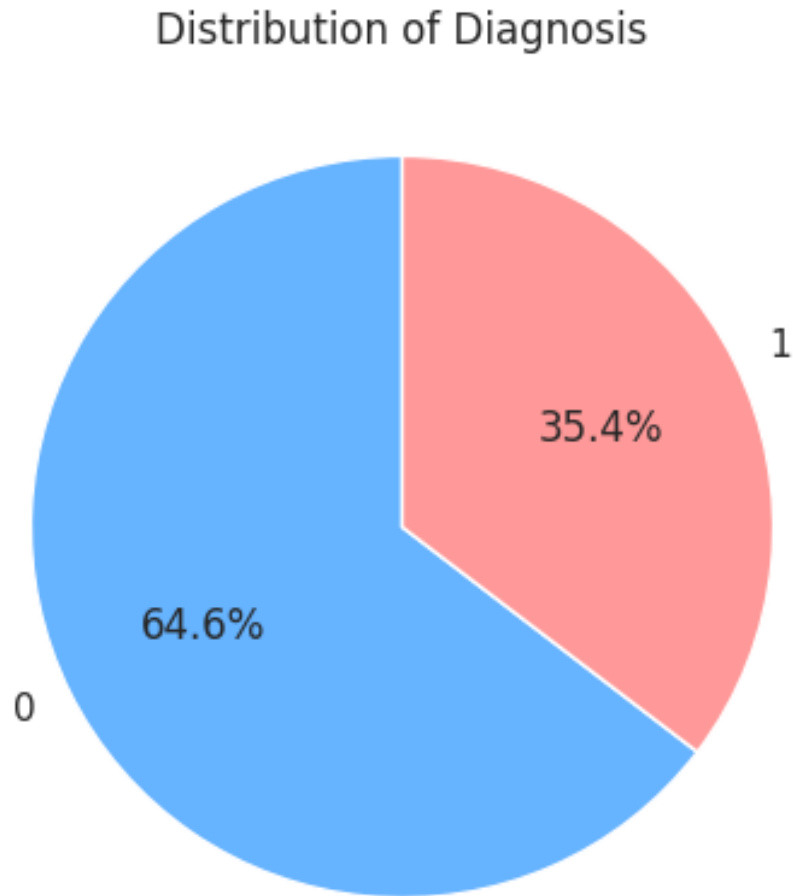
'DietQuality', 'SleepQuality', 'SystolicBP', 'DiastolicBP',
'CholesterolTotal', 'CholesterolLDL', 'CholesterolHDL',
'CholesterolTriglycerides', 'MMSE', 'FunctionalAssessment', 'ADL'.

- Chúng ta sẽ bỏ 2 cột là PatientID và DoctorInCharge.

2. Trực quan hóa phân bố dữ liệu

Phân bố của các đặc trưng chính được biểu diễn bằng biểu đồ cột, boxplot, pieplot để quan sát xu hướng tổng thể.

Với biến mục tiêu: Số quan sát của lớp 0 gần gấp đôi lớp 1.



Hình 5.1: Phân bố của biến mục tiêu.

Với các biến phân loại:

- Vấn đề sức khỏe/ tiền sử bệnh gia đình: 'No' chiếm phần lớn;
- Dân tộc: đa phần Caucasian;
- Học vấn: trung học chiếm nhiều nhất, tiếp theo là cử nhân;
- Giới tính: tỉ lệ nam-nữ rất cân bằng.

Với các biến liên tục:

- Đa phần các biến đều có phân bố khá đều (density line đường thẳng);
- MMSE có hai đỉnh khá rõ, biểu thị rằng trong tập dữ liệu tồn tại hai nhóm bệnh nhân có mức điểm MMSE khác biệt — một nhóm với điểm thấp hơn (suy giảm nhận thức) và một nhóm với điểm cao hơn (nhận thức bình thường).

3. Biểu diễn đặc trưng theo biến mục tiêu

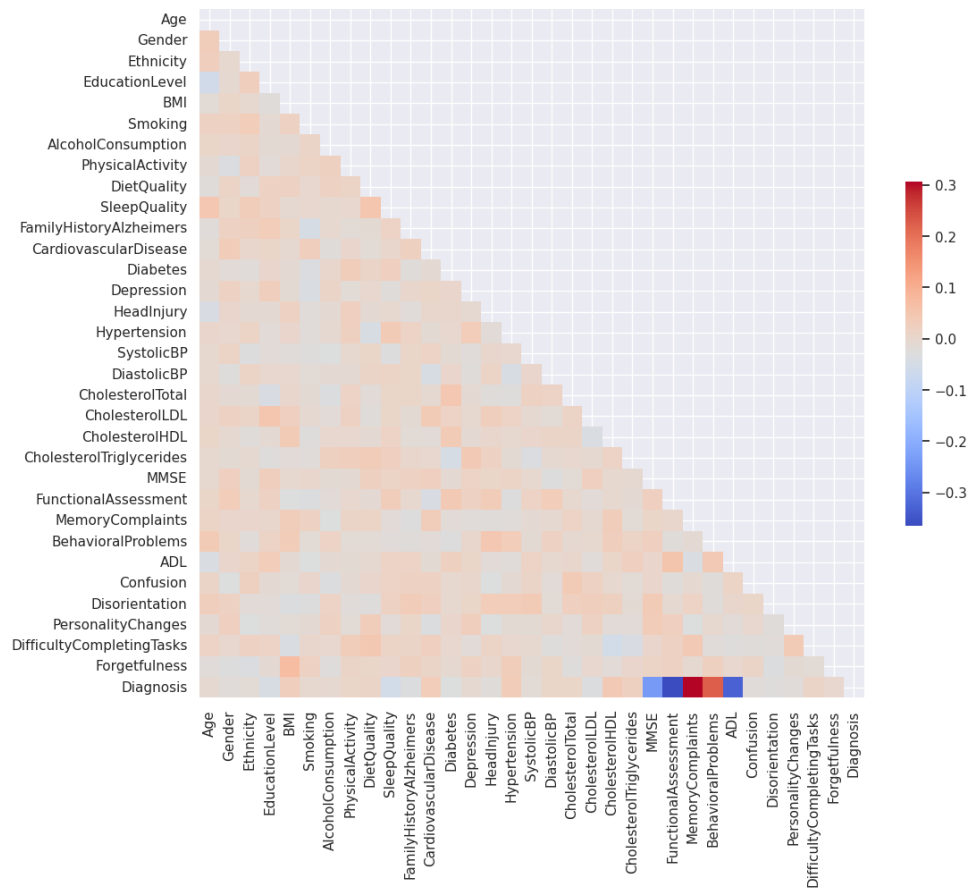
Các đặc trưng được biểu diễn so với biến mục tiêu (*Diagnosis*) để kiểm tra sự khác biệt giữa nhóm mắc Alzheimer và nhóm không mắc.

Với các biến liên tục:

- Tỉ lệ giữa class 0 và class 1 ko chênh lệch nhiều giữa các trường của đa số các biến (Gender, Confusion, Depression, Diabetes,...)
- Với BehavioralProblems, Ethnicity, EducationLevel, MemoryComplaints: có sự chênh lệch rõ ràng hơn.

4. Phân tích tương quan

Hệ số tương quan Pearson được tính để đánh giá mối liên hệ giữa các biến số.



Hình 5.2: Ma trận tương quan giữa các biến số.

- MMSE, Functional Assessment, Memory Complaints, Behavioral Problems, ADL có tương quan đáng kể nhất.
- Các biến khác đều có tương quan rất gần .
- Chúng ta cũng thấy xu hướng này trong các plots phía trên.

Kết luận: Quá trình tiền xử lý và phân tích khám phá giúp hiểu rõ hơn cấu trúc dữ liệu, mối liên hệ giữa các đặc trưng và biến mục tiêu, đồng thời hỗ trợ việc lựa chọn đặc trưng phù hợp cho các mô hình học máy ở bước tiếp theo.

GIẢM CHIỀU VÀ TRỰC QUAN

6.1 Phân tích Thành phần Chính (PCA)

6.1.1 Mục tiêu của PCA

Giả sử ta có tập dữ liệu gồm n điểm dữ liệu, mỗi điểm là một vector trong không gian $\mathbb{R}^p$, tức là có p đặc trưng. Mục tiêu của PCA (Principal Component Analysis) là:

- Tìm hệ trục tọa độ mới (*thành phần chính*) sao cho:
 - Các trục này **vuông góc** với nhau.
 - Khi chiếu dữ liệu lên các trục này, **phương sai của dữ liệu là lớn nhất**.
- Giữ lại $k < p$ trục đầu tiên để giảm chiều dữ liệu mà vẫn bảo toàn phần lớn thông tin.

6.1.2 Cơ sở toán học của PCA

Chuẩn hóa dữ liệu

Trước hết, dữ liệu được chuẩn hóa sao cho trung bình của mỗi đặc trưng bằng 0:

$$\mathbf{X}_{centered} = \mathbf{X} - \bar{\mathbf{X}}$$

Ma trận hiệp phương sai

$$\mathbf{C} = \frac{1}{n-1} \mathbf{X}_{centered}^T \mathbf{X}_{centered}$$

Trị riêng và vector riêng

Giải:

$$\mathbf{C} \mathbf{v}_k = \lambda_k \mathbf{v}_k$$

λ_k : trị riêng; $\mathbf{v}_k$: vector riêng (thành phần chính).

Chiều dữ liệu

$$\mathbf{Z} = \mathbf{X}_{centered} \mathbf{W}, \quad \mathbf{W} = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k]$$

6.1.3 Diễn giải kết quả PCA

- PC1: hướng có phương sai lớn nhất.
- PC2: vuông góc với PC1 và giữ phương sai còn lại.
- Phương sai giữ lại:

$$\text{Explained Variance Ratio} = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^p \lambda_i}$$

6.2 Phân tích Phân biệt Tuyến tính (LDA)

6.2.1 Mục tiêu của LDA

LDA (*Linear Discriminant Analysis*) là phương pháp giảm chiều có giám sát, tối đa hóa sự phân tách giữa các lớp.

- PCA: không giám sát (unsupervised), tối đa hóa phương sai.
- LDA: có giám sát (supervised), tối đa hóa khả năng phân biệt lớp.

6.2.2 Cơ sở toán học của LDA

Giả sử có c lớp với trung bình lớp $\boldsymbol{\mu}_i$ và trung bình toàn cục $\boldsymbol{\mu}$.

Ma trận tán xạ

- Trong lớp:

$$\mathbf{S}_W = \sum_{i=1}^c \sum_{\mathbf{x} \in C_i} (\mathbf{x} - \boldsymbol{\mu}_i)(\mathbf{x} - \boldsymbol{\mu}_i)^\top$$

- Giữa các lớp:

$$\mathbf{S}_B = \sum_{i=1}^c N_i (\boldsymbol{\mu}_i - \boldsymbol{\mu})(\boldsymbol{\mu}_i - \boldsymbol{\mu})^\top$$

Bài toán tối ưu

$$J(\mathbf{W}) = \frac{|\mathbf{W}^\top \mathbf{S}_B \mathbf{W}|}{|\mathbf{W}^\top \mathbf{S}_W \mathbf{W}|}$$

Giải bài toán trị riêng tổng quát:

$$\mathbf{S}_B \mathbf{w}_i = \lambda_i \mathbf{S}_W \mathbf{w}_i$$

6.2.3 So sánh PCA và LDA

Đặc điểm	PCA	LDA
Loại phương pháp	Không giám sát	Có giám sát
Mục tiêu	Tối đa hóa phương sai	Tối đa hóa phân tách giữa các lớp
Sử dụng nhãn lớp	Không	Có
Dạng bài toán	Trị riêng chuẩn	Trị riêng tổng quát
Số chiều tối đa sau chiếu	p	$c - 1$ (với c là số lớp)

6.2.4 Kết luận

LDA hữu ích khi dữ liệu có nhãn lớp rõ ràng, giúp không gian đặc trưng mới tách biệt tốt hơn giữa các lớp.

6.3 Giảm chiều bằng UMAP (Uniform Manifold Approximation and Projection)

6.3.1 Giới thiệu

UMAP là phương pháp giảm chiều dữ liệu phi tuyến hiện đại, dựa trên lý thuyết hình học và tô pô. Nó đặc biệt hiệu quả trong trực quan hóa dữ liệu có chiều cao (high-dimensional data) và thường được dùng thay thế t-SNE.

6.3.2 Nguyên lý hoạt động

UMAP giả định dữ liệu nằm trên một **đa tạp Riemann** (Riemannian manifold). Mục tiêu của UMAP:

- Xây dựng đồ thị k-láng giềng gần nhất (kNN graph) biểu diễn cấu trúc cục bộ.

- Biểu diễn đồ thị đó trong không gian thấp hơn sao cho **cấu trúc cục bộ và toàn cục được bảo toàn tốt nhất**.

6.3.3 Các bước chính

1. **Tạo đồ thị lân cận:** Với mỗi điểm dữ liệu, tìm k láng giềng gần nhất và tính trọng số:

$$w_{ij} = \exp \left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i} \right)$$

trong đó ρ_i là khoảng cách tới láng giềng gần nhất và σ_i được chọn sao cho mật độ được chuẩn hóa.

2. **Tối ưu đồ thị thấp chiều:** Xây dựng không gian nhúng mới (thường 2D hoặc 3D) bằng cách cực tiểu hóa độ mất mát cross-entropy giữa đồ thị gốc và đồ thị trong không gian thấp hơn.

6.3.4 Hàm mất mát của UMAP

UMAP tối thiểu hóa hàm mất mát dựa trên **cross-entropy** giữa phân bố đồ thị cao chiều và thấp chiều:

$$L = \sum_{i \neq j} \left[w_{ij} \log \frac{w_{ij}}{w'_{ij}} + (1 - w_{ij}) \log \frac{1 - w_{ij}}{1 - w'_{ij}} \right]$$

Trong đó w_{ij} là trọng số trong không gian gốc và w'_{ij} là trọng số trong không gian nhúng.

6.3.5 Đặc điểm nổi bật của UMAP

- Bảo toàn tốt cả cấu trúc **cục bộ** và **toàn cục**.
- Tốc độ nhanh hơn t-SNE và mở rộng được cho dữ liệu lớn.
- Cho phép tái sử dụng embedding khi thêm dữ liệu mới.

6.3.6 So sánh PCA, LDA và UMAP

Đặc điểm	PCA	LDA	UMAP
Loại phương pháp	Tuyến tính	Tuyến tính (có nhãn)	Phi tuyến
Tính giám sát	Không giám sát	Có giám sát	Thường không giám sát (có thể mở rộng supervised)
Bảo toàn cấu trúc	Toàn cục (global variance)	Phân tách lớp	Cục bộ và toàn cục
Khả năng mở rộng	Cao	Trung bình	Cao (nhanh hơn t-SNE)
Ứng dụng chính	Giảm chiều, nén dữ liệu	Phân loại	Trực quan hóa dữ liệu phức tạp

6.4 Thực hành trên tập Alzheimer

6.4.1 Chuẩn hóa dữ liệu

- Biến nhị phân: giữ nguyên
- Ordinal: giữ nguyên (0, 1, 2,...) nếu thứ tự class có ý nghĩa, nếu không dùng one-hot encode
(Chúng ta có thể coi EducationalLevel là biến ordinal hoặc nominal đều được.);
- Nominal: one-hot encoding (Ethnicity);
- Biến liên tục: standard scaler (std=1, mean=0).

6.4.2 PCA

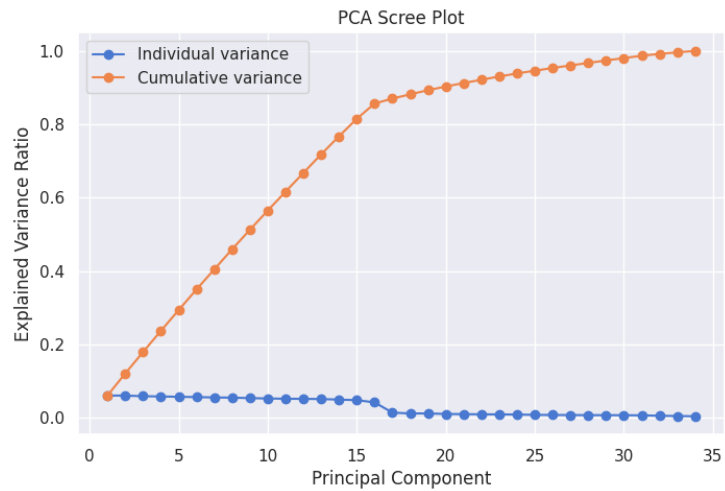
PCA tìm các tổ hợp tuyến tính của các đặc trưng nhằm tối đa hóa phương sai của dữ liệu và giảm chiều trong khi vẫn giữ được phần lớn thông tin. Phương pháp này hữu ích cho trực quan hóa, khám phá cấu trúc dữ liệu và tiền xử lý, nhưng cần chuẩn hóa dữ liệu vì nhạy cảm với thang đo.

Bảng 6.1: Tỷ lệ phương sai giải thích của các thành phần chính (PCA)

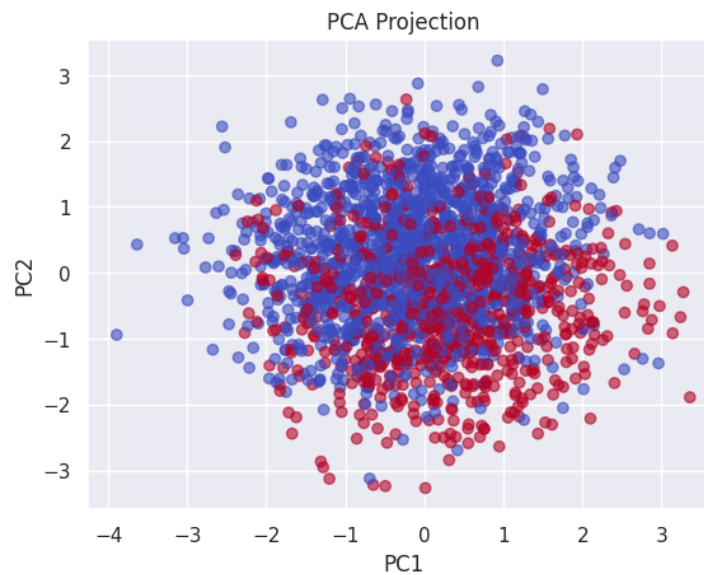
Thành phần chính	Tỷ lệ phương sai giải thích	Phương sai tích lũy
PC1	0.0603	0.0603
PC2	0.0601	0.1204
PC3	0.0585	0.1789
PC4	0.0577	0.2365
PC5	0.0569	0.2934
PC6	0.0560	0.3495
PC7	0.0551	0.4045
PC8	0.0545	0.4591
PC9	0.0533	0.5123
PC10	0.0522	0.5646
PC11	0.0515	0.6161
PC12	0.0512	0.6673
PC13	0.0508	0.7180
PC14	0.0491	0.7672
PC15	0.0477	0.8149
PC16	0.0417	0.8566
PC17	0.0136	0.8701
PC18	0.0115	0.8816
PC19	0.0114	0.8930
PC20	0.0100	0.9030
PC21	0.0094	0.9124
PC22	0.0090	0.9215
PC23	0.0087	0.9302
PC24	0.0085	0.9387
PC25	0.0075	0.9462
PC26	0.0073	0.9534

Kết quả PCA cho thấy 15 thành phần chính đầu tiên giải thích phần lớn phương sai của dữ liệu, khoảng **81.49%**. Sau thành phần thứ 16, giá trị phương sai giảm đáng kể.

Dựa trên biểu đồ phương sai tích lũy (Scree plot), có thể lựa chọn từ **13 đến 15 thành phần chính** để giữ được phần lớn thông tin của dữ liệu mà vẫn giảm được độ phức tạp của mô hình.

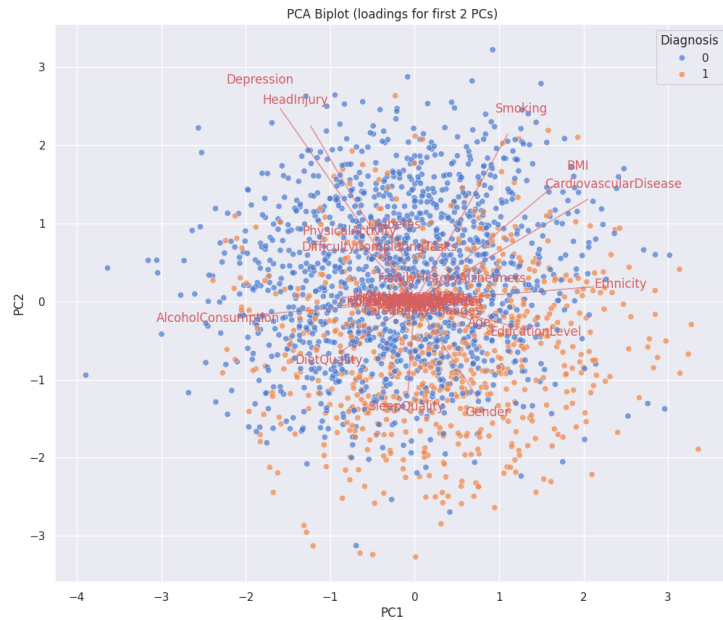


Hình 6.1: Scree plot.



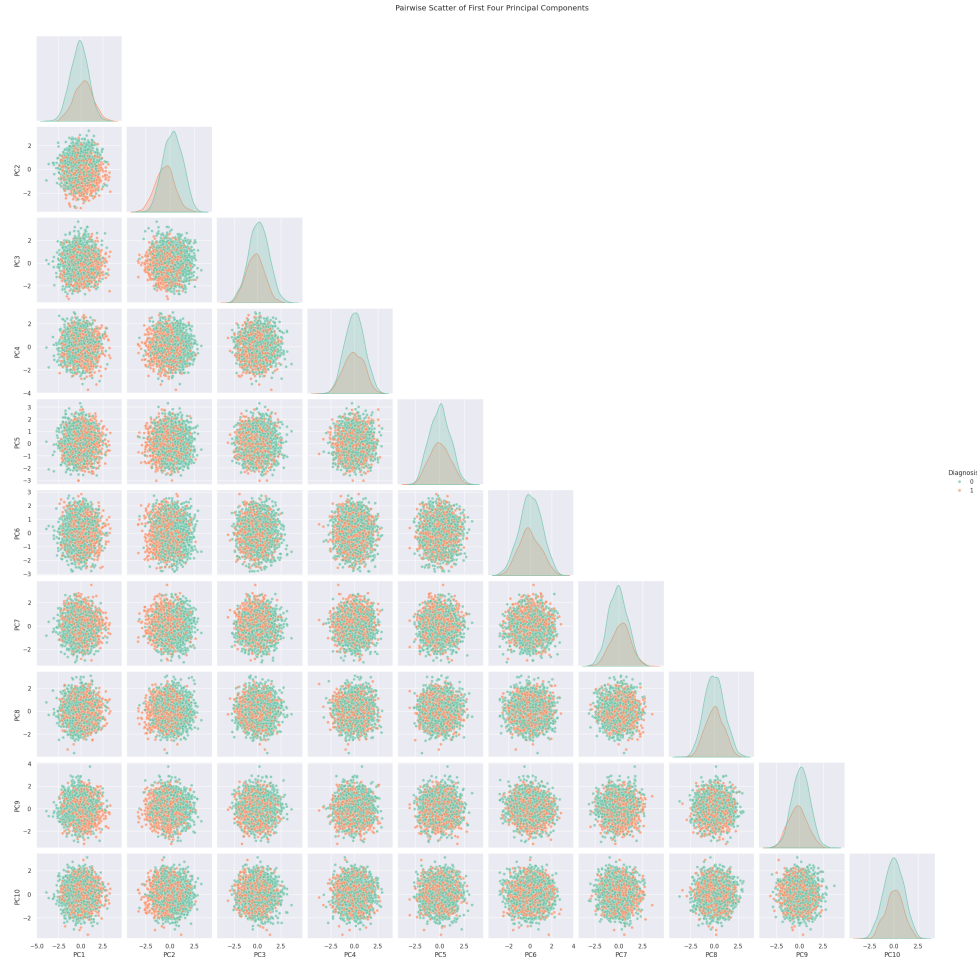
Hình 6.2: Plot dữ liệu theo PC1, PC2.

Biểu đồ phân tán theo hai thành phần chính (**PC1** và **PC2**) cho thấy **sự chồng lấn đáng kể** giữa các nhóm dữ liệu, cho thấy PC1 và PC2 chưa thể tách biệt rõ ràng giữa các bệnh nhân Alzheimer và không Alzheimer.



Hình 6.3: Biplot.

Tất cả các vectơ đặc trưng đều hướng theo các hướng khác nhau, cho thấy rằng các đặc trưng **gần như không tương quan** và **đóng góp theo nhiều hướng độc lập** vào hai thành phần chính. Điều này phản ánh rằng dữ liệu có **cấu trúc phức tạp**, không phát hiện **trục phương sai nổi trội**.



Hình 6.4: Plot từng cặp PCs.

Tất cả các biểu đồ cặp (*pairplots*) cho thấy **sự chồng lấn đáng kể** giữa các nhóm dữ liệu, cho thấy rằng các cặp thành phần chính này **chưa nắm bắt được nhiều thông tin phân biệt giữa các lớp**.

Tóm tắt kết quả PCA: Phân tích thành phần chính (PCA) cho thấy cần ít nhất **15 thành phần chính** để giải thích trên **80% phương sai** của dữ liệu gồm 33 đặc trưng. Điều này cho thấy mức độ tương quan giữa các đặc trưng không cao và phương sai được phân bố tương đối đồng đều giữa nhiều hướng trong không gian đặc trưng. Các biểu đồ phân tán (PC1–PC2) và biểu đồ cặp cho thấy **sự chồng lấn đáng kể** giữa các nhóm, chứng tỏ các thành phần chính đầu tiên **chưa thể hiện rõ khả năng phân biệt lớp**. Biplot cũng cho thấy các vectơ đặc trưng hướng theo nhiều hướng khác nhau, phản ánh cấu trúc dữ liệu phức tạp và không tồn tại trục biến thiên chi phối rõ rệt.

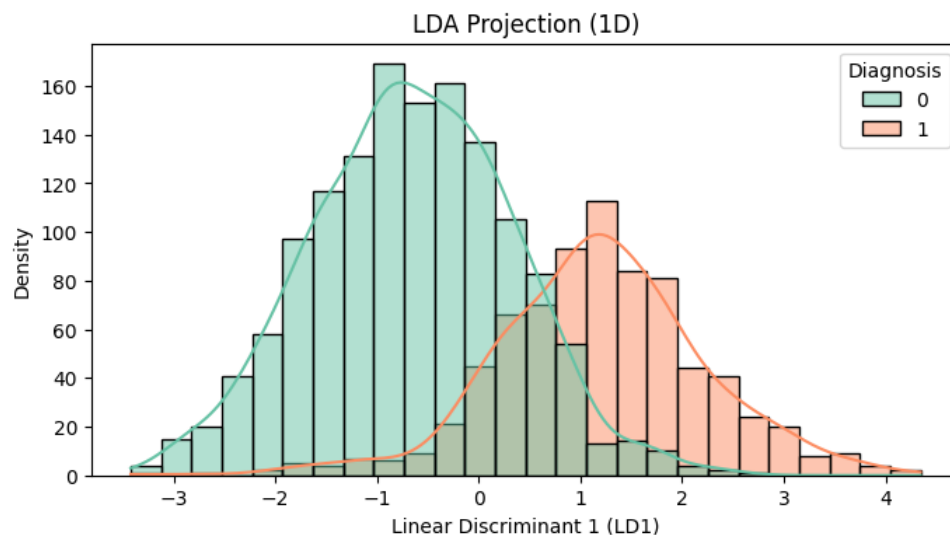
Nhận xét về tác động của PCA:

- **Giảm chiều dữ liệu:** PCA giúp loại bỏ nhiễu và mối tương quan tuyến tính giữa các đặc trưng, tuy nhiên mức độ giảm chiều còn hạn chế (từ 33 xuống khoảng 15 chiều).
- **Trực quan hóa dữ liệu:** Hai hoặc ba thành phần đầu tiên cho phép biểu diễn dữ liệu trong không gian thấp chiều, giúp quan sát được xu hướng phân bố và cấu trúc tổng thể của các nhóm.
- **Phục vụ bài toán phân loại:** Mặc dù PCA không tách biệt rõ hai lớp, nhưng kết quả giảm chiều giúp **chuẩn hóa và nén thông tin** đầu vào, hỗ trợ hiệu quả cho các phương pháp phân loại tuyến tính và phi tuyến được áp dụng ở các bước tiếp theo.

Tổng thể, PCA đóng vai trò quan trọng trong việc **chuẩn bị dữ liệu, giảm nhiễu và tăng tính ổn định cho mô hình**, dù khả năng tách biệt lớp còn hạn chế trong không gian tuyến tính.

6.4.3 LDA

LDA tìm các tổ hợp tuyến tính của đặc trưng nhằm tối đa hóa khả năng phân tách giữa các lớp và tối thiểu hóa phương sai trong lớp. Phương pháp này nhạy cảm với thang đo, nên cần chuẩn hóa dữ liệu trước khi áp dụng.



Hình 6.5: LDA

Xét về khả năng **phân tách tuyến tính giữa các lớp**, thành phần phân biệt đầu tiên của LDA (**LD1**) cho thấy hiệu quả tốt hơn so với các thành phần

chính đầu tiên của PCA (**PC1, PC2**). Phân bố của hai lớp dọc theo LD1 thể hiện **các đỉnh mật độ tách biệt rõ hơn** so với khi quan sát theo PC1 hoặc PC2, cho thấy LDA đã tìm được hướng chiếu tối ưu hơn cho việc phân loại. Tuy nhiên, vẫn tồn tại **sự chồng lấn đáng kể** giữa hai lớp, phản ánh rằng ranh giới tuyến tính chưa thể tách biệt hoàn toàn các nhóm dữ liệu — điều này có thể do sự tương đồng về đặc trưng giữa các đối tượng trong tập dữ liệu.

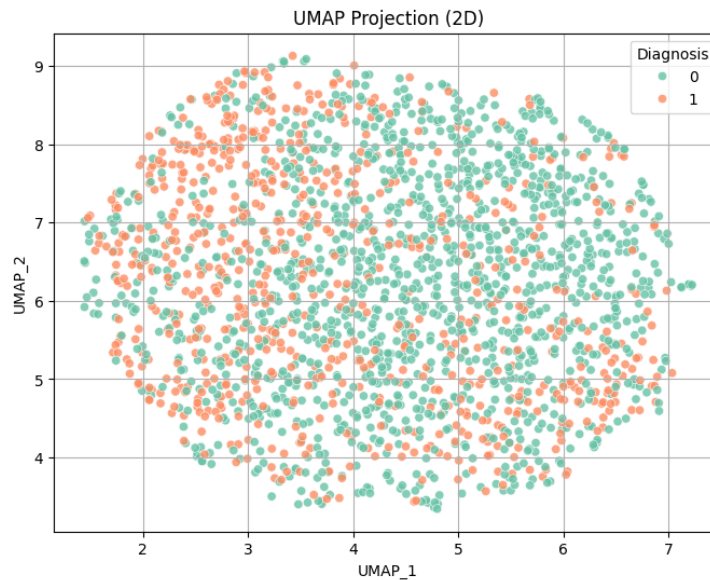
6.4.4 UMAP

UMAP là một phương pháp giảm chiều dữ liệu phi tuyến, được thiết kế để **bảo tồn cấu trúc toàn cục và cục bộ** của dữ liệu gốc trong không gian có chiều thấp hơn. Khác với PCA hay LDA, UMAP không chỉ dựa trên các mối quan hệ tuyến tính mà còn khai thác các **mối quan hệ phi tuyến** giữa các điểm dữ liệu, giúp:

- Giữ gần các điểm dữ liệu có quan hệ gần nhau trong không gian cao chiều;
- Tách biệt các nhóm dữ liệu có cấu trúc khác nhau, ngay cả khi các ranh giới giữa lớp là phi tuyến.

UMAP đặc biệt hữu ích trong các bài toán **trực quan hóa dữ liệu phức tạp** và **khám phá cấu trúc nhóm** mà các phương pháp tuyến tính như PCA hay LDA khó phát hiện được. Phương pháp này **không yêu cầu chuẩn hóa dữ liệu** nghiêm ngặt, nhưng kết quả có thể bị ảnh hưởng bởi các siêu tham số như `n_neighbors` và `min_dist`, do đó cần điều chỉnh cẩn thận để đạt hiệu quả trực quan tốt nhất.

Với 2 thành phần chính:

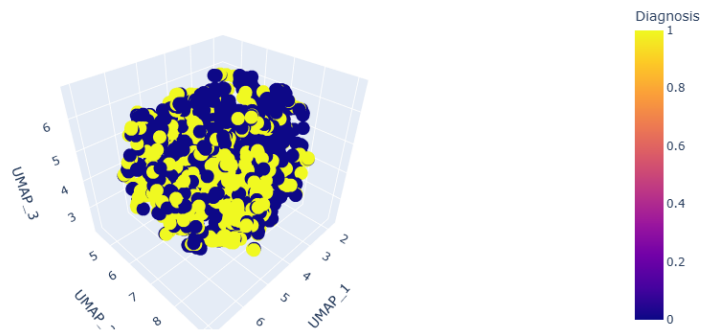


Hình 6.6: Unsupervised UMAP 2D

Chúng ta chưa thấy được sự phân cụm rõ ràng, thực tế 2 lớp 0, 1 bị trùng lặp rất nhiều. Có thể thấy lớp 0 dày hơn ở bên phải (tương ứng giá trị cao hơn của UMAP 1).

Với 3 thành phần chính:

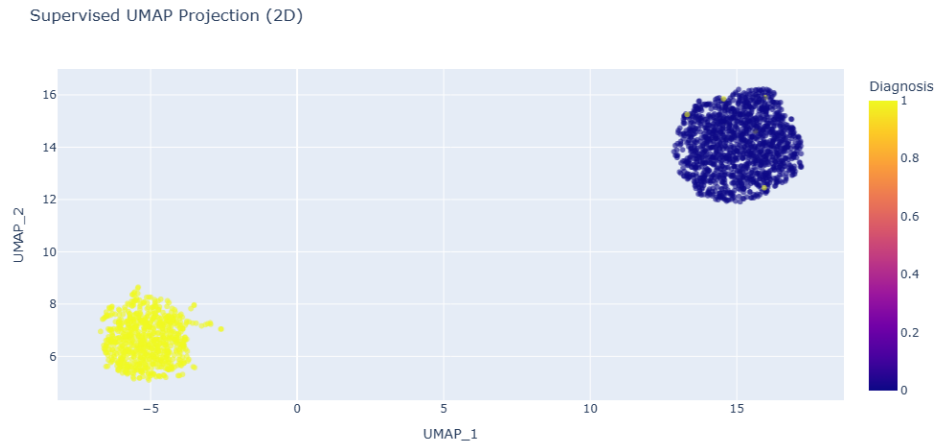
UMAP Projection (3D) - n_neighbors=5, min_dist=0.05



Hình 6.7: Unsupervised UMAP 3D

Ta thấy thêm một số góc tập hợp nhiều class 0/1 (với giá trị lớn hơn của UMAP-1 ta thấy tập trung nhiều lớp 0 hơn, lớp 1 thường có UMAP-1 nhỏ), tuy nhiên vẫn ko có sự tách biệt rõ ràng giữa 2 lớp.

UMAP có giám sát



Hình 6.8: Supervised UMAP 2D

Khả năng phân tách lớp mạnh: Việc UMAP, được hướng dẫn bởi nhãn chẩn đoán, có thể tìm ra một không gian 2 chiều mà các lớp dữ liệu tách biệt rõ ràng cho thấy tồn tại các mẫu và sự khác biệt cơ bản đáng kể trong các đặc trưng, có mối liên hệ chặt chẽ với nhãn *Diagnosis*. Dữ liệu có cấu trúc tốt, cho phép tách biệt các lớp dựa trên thông tin phân loại.

Biểu diễn hiệu quả trong không gian chiều thấp: Điều này cho thấy thông tin quan trọng nhất để phân biệt hai nhóm chẩn đoán có thể được biểu diễn hiệu quả chỉ với hai chiều. Điều này thuận lợi cho việc trực quan hóa dữ liệu và gợi ý rằng các mô hình dự đoán có thể không cần phải xử lý toàn bộ độ phức tạp của không gian gốc cao chiều để đạt độ chính xác cao.

Mối quan hệ phi tuyến là chìa khóa: Vì PCA (không giám sát) và UMAP (không giám sát) không thể hiện mức độ phân tách rõ ràng như vậy, trong khi UMAP có giám sát lại làm được, điều này nhấn mạnh rằng các mẫu phân biệt giữa các lớp khả năng cao là phi tuyến. UMAP có giám sát tận dụng nhãn lớp để tìm các ranh giới và cấu trúc phi tuyến này, giúp phân tách các lớp hiệu quả hơn.

6.4.5 Tổng kết

PCA (Phân tích thành phần chính)

Mục tiêu: Tìm các thành phần trực giao (orthogonal components) nhằm nắm bắt tối đa phương sai trong dữ liệu. PCA là phương pháp **không giám sát**.

Phương pháp: Biến đổi tuyến tính các đặc trưng.

Kết quả:

- PC1 chỉ giải thích một tỷ lệ phương sai tương đối thấp (khoảng 6%), cho thấy dữ liệu có nhiều chiều và phương sai phân tán trên nhiều đặc trưng.
- Trục quan hóa PC1 vs PC2 (và các cặp thành phần khác) cho thấy sự **chồng lấn đáng kể** giữa hai nhóm chẩn đoán.
- Cần giữ một số lượng lớn thành phần (khoảng 16 PC) để bảo toàn phần lớn phương sai tổng thể (khoảng 85%).

Nhận xét: PCA hữu ích để hiểu cấu trúc phương sai và giảm chiều dữ liệu trong khi vẫn giữ được phương sai, nhưng các thành phần đầu tiên không thể hiện rõ sự tách biệt tuyến tính giữa các lớp cho trục quan hóa.

LDA (Phân tích discriminant tuyến tính)

Mục tiêu: Tìm các thành phần tuyến tính tối đa hóa sự phân tách giữa các lớp trong khi giảm thiểu phương sai trong từng lớp. LDA là phương pháp **giám sát** sử dụng nhãn mục tiêu.

Phương pháp: Biến đổi tuyến tính hướng tới phân loại, số lượng thành phần giới hạn bằng `n_classes - 1`.

Kết quả:

- LDA tạo ra một thành phần duy nhất (LD1) cho bài toán phân loại nhị phân.
- Biểu đồ histogram của LD1 cho thấy **chồng lấn vừa phải** giữa hai nhóm chẩn đoán, mặc dù các đỉnh phân bố có sự khác biệt nhất định.

Nhận xét: LDA tìm được trục tuyến tính tốt nhất để phân tách các lớp. Mặc dù có một số tách biệt, nhưng không hoàn toàn tuyến tính, phù hợp với kết quả UMAP gợi ý sự cấu trúc phi tuyến có thể quan trọng.

UMAP (Uniform Manifold Approximation and Projection)

Mục tiêu: Tìm một biểu diễn không gian chiều thấp bảo tồn cấu trúc topo của dữ liệu. UMAP là phương pháp **phi tuyến, không giám sát** thường dùng cho trực quan hóa.

Phương pháp: Giảm chiều phi tuyến dựa trên học manifold.

Kết quả: Biểu đồ 2D UMAP cho thấy sự tách biệt trực quan rõ ràng hơn giữa hai nhóm chẩn đoán so với PCA 2D.

Nhận xét: UMAP hiệu quả hơn PCA trong việc phát hiện cấu trúc hoặc các cụm phi tuyến liên quan đến chẩn đoán, là công cụ tốt hơn để trực quan hóa sự tách biệt các lớp.

So sánh tổng quan

- Về trực quan hóa sự phân tách các lớp, UMAP tỏ ra hiệu quả nhất, gợi ý rằng các mối quan hệ phi tuyến quan trọng để phân biệt các nhóm chẩn đoán.
- Về giảm chiều dữ liệu trong khi giữ phương sai tổng thể, PCA vẫn là phương pháp tiêu chuẩn.
- Về tìm chiều tuyến tính tối ưu cho phân tách lớp, LDA có thể áp dụng, nhưng hiệu quả bị giới hạn bởi giả định tuyến tính và số lượng thành phần.

Việc UMAP cho thấy tách biệt trực quan tốt hơn cả PCA và LDA cho thấy việc thử các mô hình phân loại phi tuyến trên dữ liệu gốc hoặc dữ liệu đã giảm chiều bằng PCA là một hướng đi triển vọng.

KẾT QUẢ THỰC NGHIỆM

7.1 K-nearest neighbor

Việc chọn giá trị tối ưu cho các tham số như k (số lượng láng giềng) trong mô hình K-Nearest Neighbors rất quan trọng vì chúng ảnh hưởng trực tiếp đến hiệu suất và độ chính xác của mô hình. Quá trình tìm kiếm các giá trị này có thể được thực hiện thông qua phương pháp GridSearchCV, cho phép thử nghiệm với nhiều giá trị khác nhau của k , đồng thời đánh giá hiệu quả của mô hình trên mỗi giá trị này.

Nếu giá trị k quá nhỏ, mô hình có thể bị overfitting (quá khớp với dữ liệu huấn luyện), tức là mô hình sẽ học quá kỹ vào các đặc điểm cụ thể của dữ liệu huấn luyện, dẫn đến khả năng tổng quát kém trên dữ liệu mới. Ngược lại, nếu k quá lớn, mô hình có thể bị underfitting, tức là không học đủ các đặc trưng quan trọng của dữ liệu.

Để xác định giá trị k tối ưu, chúng ta có thể sử dụng GridSearchCV để thử nghiệm trên nhiều giá trị của k . Phương pháp này sẽ phân chia dữ liệu thành các phần nhỏ và huấn luyện mô hình với các giá trị của k trên các phần dữ liệu khác nhau, sau đó tính toán độ chính xác của mô hình trên mỗi tập kiểm tra. Cuối cùng, giá trị k mang lại độ chính xác cao nhất sẽ được chọn là tham số tối ưu cho mô hình KNN.

7.1.1 Trên tập dữ liệu gốc

Sau khi chạy GridSearchCV trên ba tỷ lệ phân chia dữ liệu 4:1, 7:3, 6:4, với giá trị K được chọn là 15 cho phân chia 4:1 và K là 11 cho phân chia 7:3, 6:4, mô hình KNN cho thấy một số kết quả như sau:

- **Tổng quan độ chính xác:** Độ chính xác (Accuracy) của mô hình giữ được hiệu suất ổn định trong khoảng 86-89% ở ba tỷ lệ phân chia 4:1, 7:3, 6:4. Điều này cho thấy mô hình có khả năng phân loại ổn định bất kể tỷ lệ huấn luyện và kiểm tra. Sự ổn định này chứng tỏ rằng mô hình không bị ảnh hưởng nhiều bởi sự thay đổi trong tỷ lệ phân chia giữa tập huấn luyện và tập kiểm tra, giúp tăng tính tổng quát của mô hình.

Class	Precision 4:1	Recall 4:1	F1-score 4:1
0	0.92	0.88	0.90
1	0.80	0.86	0.83
Accuracy			0.88
Macro Avg	0.86	0.87	0.87
Weighted Avg	0.88	0.88	0.88

Bảng 7.1: Bảng kết quả phân loại cho phân chia 4:1

Class	Precision 7:3	Recall 7:3	F1-score 7:3
0	0.93	0.89	0.91
1	0.82	0.88	0.85
Accuracy			0.89
Macro Avg	0.87	0.89	0.88
Weighted Avg	0.89	0.89	0.89

Bảng 7.2: Bảng kết quả phân loại cho phân chia 7:3

Class	Precision 6:4	Recall 6:4	F1-score 6:4
0	0.91	0.87	0.89
1	0.77	0.84	0.80
Accuracy			0.86
Macro Avg	0.84	0.85	0.85
Weighted Avg	0.86	0.86	0.86

Bảng 7.3: Bảng kết quả phân loại cho phân chia 6:4

- **Hiệu suất theo lớp:** Các lớp 0 và 1 đạt kết quả Precision, Recall và F1-score khá cao, đặc biệt là Precision của lớp 0 là 0.93, F1-score của lớp 0 là 0.91. Nhưng lớp 1 có hiệu suất kém hơn, đặc biệt ở Precision cho thấy rằng một số dự đoán dương cho lớp 1 là sai (false positives) hoặc đặc trưng lớp 1 có phần trùng lặp với lớp khác.
- **Khó khăn khi phân loại:** Mô hình KNN dựa trên khoảng cách giữa các điểm dữ liệu. Vì vậy, khi các mẫu của hai lớp phân bố gần nhau trong không gian đặc trưng, ranh giới giữa các lớp trở nên không rõ ràng, dễ nhầm lẫn. Sự mất cân bằng giữa hai lớp cũng khiến lớp thiểu số khó được nhận dạng chính xác. Ở đây số lượng mẫu lớp 0 nhiều hơn, dẫn đến việc các điểm lân cận của lớp thiểu số (lớp 1) thường bị bao quanh bởi các điểm thuộc lớp đa

số. Điều này khiến mô hình có xu hướng dự đoán sai sang lớp chiếm ưu thế.

- **Sự cải thiện khi sử dụng GridSearchCV:** GridSearchCV giúp mô hình KNN lựa chọn giá trị K tối ưu cho mỗi tỉ lệ phân chia dữ liệu. Dựa vào đó hiệu suất phân loại được cải thiện rõ so với kết quả ban đầu. Đặc biệt, độ chính xác tổng thể tăng lên thể hiện rằng việc tinh chỉnh siêu tham số bằng GridSearchCV giúp mô hình tổng quát tốt hơn, giảm hiện tượng overfitting và cải thiện khả năng nhận diện chính xác giữa các lớp.

Mô hình KNN với việc lựa chọn K tối ưu thông qua GridSearchCV cho thấy sự ổn định và hiệu quả cao trong việc phân loại dữ liệu bất kể tỉ lệ phân chia huấn luyện và kiểm tra. Mặc dù mô hình hoạt động rất tốt với lớp 0 và 1, nhưng vẫn gặp một số khó khăn đó là lớp 1 có hiệu suất kém hơn so với lớp 0. Tuy nhiên, độ chính xác tổng thể của mô hình vẫn ổn định ở mức cao (khoảng 89%).

7.1.2 Trên tập dữ liệu đã giảm chiều PCA

Sau khi chạy GridSearchCV trên ba tỉ lệ phân chia dữ liệu 4:1, 7:3, 6:4, với giá trị K được chọn là 15 cho phân chia 4:1, 7:3 và 6:4, mô hình KNN cho thấy một số kết quả như sau:

Class	Precision 4:1	Recall 4:1	F1-score 4:1
0	0.82	0.68	0.74
1	0.55	0.74	0.63
Accuracy			0.70
Macro Avg	0.69	0.71	0.69
Weighted Avg	0.73	0.70	0.70

Bảng 7.4: Bảng kết quả phân loại cho phân chia 4:1

Class	Precision 7:3	Recall 7:3	F1-score 7:3
0	0.85	0.68	0.76
1	0.57	0.79	0.66
Accuracy			0.72
Macro Avg	0.71	0.73	0.71
Weighted Avg	0.75	0.72	0.72

Bảng 7.5: Bảng kết quả phân loại cho phân chia 7:3

Class	Precision 6:4	Recall 6:4	F1-score 6:4
0	0.83	0.64	0.73
1	0.54	0.77	0.63
Accuracy			0.69
Macro Avg	0.69	0.70	0.68
Weighted Avg	0.73	0.69	0.69

Bảng 7.6: Bảng kết quả phân loại cho phân chia 6:4

- **Tổng quan độ chính xác:** Độ chính xác của mô hình mang lại hiệu suất ổn định ở mức trung bình khá khoảng 69-72% ở ba tỷ lệ phân chia 4:1, 7:3, 6:4. Tỷ lệ 7:3 cho hiệu suất tổng thể tốt nhất về độ chính xác. Ngoài ra tỷ lệ 6:4 cho F1 cao nhất trong cross-validation, cho thấy mô hình tổng quát hóa tốt hơn trong huấn luyện nhưng bị giảm nhẹ khi test do nhiều dữ liệu kiểm thử.
- **Hiệu suất theo lớp:** Cả ba tỷ lệ đều cho ra precision ở lớp 0 rất tốt (82-85%) nhưng recall dao động thấp chỉ (64-68%). Ngược lại, lớp 1 có recall cao (74-79%) cho thấy mô hình phát hiện tốt nhiều mẫu của lớp 1 nhưng cũng dễ bị nhầm lẫn. Điều đó cũng không ảnh hưởng quá nhiều, độ chính xác tổng thể duy trì ổn định hơn, chứng tỏ rằng mô hình thích nghi tốt hơn với dữ liệu đã chiếu sang không gian đặc trưng mới.
- **Sự cải thiện khi sử dụng GridSearchCV:** Khi kết hợp giảm chiều bằng PCA với tinh chỉnh siêu tham số bằng GridSearchCV thì PCA loại bỏ các chiều dữ liệu không quan trọng hoặc tương quan cao, tập trung vào những thành phần đặc trưng có sức phân biệt tốt nhất. Ngoài ra, cải thiện độ ổn định và khả năng tổng quát giúp hoạt động ổn định hơn trên các tập dữ liệu khác nhau, thể hiện qua Accuracy dao động trong khoảng 69-72% nhưng không giảm mạnh ở bất kỳ tỷ lệ chia dữ liệu nào.

Việc giảm chiều dữ liệu bằng PCA đã không làm giảm quá nhiều hiệu suất của mô hình, mặc dù có một sự giảm nhẹ về độ chính xác và các chỉ số liên quan đến lớp '1'. Các chỉ số tổng thể vẫn duy trì ổn định, cho thấy mô hình có thể hoạt động khá tốt với dữ liệu đã giảm chiều. Mô hình KNN cho thấy khả năng phân loại ổn định và tốt đối với lớp '0', tuy nhiên cần cải thiện khả năng phân loại lớp '1' khi giảm chiều dữ liệu.

Mô hình vẫn có thể cải thiện thêm hiệu suất, đặc biệt đối với các lớp có sự chồng chéo giữa các đặc trưng, thông qua việc điều chỉnh tham số GridSearchCV tốt hơn hoặc thử nghiệm các phương pháp giảm chiều dữ liệu khác.

7.1.3 Trên tập dữ liệu đã giảm chiều LDA

Sau khi chạy GridSearchCV trên ba tỷ lệ phân chia dữ liệu 4:1, 7:3, 6:4, với giá trị K được chọn là 15 cho phân chia 4:1 và K là 13 cho phân chia 7:3, 6:4, mô hình KNN cho thấy một số kết quả như sau:

Class	Precision 4:1	Recall 4:1	F1-score 4:1
0	0.85	0.86	0.86
1	0.74	0.72	0.73
Accuracy			0.81
Macro Avg	0.80	0.79	0.80
Weighted Avg	0.81	0.81	0.81

Bảng 7.7: Bảng kết quả phân loại cho phân chia 4:1

Class	Precision 7:3	Recall 7:3	F1-score 7:3
0	0.86	0.88	0.87
1	0.76	0.74	0.75
Accuracy			0.83
Macro Avg	0.81	0.81	0.81
Weighted Avg	0.83	0.83	0.83

Bảng 7.8: Bảng kết quả phân loại cho phân chia 7:3

Class	Precision 6:4	Recall 6:4	F1-score 6:4
0	0.86	0.88	0.87
1	0.77	0.74	0.76
Accuracy			0.83
Macro Avg	0.82	0.81	0.81
Weighted Avg	0.83	0.83	0.83

Bảng 7.9: Bảng kết quả phân loại cho phân chia 6:4

- **Tổng quan độ chính xác:** Độ chính xác của mô hình có hiệu suất ổn định (81-83%) ở ba tỷ lệ phân chia, cao hơn rõ rệt so với giảm chiều bằng PCA. Điều này chứng tỏ rằng LDA hoạt động hiệu quả và ổn định hơn nhờ khả năng tìm ra tổ hợp tuyến tính tối ưu giữa các đặc trưng để tách biệt hai lớp.

- **Hiệu suất theo lớp:** Ở lớp 0 có Precision và Recall cao dao động 85-88%, F1-score dao động 86-87% cho thấy rằng mô hình phân loại đúng hầu hết các mẫu thuộc lớp này. Lớp 1 mặc dù thấp hơn lớp 0 nhưng Precision và Recall đều ở mức 72-77%, F1-score khoảng 73-76% cho thấy giảm chiều LDA nhận diện lớp thiếu sót tốt hơn so với dữ liệu gốc.
- **Sự cải thiện khi sử dụng GridSearchCV:** Khi kết hợp giảm chiều bằng LDA không chỉ giúp tăng độ chính xác tổng thể mà còn cải thiện rõ rệt Precision, Recall và F1-score cho cả hai lớp. Sự chênh lệch giữa hai lớp đã được thu hẹp giúp mô hình cân bằng hơn trong việc nhận dạng cả hai lớp. Giảm chiều đồng thời giữ lại thông tin quan trọng giúp phân biệt lớp giúp mô hình tránh được hiện tượng nhiễu và chồng lấn dữ liệu.

Như vậy, việc giảm chiều dữ liệu bằng LDA đã giúp mô hình duy trì hiệu suất phân loại ổn định, nhưng vẫn cần cải thiện khả năng phân loại lớp 1, đặc biệt khi có sự chồng chéo giữa các lớp tuy nhiên độ chính xác nhỏ hơn dữ liệu gốc.

7.1.4 Tổng kết trên tập dữ liệu gốc và dữ liệu giảm chiều

Sau khi thử nghiệm mô hình KNN với phương pháp GridSearchCV trên ba tập dữ liệu với ba phương pháp phân chia khác nhau 4:1, 7:3 và 6:4. Chúng ta nhận thấy mô hình vẫn duy trì hiệu suất phân loại ổn định bất kể việc giảm chiều dữ liệu bằng PCA hay LDA. Tuy nhiên, có sự khác biệt nhẹ trong kết quả giữa dữ liệu gốc và dữ liệu sau khi giảm chiều.

- **Dữ liệu gốc:** Độ chính xác trên dữ liệu gốc đạt khoảng 89%, với mô hình phân loại rất tốt cho cả lớp 0 và lớp 1. Các lớp đạt kết quả gần như xuất sắc, đặc biệt là lớp 0 với Precision đạt 0.93.
- **Dữ liệu giảm chiều PCA:** Sau khi giảm chiều dữ liệu bằng PCA, độ chính xác của mô hình giảm nhẹ xuống 69-72% so với dữ liệu gốc. Các chỉ số Precision, Recall và F1-score đối với hai lớp giảm nhẹ, đặc biệt lớp 1 gặp khó khăn hơn, đặc biệt là giảm nhẹ Recall và F1-score, cho thấy mô hình gặp khó khăn trong việc phân loại lớp 1. Tuy nhiên, các chỉ số tổng thể duy trì ở mức ổn định.
- **Dữ liệu giảm chiều LDA:** Sau khi giảm chiều dữ liệu bằng LDA, mô hình vẫn duy trì độ chính xác ở mức 81-83%, có sự giảm nhẹ so với dữ liệu gốc, nhưng vẫn ổn định. Các chỉ số cho lớp 0 vẫn giữ ở mức rất tốt, nhưng lớp 1 gặp phải sự giảm nhẹ ở Recall và F1-score, đặc biệt là ở các tỷ lệ phân chia

như 4:1. Mặc dù vậy, mô hình vẫn duy trì được kết quả khá tốt với các chỉ số tổng thể ở mức ổn định.

7.2 Softmax regression

Mặc dù bài toán đặt ra là phân loại nhị phân, nhóm vẫn lựa chọn sử dụng Logistic Regression dạng softmax với thiết lập `mult_class='multinomial'` nhằm khai thác khả năng ước lượng xác suất chính xác cho từng lớp. Việc sử dụng hàm softmax thay vì hàm sigmoid truyền thống không gây ảnh hưởng tới độ chính xác của mô hình nhị phân, đồng thời giúp mô hình duy trì sự nhất quán nếu sau này mở rộng sang phân loại đa lớp.

Bên cạnh đó, `solver='lbfgs'` được lựa chọn do đây là thuật toán tối ưu phù hợp với bài toán logistic regression có sử dụng hàm mất mát mềm như softmax. Để đảm bảo mô hình hội tụ ổn định ngay cả trong trường hợp dữ liệu nhiều chiều hoặc phân bố phức tạp, tham số `max_iter` được đặt giá trị lớn là 20000. Nếu để giá trị quá thấp, mô hình có thể không hội tụ được, dẫn đến kết quả không chính xác hoặc cảnh báo dừng sớm khi huấn luyện.

7.2.1 Trên tập dữ liệu gốc

Class	Precision 4:1	Recall 4:1	F1-score 4:1
0	0.87	0.87	0.87
1	0.76	0.76	0.76
Accuracy			0.83
Macro Avg	0.81	0.82	0.81
Weighted Avg	0.83	0.83	0.83

Bảng 7.10: Bảng kết quả phân loại cho phân chia 4:1

Class	Precision 7:3	Recall 7:3	F1-score 7:3
0	0.87	0.88	0.88
1	0.78	0.77	0.77
Accuracy			0.84
Macro Avg	0.83	0.83	0.83
Weighted Avg	0.84	0.84	0.84

Bảng 7.11: Bảng kết quả phân loại cho phân chia 7:3

Class	Precision 6:4	Recall 6:4	F1-score 6:4
0	0.88	0.87	0.87
1	0.77	0.77	0.77
Accuracy			0.84
Macro Avg	0.82	0.82	0.82
Weighted Avg	0.84	0.84	0.84

Bảng 7.12: Bảng kết quả phân loại cho phân chia 6:4

Mô hình phân loại cho các tỉ lệ phân chia dữ liệu 4:1, 7:3 và 6:4 đều đạt độ chính xác (accuracy) tốt, dao động quanh mức 83-84%. Tuy nhiên, các chỉ số precision, recall và F1-score phân bố chưa thật sự đồng đều giữa hai lớp. Cụ thể, lớp 0 đạt kết quả cao hơn với precision, recall và F1-score trong khoảng 87-88%; trong khi đó, lớp 1 có các chỉ số thấp hơn, dao động từ 76-78%.

Dù vậy, sự chênh lệch giữa hai lớp không quá lớn, thể hiện qua các chỉ số macro average và weighted average đều nằm trong khoảng 0.81-0.84, phản ánh hiệu suất mô hình tương đối ổn định và cân bằng giữa các lớp.

7.2.2 Trên tập dữ liệu đã giảm chiều PCA

Class	Precision 4:1	Recall 4:1	F1-score 4:1
0	0.86	0.87	0.86
1	0.75	0.74	0.74
Accuracy			0.82
Macro Avg	0.80	0.80	0.80
Weighted Avg	0.82	0.82	0.82

Bảng 7.13: Bảng kết quả phân loại cho phân chia 4:1

Class	Precision 7:3	Recall 7:3	F1-score 7:3
0	0.87	0.88	0.88
1	0.78	0.76	0.77
Accuracy			0.84
Macro Avg	0.82	0.82	0.82
Weighted Avg	0.84	0.84	0.84

Bảng 7.14: Bảng kết quả phân loại cho phân chia 7:3

Class	Precision 6:4	Recall 6:4	F1-score 6:4
0	0.87	0.87	0.87
1	0.76	0.76	0.76
Accuracy			0.83
Macro Avg	0.82	0.82	0.82
Weighted Avg	0.83	0.83	0.83

Bảng 7.15: Bảng kết quả phân loại cho phân chia 6:4

Sau khi giảm chiều dữ liệu bằng PCA, mô hình đạt độ chính xác (accuracy) dao động từ 0.82 đến 0.84 cho ba tỉ lệ phân chia dữ liệu (4:1, 7:3 và 6:4), thấp hơn một chút so với khi huấn luyện trên tập dữ liệu gốc. Các chỉ số precision, recall và F1-score của cả hai lớp đều giảm nhẹ, đặc biệt là ở lớp 1, với F1-score chỉ đạt khoảng 0.74-0.77, trong khi lớp 0 vẫn duy trì kết quả khá tốt, khoảng 0.86-0.88.

Việc giảm chiều dữ liệu bằng kỹ thuật PCA (Principal Component Analysis) giúp loại bỏ nhiễu và giảm độ phức tạp của mô hình, tuy nhiên cũng có thể dẫn đến mất mát thông tin quan trọng. PCA chỉ giữ lại những thành phần chính có phương sai lớn nhất trong dữ liệu, nhưng không đảm bảo rằng các thành phần đó là những đặc trưng tốt nhất cho mục tiêu phân loại. Do đó, một phần thông tin đặc trưng phân lớp có thể bị loại bỏ, làm mô hình khó phân biệt hai lớp chính xác như khi sử dụng dữ liệu gốc đầy đủ.

Ngoài ra, việc giảm chiều đồng nghĩa với việc biểu diễn dữ liệu trên một không gian con có ít thông tin hơn, dẫn đến rủi ro giảm độ phân tách giữa các lớp. Điều này giải thích tại sao các chỉ số như accuracy, precision, recall, F1-score hay macro/weighted average đều có xu hướng giảm nhẹ sau PCA.

Mặc dù vậy, mô hình vẫn thể hiện được hiệu suất ổn định trên các tỉ lệ phân chia khác nhau, thể hiện qua các giá trị macro average và weighted average dao động quanh 0.80-0.84. Điều này cho thấy PCA đã giữ lại được phần lớn thông tin quan trọng, giúp mô hình tiếp tục học hiệu quả trên không gian đặc trưng đã giảm chiều, dù hiệu suất tổng thể có giảm nhẹ so với dữ liệu gốc.

7.2.3 Trên tập dữ liệu đã giảm chiều LDA

Class	Precision 4:1	Recall 4:1	F1-score 4:1
0	0.87	0.86	0.86
1	0.75	0.76	0.75
Accuracy			0.83
Macro Avg	0.81	0.81	0.81
Weighted Avg	0.83	0.83	0.83

Bảng 7.16: Bảng kết quả phân loại cho phân chia 4:1

Class	Precision 7:3	Recall 7:3	F1-score 7:3
0	0.87	0.89	0.88
1	0.79	0.76	0.77
Accuracy			0.84
Macro Avg	0.83	0.82	0.83
Weighted Avg	0.84	0.84	0.84

Bảng 7.17: Bảng kết quả phân loại cho phân chia 7:3

Class	Precision 6:4	Recall 6:4	F1-score 6:4
0	0.87	0.88	0.88
1	0.77	0.77	0.77
Accuracy			0.84
Macro Avg	0.82	0.82	0.82
Weighted Avg	0.84	0.84	0.84

Bảng 7.18: Bảng kết quả phân loại cho phân chia 6:4

Trên tập dữ liệu sau khi giảm chiều bằng LDA (Linear Discriminant Analysis), mô hình đạt hiệu suất gần như tương đồng với khi huấn luyện trên tập dữ liệu gốc, với độ chính xác (accuracy) dao động trong khoảng 0.83–0.84 ở cả ba tỉ lệ phân chia dữ liệu (4:1, 7:3 và 6:4). Các chỉ số precision, recall và F1-score của hai lớp đều được duy trì ở mức cao, chỉ có một vài thay đổi rất nhỏ, chẳng hạn precision của lớp 0 ở tỉ lệ 6:4 giảm khoảng 1%, hay macro average của recall giảm khoảng 1%, mức chênh lệch này là không đáng kể.

Khác với PCA, kỹ thuật LDA không chỉ quan tâm đến phương sai dữ liệu mà còn tối đa hóa khả năng phân tách giữa các lớp. Cụ thể, LDA tìm kiếm một không gian con mà tại đó khoảng cách giữa các lớp là lớn nhất, trong khi khoảng cách nội lớp là nhỏ nhất. Do đó, đặc trưng tạo ra từ LDA giữ lại được

thông tin phân loại cốt lõi, giúp mô hình vẫn phân biệt tốt giữa hai lớp ngay cả khi số chiều bị giảm.

Sự dao động nhẹ ở một vài chỉ số có thể xuất phát từ biến động tự nhiên của dữ liệu khi chia tập train-test, chứ không phải do mất mát thông tin trong quá trình giảm chiều. Nhìn chung, mô hình vẫn giữ được độ chính xác và tính ổn định cao, cho thấy LDA là phương pháp giảm chiều hiệu quả cho bài toán phân loại nhị phân, đặc biệt trong các trường hợp phân bố của hai lớp có độ tách biệt rõ ràng.

Kết quả này cũng khẳng định rằng việc áp dụng LDA không chỉ giúp giảm số chiều và rút ngắn thời gian huấn luyện, mà còn duy trì được hiệu suất tương đương với mô hình huấn luyện trên tập dữ liệu đầy.

7.2.4 Tổng kết trên tập dữ liệu gốc và dữ liệu giảm chiều

Qua quá trình thực nghiệm trên tập dữ liệu gốc và hai phương pháp giảm chiều PCA và LDA, có thể nhận thấy rằng mô hình duy trì được hiệu suất ổn định và độ chính xác cao ở cả ba trường hợp. Trong đó, mô hình trên tập dữ liệu gốc vẫn đạt kết quả cao và ổn định nhất. Từ đó, phản ánh được khả năng khai thác thông tin đặc trưng đầy đủ của tập dữ liệu ban đầu.

Với PCA, mặc dù độ chính xác và các chỉ số như precision, recall, F1-score giảm nhẹ do mất mát một phần thông tin trong quá trình chiếu dữ liệu, song mô hình vẫn đảm bảo khả năng phân loại tốt và tương đối ổn định.

Ngược lại, LDA cho thấy hiệu quả vượt trội hơn khi duy trì hiệu suất gần như tương đương dữ liệu gốc, đồng thời tăng tính ổn định giữa các tỉ lệ phân chia và giảm đáng kể số chiều đặc trưng, giúp mô hình huấn luyện nhanh hơn mà không ảnh hưởng nhiều đến chất lượng.

Nhìn chung, việc áp dụng các kỹ thuật giảm chiều không làm suy giảm đáng kể hiệu suất mô hình; trong đó, LDA được đánh giá là phương pháp phù hợp hơn PCA cho bài toán phân loại nhị phân này, nhờ khả năng giữ vững độ chính xác, ổn định và tối ưu hóa hiệu quả tính toán.

7.3 So sánh hai mô hình KNN và Softmax

Trong khuôn khổ bài thực hành, hai mô hình phân loại K-Nearest Neighbors (KNN) và Softmax Regression đã được triển khai trên cùng một tập dữ liệu nhị phân với ba dạng biểu diễn khác nhau gồm dữ liệu gốc, dữ liệu giảm chiều bằng PCA, và dữ liệu giảm chiều bằng LDA. Dựa trên kết quả đánh giá định lượng và trực quan hóa, có thể rút ra một số nhận định tổng quan như sau:

Kết quả thực nghiệm cho thấy mô hình KNN đạt hiệu năng cao hơn trên dữ liệu gốc, trong khi Softmax Regression thể hiện độ ổn định tốt hơn khi giảm chiều dữ liệu bằng PCA, và cả hai mô hình cho kết quả tương đương nhau trên LDA. Cụ thể, trên tập dữ liệu gốc, KNN đạt độ chính xác trung bình khoảng 86–89%, cao hơn Softmax Regression khoảng 3–6%. Tuy nhiên, khi giảm chiều dữ liệu bằng PCA, cả hai mô hình đều bị suy giảm hiệu suất do mất mát thông tin, trong đó Softmax Regression thể hiện sự ổn định hơn, giữ được độ chính xác quanh mức 78–80%, còn KNN giảm rõ rệt xuống còn 60–72%. Ngược lại, với LDA, hiệu năng của cả hai mô hình đều được phục hồi đáng kể và gần tương đương nhau, dao động quanh 83–84%, cho thấy LDA là phương pháp giảm chiều hiệu quả trong việc bảo toàn thông tin phân biệt lớp.

Xét về độ ổn định khi thay đổi biểu diễn dữ liệu, Softmax Regression có xu hướng giữ được hiệu năng khá ổn định. Là một mô hình tuyến tính với cơ chế tối ưu hóa rõ ràng, Softmax ít bị ảnh hưởng tiêu cực khi chiều dữ liệu giảm, đặc biệt là với LDA - nơi không gian được tối ưu theo hướng phân biệt các lớp. KNN thì ngược lại, phụ thuộc nhiều vào cấu trúc hình học của dữ liệu trong không gian đặc trưng, do đó hiệu năng của nó bị suy giảm đáng kể khi sử dụng PCA (vì PCA không đảm bảo duy trì khoảng cách giữa các lớp). Tuy nhiên, khi chuyển sang LDA - vốn bảo toàn biên phân lớp - hiệu năng của KNN lại phục hồi rõ rệt, thậm chí có thể vượt Softmax trong một số trường hợp.

Về khả năng phân biệt lớp, KNN thể hiện độ nhạy cao hơn đối với lớp dương tính (class 1), với giá trị Recall cao hơn, cho thấy mô hình có khả năng phát hiện các mẫu dương tốt hơn, dù đôi khi phải đánh đổi bằng Precision thấp hơn. Ngược lại, Softmax Regression có xu hướng dự đoán ổn định hơn giữa các lớp, nhưng đôi khi bỏ sót một số mẫu dương, thể hiện qua Recall thấp hơn và tỉ lệ False Negative cao hơn.

Cuối cùng, xét về hiệu quả tính toán, Softmax Regression có ưu thế rõ rệt nhờ thời gian huấn luyện nhanh, khả năng mở rộng tốt và tiêu tốn ít tài nguyên trong giai đoạn dự đoán, phù hợp với các hệ thống lớn hoặc các bài toán thời gian thực. Trong khi đó, KNN không cần giai đoạn huấn luyện phức tạp nhưng lại tốn kém chi phí tính toán khi dự đoán vì phải tính khoảng cách đến toàn bộ tập huấn luyện. Điều này làm cho KNN kém phù hợp với các tập dữ liệu có kích thước lớn hoặc yêu cầu phản hồi nhanh.

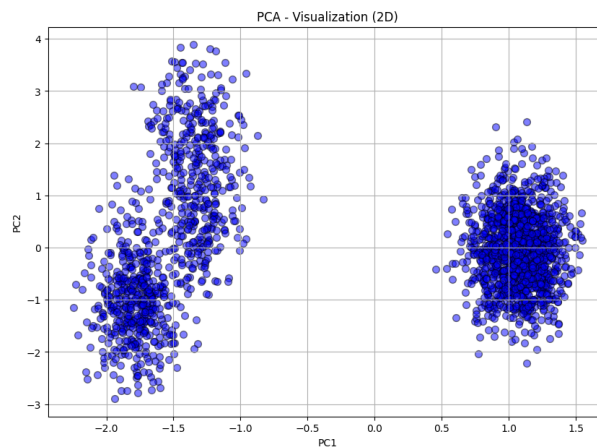
Tóm lại, có thể thấy rằng KNN phù hợp hơn với những bài toán có quy mô dữ liệu vừa phải, yêu cầu độ chính xác cao và khả năng phát hiện mẫu dương nhạy, trong khi Softmax Regression lại là lựa chọn tối ưu cho các ứng dụng đòi hỏi tốc độ, khả năng triển khai nhanh và hoạt động tốt trên không gian dữ liệu đã được giảm chiều hiệu quả như LDA.

7.4 Phân cụm Kmeans

K-Means là thuật toán phân cụm dựa trên khoảng cách. Thuật toán chia dữ liệu thành các cụm sao cho tổng bình phương khoảng cách từ các điểm đến tâm cụm là nhỏ nhất và những điểm dữ liệu trong 1 cụm phải có tính chất giống nhau, tương tự nhau.

Bởi vì thuật toán KMeans là thuật toán phân cụm không giám sát nên ta sẽ loại bỏ biến mục tiêu *Diagnosis* của dữ liệu vì bản chất khi phân cụm không giám sát thì ta sẽ không biết được dữ liệu sẽ chia thành bao nhiêu cụm là phù hợp nhất với dữ liệu. Do đó ta sử dụng chỉ số **Silhouette Score**, **Davies-Bouldin Score**, **Calinski-Harabasz Score** để xác định số cụm phù hợp nhất.

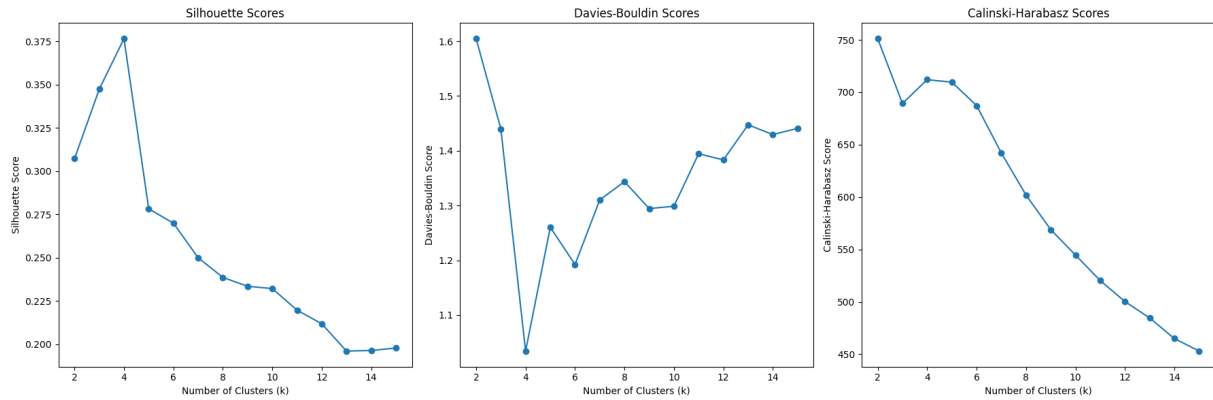
7.4.1 Kmeans với PCA



Hình 7.1: Hình ảnh trực quan Dữ liệu giảm chiều bằng PCA 2D [7]

Biểu đồ trực quan hóa dữ liệu PCA 2D cho thấy cho thấy các cụm dữ liệu vẫn “dính” nhau ở vùng biên: những đám mây điểm giao thoa, ranh giới giữa các cụm mờ nhạt, thậm chí chồng lấn cục bộ. Mật độ và kích thước từng cụm cũng không đồng đều - có cụm dày đặc, nén chặt, có cụm thưa thớt, trải rộng - tô đậm sự pha trộn và bất cân xứng trong cấu trúc dữ liệu.

Bởi vì chưa biết chọn số K nào phù hợp nhất cho thuật toán KMeans, nên thử các giá trị K từ 2 đến 15 để xem các chỉ số **Silhouette Score**, **Davies-Bouldin Score**, **Calinski-Harabasz Score** nào tốt nhất thì đó chính là số cụm K phù hợp nhất.



Hình 7.2: Hình ảnh chỉ số đánh giá với K chạy từ 2 đến 15 Kmeans+PCA [7]

K	Silhouette	Davies–Bouldin	Calinski–Harabasz
2	0.3073	1.6048	751.13
3	0.3474	1.4398	689.29
4	0.3766	1.0342	711.98
5	0.2783	1.2602	709.62
6	0.2700	1.1923	687.26
7	0.2500	1.3103	642.07
8	0.2386	1.3436	601.83
9	0.2335	1.2944	568.90
10	0.2322	1.2987	544.63
11	0.2197	1.3946	520.29
12	0.2119	1.3833	500.33
13	0.1961	1.4474	484.63
14	0.1964	1.4296	464.97
15	0.1978	1.4408	453.04

Bảng 7.19: Các chỉ số đánh giá Kmeans+PCA theo số cụm K [7]

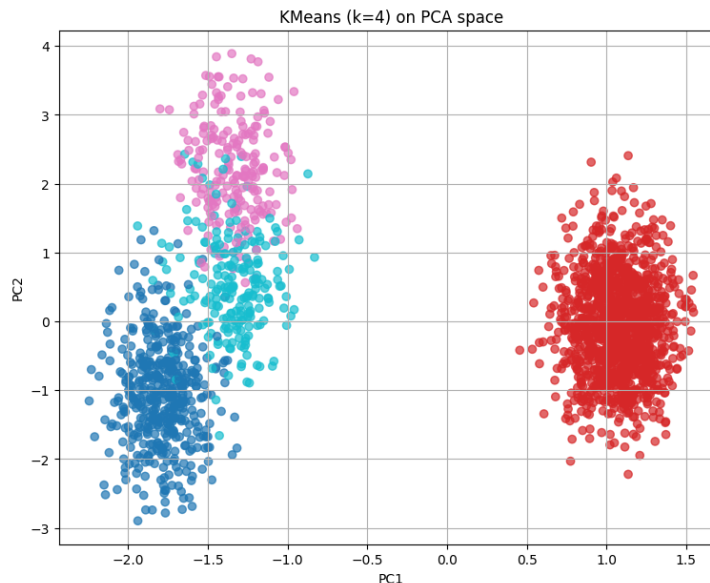
Dựa vào kết quả Bảng 7.19, thấy rằng ba chỉ số **Silhouette** = 0.3766, **Davies-Bouldin** = 1.0342 và **Calinski-Harabasz** = 711.98 cho thấy phương án $K = 4$ là tốt nhất trong phạm vi các giá trị K đã thử, cấu trúc phân cụm hiện tại:

- **Silhouette trung bình 0.3766** chỉ nằm ở mức “khá” (0.3–0.5). Điều này chứng tỏ *độ biệt lập giữa cụm và tính kết dính nội cụm vẫn còn yếu*, một phần không nhỏ điểm dữ liệu có thể nằm sát biên, thậm chí bị gán vào cụm

chưa thực sự phù hợp.

- **Davies–Bouldin 1.0342** thấp nhất trong các K khảo sát, xấp xỉ ngưỡng 1. Chỉ số này phản ánh:
 - **Nội cụm:** Các điểm dữ liệu trong cùng một cụm nằm khá sát nhau và tập trung xung quanh tâm cụm, không bị phân tán quá rộng. Các bệnh nhân trong cùng một nhóm có các đặc điểm khá tương đồng.
 - **Giữa các cụm:** Tâm của các cụm nằm đủ xa nhau. Thuật toán đã tìm ra các nhóm có sự khác biệt rõ ràng về vị trí trong không gian dữ liệu. Các nhóm được định hình tốt, sát nhau nhưng không bị trộn lẫn hỗn loạn.
- **Calinski–Harabasz 711.98** đạt kết quả khá tốt. Các điểm dữ liệu (bệnh nhân) trong cùng một cụm nằm rất sát nhau, tụ lại thành một khối đặc (dense) xung quanh tâm cụm. Tuy nhiên, CH không đánh giá trực tiếp độ chồng lấn biên, vì vậy phải đọc cùng với Silhouette và DBI để tránh ngộ nhận “cụm hoàn hảo”.

Tóm lại, *chọn $K = 4$ là tối ưu về mặt tương đối*, tuy nhiên "tốt về mặt toán học" chưa chắc đã đồng nghĩa với "tốt về mặt lâm sàng" điều này phản ánh bản chất phức tạp của bệnh Alzheimer.



Hình 7.3: Dữ liệu sau khi phân cụm KMeans+PCA với $K=4$ [7]

Cluster \ Output	0	1
0	308	146
1	815	463
2	122	84
3	144	67

Bảng 7.20: Ma trận tần suất giữa nhãn gốc (*Output*) và nhãn do K-means gán (*Cluster*) [7]

Dữ liệu gốc vốn đã bị mất cân bằng. Vì vậy, việc các cụm (Cluster) chứa nhiều nhóm 0 hơn nhóm 1 là điều hiển nhiên, vì nhóm 0 chiếm đa số trong quần thể chung. Từ ma trận tần suất giữa nhãn gốc (*Output*) và nhãn do K-means gán (*Cluster*) với $K = 4$, ta có tổng số quan sát là $N = 2149$. Các nhận xét chi tiết như sau:

- **Cụm 0** gồm 454 quan sát, trong đó 308 (67.8) mang nhãn gốc 0 và 146 (32.2) mang nhãn 1. Tỷ lệ này tương đương với tỷ lệ trung bình của toàn bộ tập dữ liệu (khoảng 64.6 là nhãn 0), cho thấy cụm này không thể hiện đặc trưng rõ rệt cho nhóm bệnh hay nhóm khỏe.
- **Cụm 1** là cụm lớn nhất chứa 1278 quan sát (chiếm 59.5 dữ liệu), với 815 (63.8) mang nhãn 0 và 463 (36.2) mang nhãn 1. Sự phân bố trong cụm này gần như trùng khớp hoàn toàn với phân bố gốc của dữ liệu, cho thấy thuật toán chưa tách được nhiều ở cụm này.
- **Cụm 2** có quy mô nhỏ với 206 quan sát. Đây là cụm có tỷ lệ nhãn 1 cao nhất trong 4 cụm (84 trường hợp, chiếm 40.8), tuy nhiên nhãn 0 vẫn chiếm đa số (122 trường hợp, chiếm 59.2). Dù có xu hướng gom nhóm bệnh nhân tốt hơn một chút, nhưng sự tách biệt là không đáng kể.
- **Cụm 3** bao gồm 211 quan sát, với 144 (68.2) nhãn 0 và 67 (31.8) nhãn 1. Cấu trúc của cụm này khá tương đồng với Cụm 0, thiên về nhóm không mắc bệnh nhưng độ tinh khiết (purity) vẫn thấp.

Các cụm đều phản ánh xu hướng chung của dữ liệu (Nhóm 0 luôn chiếm đa số $> 50\%$). Thuật toán K-Means là học không giám sát (unsupervised), nó gom nhóm dựa trên sự tương đồng về đặc trưng (như tuổi, chỉ số BMI, cholesterol...) chứ không cố gắng phân tách theo nhãn bệnh (Diagnosis). Việc các cụm vẫn trộn lẫn nhóm 0 và 1 chứng tỏ rằng các đặc trưng hiện tại chưa đủ mạnh để tách biệt rạch ròi người bệnh và người không mắc bệnh trên không gian dữ liệu.

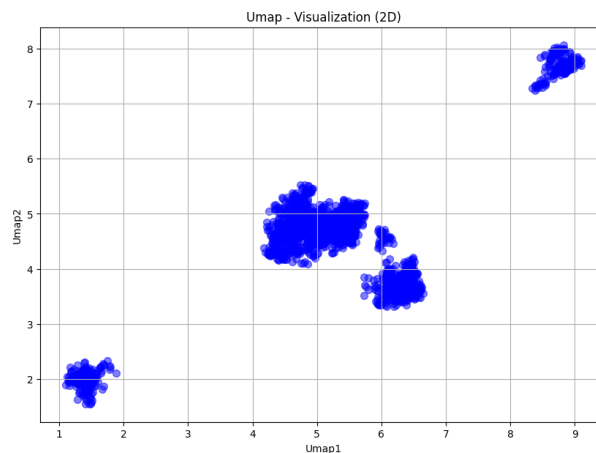
này. Độ chính xác (Accuracy) trong trường hợp này chỉ đơn giản là tỷ lệ của lớp đa số trong tập dữ liệu:

$$\text{Accuracy} = \frac{1389}{2149} \approx 0.646 \text{ (64.6\%)}.$$

Dù bị trộn lẫn, vẫn thấy được rằng Cụm 2 có tỷ lệ người bệnh cao hơn đáng kể (gần 41%) so với Cụm 3 (chỉ 32%). Điều này cho thấy rằng những bệnh nhân trong Cụm 2 có những đặc điểm sinh học (feature) rủi ro cao hơn, trong khi Cụm 3 tập trung những người có chỉ số an toàn hơn. Đó chính là giá trị của việc phân cụm tìm ra các nhóm rủi ro khác nhau dù nhãn bệnh chưa tách biệt hoàn toàn.

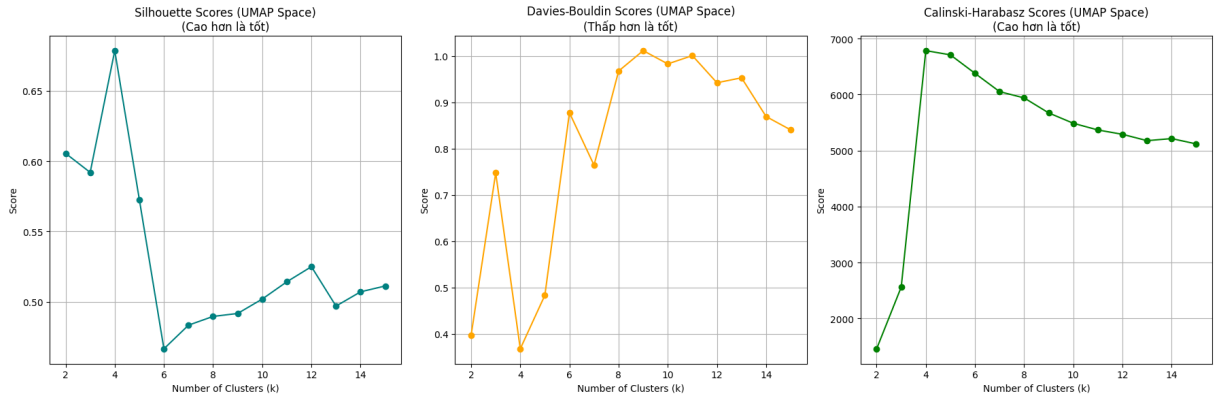
7.4.2 Kmean với Umap

Biểu đồ Umap (2 chiều) cho thấy không phải là một đám mây điểm hỗn độn (như PCA) mà tách thành các "hòn đảo" hoặc các khối (blobs) riêng biệt. Có thể thấy Umap thành công trong việc bẻ gãy các liên kết phi tuyến phức tạp của dữ liệu bệnh Alzheimer để gom những bệnh nhân thực sự giống nhau lại.



Hình 7.4: Hình ảnh Dữ liệu giảm chiều bằng Umap [7]

Bởi vì chưa biết chọn số K nào phù hợp nhất cho thuật toán KMeans, nên thử các giá trị K từ 2 đến 15 để xem các chỉ số **Silhouette Score**, **Davies-Bouldin Score**, **Calinski-Harabasz Score** nào tốt nhất thì đó chính là số cụm K phù hợp nhất.



Hình 7.5: Hình ảnh chỉ số đánh giá với K chạy từ 2 đến 15 Kmeans+Umap [7]

K	Silhouette	Davies–Bouldin	Calinski–Harabasz
2	0.6055	0.3968	1458.01
3	0.5919	0.7475	2561.43
4	0.6785	0.3674	6786.46
5	0.5726	0.4831	6710.77
6	0.4665	0.8776	6380.70
7	0.4833	0.7641	6053.90
8	0.4895	0.9675	5941.43
9	0.4916	1.0117	5673.08
10	0.5019	0.9830	5488.37
11	0.5141	1.0009	5368.61
12	0.5249	0.9422	5290.85
13	0.4969	0.9530	5177.93
14	0.5070	0.8692	5214.71
15	0.5112	0.8404	5119.64

Bảng 7.21: Các chỉ số đánh giá Kmeans+Umap theo số cụm K [7]

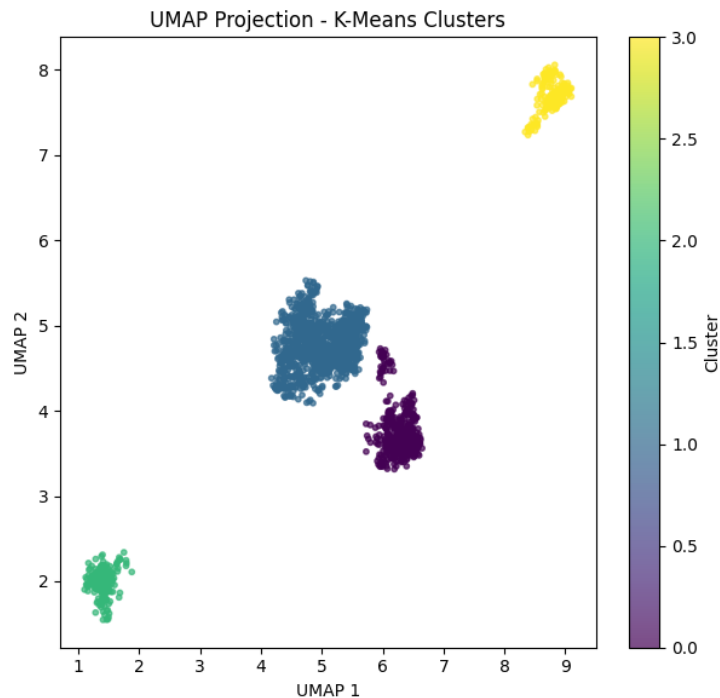
Dựa vào kết quả Bảng 7.21, ba giá trị $\text{Silhouette} = 0.6785$, $\text{Davies-Bouldin} = 0.3674$ và $\text{Calinski-Harabasz} = 6786.46$ cho thấy phương án $K = 4$ tối ưu nhất trong các mức K đã thử, trong đó:

- **Silhouette trung bình 0.6785** vượt mốc 0.5, hàm ý độ biệt lập giữa cụm cao và tính kết dính nội cụm tốt; chỉ một lượng nhỏ điểm nằm sát biên.
- **Davies–Bouldin 0.3674** nhỏ hơn 0.5 chứng tỏ khoảng cách giữa centroid các cụm lớn hơn đáng kể so với độ phân tán nội cụm, tức hai cụm hầu như

không chồng lấn.

- **Calinski–Harabasz** 6786.46 tăng không đều khi K tăng. Giá trị cao phản ánh nó có xu hướng phình ra khi thêm cụm, nên cần kết hợp Silhouette và DBI để tránh hiểu nhầm rằng “CH cao nhất” luôn tốt.

Tóm lại, *chọn* $K = 4$ là hợp lý vì đạt Silhouette cao nhất và Davies–Bouldin thấp nhất; các cụm được tách biệt tương đối rõ. Tuy nhiên, việc Calinski–Harabasz tăng giảm thất thường, tăng mạnh từ $K = 2$ đến $K = 4$ tức chia dữ liệu thành 3 rồi 4 cụm, cấu trúc dữ liệu trở nên rõ ràng hơn. Nếu cố chia nhỏ thêm (thành 5, 6 cụm...), là đang "cắt vụn" các cụm tự nhiên, làm giảm độ kết tụ của chúng mà không tăng thêm được bao nhiêu độ tách biệt, dẫn đến chỉ số CH bị giảm.



Hình 7.6: Dữ liệu sau khi phân cụm KMeans+Umap với $K=4$ [7]

Từ ma trận tần suất giữa nhãn gốc (*Output*) và nhãn do K-means (sau giảm chiều bằng Umap) gán (*Cluster*), có thể rút ra các nhận xét sau:

- **Cụm 0** gồm 453 quan sát, trong đó 307 (67.8) mang nhãn gốc 0 và 146 (32.2) mang nhãn 1. Tỷ lệ này tương đương với tỷ lệ trung bình của toàn bộ tập dữ liệu (khoảng 64.6 là nhãn 0), cho thấy cụm này không thể hiện đặc trưng rõ rệt cho nhóm bệnh hay nhóm khỏe.

Cluster \ Output	0	1
0	307	146
1	816	463
2	144	67
3	122	84

Bảng 7.22: Ma trận tần suất giữa nhãn gốc (*Output*) và nhãn do K-means gán (*Cluster*) [7]

- **Cụm 1** là nhóm "trung bình", chứa đa số những người có các chỉ số sức khỏe phổ biến. Mô hình khó phân tách nhóm này sâu hơn vì các đặc điểm của họ không quá nổi trội về phía "rất khỏe" hay "rất yếu".
- **Cụm 2** bao gồm 211 quan sát, với 144 (68.2) nhãn 0 và 67 (31.8) nhãn 1. Cấu trúc của cụm này khá tương đồng với Cụm 0, thiên về nhóm không mắc bệnh nhưng độ tinh khiết (purity) vẫn thấp.
- **Cụm 3** là cụm đáng chú ý nhất. Tỷ lệ mắc bệnh lên tới 40.8%, cao hơn đáng kể so với mức trung bình của quần thể (35.4%) và cao hơn nhiều so với Cụm 2 (31.8%). Dù có xu hướng gom nhóm bệnh nhân tốt hơn một chút, nhưng sự tách biệt là không đáng kể.

Mô hình phân loại không rạch ròi nhánh "Nhóm bệnh" và "Nhóm không bệnh" trên dữ liệu y tế phức tạp nhưng nó đã tách được các tầng rủi ro.

7.4.3 Kết luận

Với đặc trưng dữ liệu có nhiều điểm nhiễu và vùng chồng lấn, so sánh hai cách tiền xử lý cho thấy K-means kết hợp Umap đem lại kết quả tốt hơn so với K-means kết hợp PCA. Sau khi chiếu lên trục UMAP1, UMAP2 thể hiện sự kết dính nội cụm tốt và ranh giới liên cụm rõ ràng. Ngược lại, khi giảm chiều bằng PCA các chỉ số thấp hơn UMAP cho thấy dữ liệu không phân bố theo đường thẳng, khi PCA cố ép dữ liệu làm mất cấu trúc khiến các điểm bệnh nhân trộn lẫn. Như vậy, trong bối cảnh dữ liệu chồng chéo UMAP cung cấp trực phân tách tuyến tính tối ưu. Nó hiểu dữ liệu y sinh phức tạp thường nằm trên các bề mặt cong xoắn vặn. UMAP đã "trải phẳng" bề mặt này ra, giúp bộc lộ rõ các cụm mà PCA không thể thấy.

7.5 DBScan Clustering

Density-Based Spatial Clustering of Applications with Noise

là phương pháp phân cụm dựa trên mật độ, không đòi hỏi phải ấn định trước số cụm K như K-means. Ý tưởng cốt lõi của DBSCAN xoay quanh hai siêu tham số: *bán kính lân cận ε* (epsilon) và *số điểm lân cận tối thiểu $minPts$* .

Trong báo cáo này, hai siêu tham số của DBSCAN được lựa chọn theo quy trình *dò-tối-ưu* thay vì cố định theo kinh nghiệm. Cụ thể, **minPts** (**min_samples**) quét qua các giá trị 3,5,10; với mỗi giá trị, ta tạm tính ε bằng *k-distance* và chạy DBSCAN, sau đó đánh giá chất lượng phân cụm bằng hai thước đo là *Silhouette* và *Calinski-Harabasz*. Giá trị $minPts^*$ cho *Silhouette* cao nhất (đồng thời CH cũng cao) được chọn làm ngưỡng mật độ tối ưu. Khi $minPts^*$ đã cố định, ε cuối cùng được xác định lại bằng đường *k-distance* (với $k = minPts^* - 1$) và thuật toán *KneeLocator* để lấy đúng “điểm gãy” (elbow). Cách tiếp cận này tự động chọn cặp $\langle minPts^*, \varepsilon \rangle$ tốt nhất cho dữ liệu, giảm phụ thuộc chủ quan và cho phép dung hòa giữa độ kết dính nội cụm và độ tách biệt liên cụm.

7.5.1 DBScan với PCA

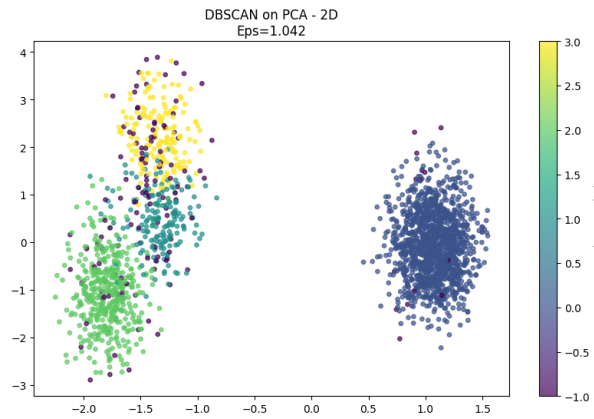
Sau khi dò tham số tự động, **DBSCAN** đạt cặp giá trị tối ưu $\langle \varepsilon, minPts \rangle = \langle 1.042, 10 \rangle$ Kết quả cụm có các đặc trưng:

- **Số cụm phát hiện:** 4 cụm và điểm nhiễu (label -1) chỉ chiếm tỉ lệ gồm 122 điểm nhiễu.
- **Silhouette Score:** 0.338 cho thấy mức gắn kết nội cụm và độ tách giữa cụm ở mức *khá tốt*, ranh giới cụm còn chồng lấn.

Điều này cho thấy, khi giảm chiều tuyến tính bằng **PCA**, DBSCAN phải dùng ngưỡng mật độ cao ($minPts = 10$) và bán kính khá rộng ($\varepsilon \approx 1.04$) để gom nhóm, sinh ra nhiều cụm kích thước nhỏ cùng độ thuần chưa cao. Nguyên nhân là PCA không bảo toàn các quan hệ phi tuyến trong dữ liệu, làm “dẹt” khoảng cách thực sự giữa những nhóm cong hoặc nghiêng, dẫn tới cụm chưa thật sự chặt chẽ so với kỳ vọng.

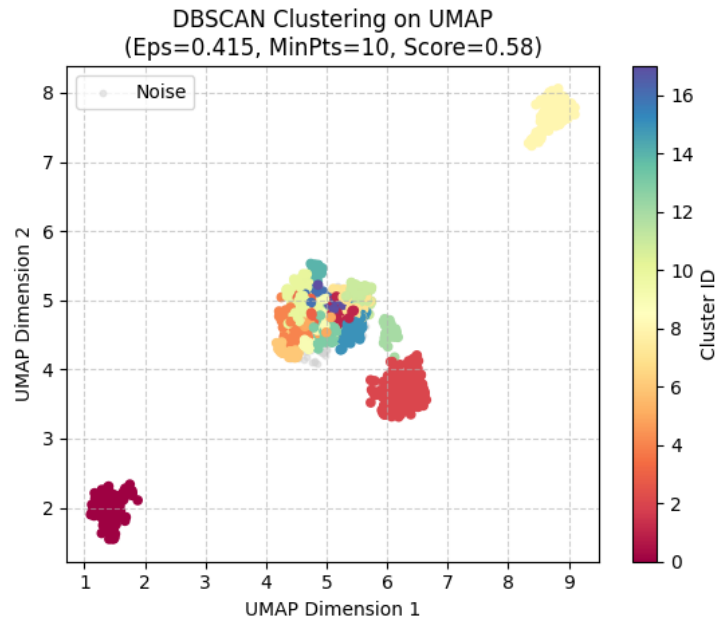
7.5.2 DBScan với Umap

Sau khi quét **min_samples** với 3 giá trị 3,5,10; đường *Silhouette* cho thấy giá trị $minPts^* = 10$ đồng thời đạt *Silhouette* cao nhất. Tại $minPts^* = 10$ thuật toán *k-distance* (Elbow Method) xác định điểm gãy ở $\varepsilon \approx 0.415$. Với cặp tham số



Hình 7.7: Số cụm và điểm nhiễu của DBSCAN+PCA [7]

tối ưu $\langle \varepsilon, \min Pts^* \rangle = \langle 0.415, 10 \rangle$, DBSCAN tìm ra **nhiều cụm** và một lượng nhiễu khá lớn.



Hình 7.8: Số cụm và điểm nhiễu của DBSCAN+Umap [7]

- **Silhouette Score:** 0.579 cho thấy độ kết dính nội cụm và độ tách biệt liên cụm khá tốt, vượt xa giá trị 0.338 thu được trong không gian PCA.
- **Số điểm nhiễu:** có khá nhiều điểm nhiễu 264 điểm, chứng tỏ Umap đã “phân rã” hầu hết dữ liệu ra nhiều cụm, chỉ có 3-4 cụm phân tách rõ thành

các cụm riêng biệt còn lại tập hợp khá sát nhau thành 1 vùng mật độ lớn để DBSCAN nhận diện.

Như vậy, kết hợp Umap với DBSCAN tạo ra các cụm rõ ràng hơn, cụm hóa chắc chắn hơn (Silhouette > 0.5) và giảm tối đa điểm nhiễu. Umap có xu hướng kéo tách các nhóm dữ liệu khác biệt ra xa nhau và gom chặt các điểm tương đồng lại. Do đó, DBSCAN trên UMAP tạo ra các cụm "sạch" hơn.

7.5.3 Kết luận

Tóm lại, *DBSCAN+Umap* cho thấy hiệu quả vượt trội trong việc bóc tách cấu trúc dữ liệu phức tạp (phi tuyến), tạo ra ranh giới cụm sắc nét với Silhouette Score cao (> 0.5). Tuy nhiên, do đặc tính co cụm quá mạnh của UMAP làm đứt gãy các liên kết dữ liệu liên tục, dẫn đến việc hình thành nhiều cụm nhỏ và gia tăng điểm nhiễu cục bộ. Ngược lại, *DBSCAN+PCA* bảo toàn tốt cấu trúc toàn cục và phương sai của dữ liệu gốc nhưng lại hạn chế trong việc phân tách các biên giới phi tuyến. Kết quả cho thấy các cụm bị dính chùm, ranh giới mờ nhạt và độ đặc thấp hơn so với UMAP.

7.6 So sánh K-means và DBSCAN

Thuật toán	Silhouette	Davies–Bouldin	Calinski–Harabasz
K-means+PCA	0.3766	1.0342	711.98
K-means+Umap	0.6785	0.3674	6786.46
DBSCAN+PCA	0.338	—	457.31
DBSCAN+Umap	0.579	—	5141.50

Bảng 7.23: Chỉ số đánh giá phân cụm của bốn thực nghiệm [7]

Nhận xét

- **K-means kết hợp PCA:** Silhouette trung bình (0.38), $DBI = 1.03$ và $CH \approx 711.98$ cho thấy cụm hoá chấp nhận được nhưng biên vẫn chông lún và không phát hiện ngoại lai.
- **K-means kết hợp Umap:** Cho Silhouette cao nhất (0.68), DBI thấp nhất (0.37) và CH vượt trội ≈ 6786.46 cho thấy ba cụm rõ ràng, kết dính chặt, phù hợp khi ranh giới tuyến tính đã tồn tại, ngoài ra một vùng mật độ chứa nhiều cụm nhỏ sát nhau không được phân tách riêng biệt.

- **DBSCAN kết hợp PCA:** Silhouette thấp nhất (0.34), CH chỉ 457.31 cho thấy phải sinh các cụm nhỏ, ít nhiều nhưng chất lượng không được tốt, chứng tỏ PCA làm “dẹt” khoảng cách phi tuyến.
- **DBSCAN kết hợp Umap:** Silhouette 0.58 tốt hơn PCA, đồng thời lọc được nhiều ngoại lai. Nhưng CH khá tốt (5141.50) các cụm được tạo ra có khoảng cách tách biệt rõ ràng và các điểm trong cùng một cụm nằm rất gần nhau. UMAP đã gom các điểm dữ liệu lại thành những khối rất đặc, giúp thuật toán DBSCAN dễ dàng nhận diện "lõi" cụm.

Kết luận chung

1. *K-means* thích hợp khi cấu trúc lớp rõ và cần cụm *đầy đủ* (không quan tâm điểm nhiễu). Giảm chiều bằng Umap giúp K-means đạt chất lượng cao nhất trong các thử nghiệm.
2. *DBSCAN* hữu ích khi muốn phát hiện ngoại lai hoặc cụm hình dạng bất kỳ. Trên trục Umap, phương pháp này chứng minh khả năng tạo ra các cụm có độ đặc rất cao và ranh giới tách biệt rõ rệt.
3. Nếu ưu tiên độ tinh khiết của cụm và sự tách biệt rõ ràng giữa các nhóm bệnh/đối tượng, UMAP là lựa chọn tối ưu. PCA chỉ nên được cân nhắc khi cần bảo toàn nguyên vẹn phương sai dữ liệu gốc mà không được phép loại bỏ bất kỳ quan sát nào. PCA kết hợp DBSCAN tỏ ra kém hiệu quả nhất do làm mất thông tin phi tuyến, dẫn tới Silhouette thấp.

7.7 Kết quả phân loại với SVM

Trong phần này, mô hình Support Vector Classifier (SVC) được đánh giá trên ba chiến lược biểu diễn dữ liệu khác nhau: (i) dữ liệu gốc chưa giảm chiều, (ii) dữ liệu sau khi giảm chiều bằng PCA, và (iii) dữ liệu sau khi giảm chiều bằng LDA. Với mỗi trường hợp, RandomizedSearchCV được sử dụng để tối ưu siêu tham số, và hiệu năng được đánh giá trên tập kiểm tra với nhiều tỉ lệ chia train–test khác nhau.

7.7.1 SVC trên dữ liệu gốc (không giảm chiều)

Trên dữ liệu gốc, SVC đạt được hiệu năng cao và ổn định nhất trong ba chiến lược. Với các tỉ lệ `test_size` khác nhau (0.2, 0.3 và 0.4), kernel **RBF** chiếm ưu thế trong hầu hết các cấu hình tối ưu.

Kết quả tốt nhất đạt được với `test_size = 0.3`, trong đó:

- Độ chính xác kiểm tra đạt **85.1%**,
- Kernel tối ưu: **RBF**,
- Tham số phạt lớn ($C \approx 310$) và γ nhỏ ($\gamma \approx 0.001$).

Ma trận nhầm lẫn và các chỉ số precision-recall cho thấy mô hình có khả năng phân loại cân bằng giữa hai lớp, với lớp 0 đạt precision và recall cao (khoảng 0.88), trong khi lớp 1 có hiệu năng thấp hơn nhưng vẫn ổn định (khoảng 0.79).

Nhận xét: Linear kernel và rbf kernel cho kết quả rất cạnh tranh, rbf nhỉnh hơn một chút. Việc giữ nguyên toàn bộ không gian đặc trưng giúp mô hình khai thác đầy đủ thông tin phân biệt giữa các lớp, dẫn đến hiệu năng tổng thể cao nhất.

7.7.2 SVC trên dữ liệu giảm chiều bằng PCA

Trong trường hợp giảm chiều bằng PCA với $n_components = 15$, hiệu năng của SVC giảm đáng kể so với dữ liệu gốc. Với mọi tỉ lệ chia tập, độ chính xác kiểm tra chỉ dao động trong khoảng **77% – 82%**.

Cấu hình tốt nhất đạt được với `test_size = 0.2`, trong đó:

- Độ chính xác kiểm tra đạt khoảng **81.6%**,
- Kernel tối ưu vẫn là **RBF**,
- Tham số C lớn (≈ 310), tương tự dữ liệu gốc.

Tuy nhiên, các chỉ số recall cho lớp 1 giảm rõ rệt (khoảng 0.60), cho thấy mô hình có xu hướng thiên lệch về lớp 0 sau khi giảm chiều.

Nhận xét: PCA là phương pháp không giám sát, tối ưu theo phương sai toàn cục, nên các thành phần giữ lại không nhất thiết là những thành phần phân biệt tốt nhất giữa các lớp (15 thành phần chính đầu chỉ giữ được 85% thông tin ban đầu). Điều này dẫn đến việc mất thông tin quan trọng cho bài toán phân loại, đặc biệt đối với lớp thiểu số (lớp 1).

7.7.3 SVC trên dữ liệu giảm chiều bằng LDA

Với LDA (giảm xuống 1 thành phần), hiệu năng của SVC được cải thiện so với PCA nhưng thấp hơn so với dữ liệu gốc. Độ chính xác kiểm tra dao động quanh **81% – 83%**.

Kết quả tốt nhất đạt được với `test_size = 0.4`, trong đó:

- Độ chính xác kiểm tra đạt **83.0%**,
- Kernel tối ưu: **Polynomial**,
- Tham số $C \approx 2.4$ và $\gamma \approx 0.043$.

Điểm đáng chú ý là kernel tối ưu trong không gian LDA một chiều không còn là RBF mà chuyển sang polynomial, cho thấy ranh giới quyết định trong không gian đã bị đơn giản hóa đáng kể.

Nhận xét: LDA là phương pháp giảm chiều có giám sát, tập trung vào khả năng phân tách lớp, nên giữ lại được nhiều thông tin phân biệt hơn PCA. Tuy nhiên, việc giảm dữ liệu xuống chỉ một chiều vẫn làm mất các mối quan hệ phi tuyến phức tạp giữa các đặc trưng, dù vậy, nó vẫn đạt được kết quả khá cao so với dữ liệu gốc, điều này khá bất ngờ so với dự đoán.

7.7.4 So sánh ba chiến lược biểu diễn dữ liệu

- **Dữ liệu gốc** cho hiệu năng cao nhất, với độ chính xác tối đa đạt **85.1%**.
- **LDA + SVC** cho kết quả tốt hơn PCA, nhưng vẫn kém dữ liệu gốc khoảng 2–3%.
- **PCA + SVC** cho hiệu năng thấp nhất, đặc biệt suy giảm rõ rệt ở recall của lớp 1.

Về kernel:

- Kernel **RBF** chiếm ưu thế trên dữ liệu gốc và PCA,
- Kernel **Polynomial** trở nên hiệu quả hơn trong không gian LDA một chiều.

7.7.5 Kết luận

Từ các thí nghiệm trên, có thể rút ra các kết luận sau:

- SVC hoạt động hiệu quả nhất khi được huấn luyện trên dữ liệu gốc, cho thấy bài toán có cấu trúc phi tuyến phức tạp và cần không gian đặc trưng đầy đủ (kết quả giảm chiều cũng cho thấy giới hạn của PCA và LDA trên bộ dữ liệu này, UMAP có giám sát cho kết quả giảm chiều rất tốt nhưng không hiệu quả như vậy trong việc biến đổi dữ liệu cho SVM - thử nghiệm ban đầu cho thấy SVM với UMAP thấp hơn với PCA và LDA). Tuy vậy, kết quả của SVM trên dữ liệu giảm chiều bằng LDA không quá thấp so với trên dữ liệu gốc, điều này có chút bất ngờ so với dự đoán ban đầu (dựa vào hiệu quả hạn chế của LDA trong việc giảm chiều trước đây).

- Giảm chiều bằng PCA hoặc LDA làm suy giảm độ chính xác và đặc biệt là tăng thời gian chạy mô hình SVC của sklearn, đặc biệt lâu hơn với PCA (n-components=15).
- Nếu mục tiêu là hiệu năng phân loại tối đa, nên **ưu tiên SVC với kernel RBF hoặc linear trên dữ liệu gốc**.
- Kernel linear có vẻ khá hiệu quả và vị thế ổn định trong trường hợp này, đặc biệt cho dữ liệu gốc và dữ liệu PCA (test accuracy không kém rbf bao nhiêu).

Chúng ta có thể thực hiện GridSearchCV tinh chỉnh sâu hơn quanh các giá trị $C \approx 300$ và $\gamma \approx 0.001$ cho kernel RBF và khai thác tiềm năng của linear kernel (có kết quả rất cạnh tranh với rbf trong nhiều trường hợp) để tìm kiếm mô hình tốt và đơn giản hơn nữa.

7.8 So sánh hiệu năng giữa các mô hình SVM, KNN và Softmax Regression

Trong phần này, ba mô hình phân loại phổ biến gồm Support Vector Machine (SVM), K-Nearest Neighbors (KNN) và Softmax Regression được so sánh một cách hệ thống trên ba chiến lược biểu diễn dữ liệu: dữ liệu gốc, dữ liệu giảm chiều bằng PCA và dữ liệu giảm chiều bằng LDA. Các mô hình được đánh giá dựa trên độ chính xác (accuracy), độ cân bằng giữa các lớp (precision, recall, F1-score).

7.8.1 So sánh trên dữ liệu gốc

Trên dữ liệu gốc, KNN cho kết quả tốt nhất trong ba mô hình. Với các tỷ lệ chia dữ liệu khác nhau, độ chính xác của KNN dao động từ **86% đến 89%**, trong đó cấu hình tốt nhất đạt khoảng **89%** với $K = 11$ hoặc $K = 15$ tùy theo tỷ lệ chia. Các chỉ số precision và recall của hai lớp đều ở mức cao, cho thấy mô hình có khả năng phân loại cân bằng.

SVM đạt hiệu năng thấp hơn KNN nhưng vẫn ổn định, với độ chính xác dao động trong khoảng **83% đến 85%**. Kernel RBF với tham số phạt lớn ($C \approx 300$) cho kết quả tốt nhất, phản ánh tính chất phi tuyến phức tạp của dữ liệu. Tuy nhiên, recall của lớp 1 vẫn thấp hơn lớp 0, cho thấy SVM nhạy hơn với sự mất cân bằng lớp.

Softmax Regression cho kết quả thấp nhất trên dữ liệu gốc, với độ chính xác khoảng **83%–84%**. Mặc dù các chỉ số giữa hai lớp khá cân bằng, mô hình tuyến tính này không khai thác được các mối quan hệ phi tuyến như SVM hay KNN.

Nhận xét: Dữ liệu gốc chứa nhiều thông tin phân biệt, và các mô hình phi tuyến hoặc dựa trên lân cận (đặc biệt là KNN) tận dụng tốt nhất không gian đặc trưng đầy đủ.

7.8.2 So sánh trên dữ liệu giảm chiều bằng PCA

Sau khi giảm chiều bằng PCA, hiệu năng của KNN suy giảm mạnh. Độ chính xác chỉ còn khoảng **69%–72%**, với sự mất cân bằng rõ rệt giữa hai lớp, đặc biệt là precision thấp cho lớp 1. Điều này cho thấy KNN phụ thuộc nhiều vào cấu trúc hình học ban đầu của dữ liệu.

SVM trên dữ liệu PCA đạt độ chính xác cao hơn KNN, dao động trong khoảng **77%–82%**. Kernel RBF vẫn là lựa chọn tối ưu trong hầu hết các trường hợp, tuy nhiên recall của lớp 1 giảm đáng kể, phản ánh việc mất thông tin phân biệt quan trọng trong quá trình giảm chiều.

Softmax Regression cho kết quả tương đối ổn định trên dữ liệu PCA, với độ chính xác khoảng **82%–84%**. Các chỉ số precision và recall giữa hai lớp khá cân bằng, cho thấy mô hình tuyến tính hoạt động tốt hơn KNN và tương đương hoặc nhỉnh hơn SVM trong không gian PCA.

Nhận xét: PCA với 15 thành phần chính chỉ giữ được 85% thông tin, dẫn đến sự suy giảm hiệu năng, đặc biệt đối với các mô hình dựa vào khoảng cách như KNN.

7.8.3 So sánh trên dữ liệu giảm chiều bằng LDA

Trên dữ liệu giảm chiều bằng LDA, hiệu năng của cả ba mô hình được cải thiện so với PCA. KNN đạt độ chính xác khoảng **81%–83%**, với các chỉ số precision và recall cân bằng hơn so với trường hợp PCA.

SVM đạt độ chính xác ổn định quanh mức **83%**, với kernel Polynomial hoặc RBF cho kết quả tốt nhất tùy theo tỷ lệ chia dữ liệu. So với PCA, khả năng phân loại lớp 1 của SVM được cải thiện đáng kể.

Softmax Regression cho kết quả rất ổn định trên dữ liệu LDA, với độ chính xác khoảng **83%–84%**, các chỉ số macro và weighted average đều cao và cân bằng giữa hai lớp.

Nhận xét: LDA là phương pháp giảm chiều có giám sát, giúp giữ lại thông tin phân biệt giữa các lớp, hiệu năng của cả ba mô hình được cải thiện, đặc biệt là các mô hình tuyến tính như Softmax Regression.

7.8.4 Kết luận

Từ các kết quả thực nghiệm, có thể rút ra một số kết luận chính như sau:

- **KNN trên dữ liệu gốc** cho hiệu năng cao nhất, phù hợp khi không gian đặc trưng đầy đủ.
- **SVM với kernel RBF** là lựa chọn mạnh mẽ và ổn định, có thể dùng linear kernel nếu chạy trên toàn dữ liệu gốc, kết quả tương tự rbf nhưng đơn giản hơn.
- **Softmax Regression** hoạt động tốt và rất ổn định trong các không gian đã được giảm chiều (đặc biệt là LDA).

Khuyến nghị:

- Nếu mục tiêu là tối đa hóa độ chính xác, nên ưu tiên **KNN hoặc SVM trên dữ liệu gốc**.
- Trong trường hợp giảm chiều, **LDA kết hợp với Softmax hoặc SVM** là lựa chọn hợp lý.
- Nên tiếp tục thử nghiệm parameter tuning hoặc các mô hình khác để so sánh và tìm ra mô hình tốt nhất.

7.9 Chuyển bài toán về dạng hồi quy với Softmax

7.9.1 Lý thuyết

Softmax Regression

Softmax Regression là một mô hình phân loại đa lớp, mở rộng từ Logistic Regression, sử dụng hàm Softmax để tính xác suất cho mỗi lớp. Với K lớp, xác suất của lớp k được tính bằng:

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}},$$

trong đó $z_k = w_k^T x + b_k$ là đầu vào tuyến tính, với w_k và b_k lần lượt là trọng số và bias, x là vector đặc trưng.

Chuyển thành bài toán hồi quy

Thay vì dự đoán nhãn, xác suất đầu ra $P(y = k | x)$ từ Softmax được sử dụng làm mục tiêu cho mô hình hồi quy tuyến tính:

$$\hat{y} = w^T x + b,$$

sai số được tối ưu bằng MSE. Với bài toán nhị phân (0 và 1), hai xác suất $P(y = 0 | x)$ và $P(y = 1 | x)$ được huấn luyện riêng biệt.

Giảm chiều dữ liệu

- PCA: Giảm chiều bằng cách giữ lại các thành phần chính có phương sai lớn nhất, thường chọn số thành phần để giữ 90% phương sai.
- Trong bài toán chuyển về hồi quy, ta giảm số chiều khoảng 1/3 so với dữ liệu gốc.

Đánh giá mô hình

Hiệu suất của mô hình hồi quy được đánh giá thông qua ba chỉ số chính: MSE, MAE, và R^2 . Các chỉ số này giúp đo lường độ chính xác và khả năng tổng quát hóa của mô hình khi dự đoán xác suất đầu ra từ Softmax Regression.

- **MSE (Mean Squared Error - Sai số bình phương trung bình):**

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

MSE đo lường trung bình bình phương của các sai số giữa giá trị thực y_i (xác suất thực tế) và giá trị dự đoán $\hat{y}_i$ (xác suất dự đoán bởi mô hình hồi quy). Giá trị MSE càng nhỏ thì mô hình càng chính xác. MSE nhạy cảm với các sai số lớn (outliers) và đơn vị của nó là bình phương của đơn vị gốc.

- **MAE (Mean Absolute Error - Sai số tuyệt đối trung bình):**

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

MAE đo lường trung bình giá trị tuyệt đối của các sai số giữa giá trị thực y_i và giá trị dự đoán $\hat{y}_i$. Không giống MSE, MAE không bình phương sai số, do đó ít nhạy cảm hơn với các sai số lớn và dễ diễn giải hơn. Giá trị MAE

càng nhỏ thì mô hình càng chính xác, phản ánh mức độ sai lệch trung bình của dự đoán.

- R^2 (Coefficient of Determination - Hệ số xác định):

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

R^2 đo lường tỷ lệ phương sai của biến phụ thuộc (xác suất thực tế y_i) được giải thích bởi mô hình hồi quy, với $\bar{y}$ là giá trị trung bình của y_i . Giá trị R^2 nằm trong khoảng $[0, 1]$, càng gần 1 thì mô hình càng tốt, nghĩa là dự đoán của mô hình giải thích được nhiều biến thiên trong dữ liệu. Nếu R^2 gần 0, mô hình không phù hợp để giải thích dữ liệu. Trong một số trường hợp hiếm, R^2 có thể âm nếu mô hình dự đoán tệ hơn so với việc chỉ sử dụng giá trị trung bình $\bar{y}$.

7.9.2 Các bước triển khai mô hình

Phần này trình bày chi tiết các bước triển khai mô hình chuyển đổi Softmax Regression thành bài toán hồi quy trên tập dữ liệu khách hàng hàng không đã được xử lý. Dữ liệu đã được giảm chiều (gốc, PCA) và chia thành các tập train-test theo các tỉ lệ 8:2, 7:3, và 6:4. Quá trình bao gồm huấn luyện Softmax Regression, chuyển sang hồi quy, và đánh giá hiệu suất.

Bước 1: Chuẩn bị dữ liệu đã xử lý

- **Dữ liệu sẵn có:** Dữ liệu đã được xử lý, bao gồm ba loại: dữ liệu gốc, dữ liệu giảm chiều bằng PCA.
- **Chia tập dữ liệu:** Dữ liệu đã được chia sẵn thành tập huấn luyện (train) và tập kiểm tra (test) với các tỉ lệ 8:2, 7:3, và 6:4.
- **Kiểm tra dữ liệu:** Đảm bảo dữ liệu đã được chuẩn hóa và sẵn sàng để huấn luyện mô hình.

Bước 2: Huấn luyện Softmax Regression

- **Khởi tạo mô hình:** Sử dụng LogisticRegression từ scikit-learn với tham số `multi_class='multinomial'` để huấn luyện Softmax Regression.
- **Huấn luyện trên từng loại dữ liệu:** Huấn luyện riêng trên dữ liệu gốc, PCA, với từng tỉ lệ chia 8:2, 7:3, và 6:4.

- **Lấy xác suất:** Sử dụng `predict_proba` để lấy xác suất đầu ra cho phân lớp 1 trên tập train và test.

Bước 3: Chuyển thành bài toán hồi quy

- **Huấn luyện mô hình hồi quy:**
 - Sử dụng `LinearRegression` từ `scikit-learn`.
 - Huấn luyện một mô hình dự đoán xác suất lớp 1.
 - Huấn luyện trên tập train của từng loại dữ liệu (gốc, PCA).

Bước 4: Đánh giá hiệu suất

- **Dự đoán trên tập test:** Sử dụng mô hình hồi quy để dự đoán xác suất lớp 1 trên tập test.
- **Tính toán chỉ số:** Tính MSE, MAE, và R^2 cho từng phân lớp, trên từng loại dữ liệu và tỉ lệ chia, bằng các hàm từ `scikit-learn` (`mean_squared_error`, `mean_absolute_error`, `r2_score`).
- **Lưu kết quả:** Tổng hợp kết quả vào bảng để so sánh giữa các hướng dữ liệu đã nêu ở trên và các tỉ lệ chia.

7.9.3 Dữ liệu gốc

Kết quả trên dữ liệu gốc với các tỉ lệ chia 8:2, 7:3, và 6:4 được trình bày trong bảng dưới đây:

Tỉ lệ	Phân lớp	MAE	MSE	R^2
8:2	1	0.0746	0.0079	0.9228
7:3	1	0.0952	0.0127	0.8937
6:4	1	0.1024	0.0146	0.8824

Bảng 7.24: Kết quả trên dữ liệu gốc với các tỉ lệ chia khác nhau

Nhận xét : Dữ liệu gốc đạt R^2 khá cao (0.8824-0.9228), với MSE (0.0079-0.0146) và MAE (0.0746-0.1024) ở mức thấp, cho thấy mô hình hồi quy có hiệu suất ổn định và khả năng dự đoán xác suất tốt. Tỉ lệ 8:2 cho kết quả tốt nhất ($R^2=0.9228$), vượt trội hơn so với 7:3 ($R^2=0.8937$) và 6:4 ($R^2=0.8824$), có thể do sự cân bằng giữa kích thước tập huấn luyện và tập kiểm tra. Tỉ lệ 6:4 có hiệu suất thấp nhất, phản ánh tác động của tập huấn luyện nhỏ hơn. Kết quả

phân lớp 1 là nhất quán, phù hợp với tính đối xứng của xác suất trong mô hình Softmax Regression.

7.9.4 Dữ liệu giảm chiều bằng PCA

Kết quả trên dữ liệu giảm chiều theo phương pháp PCA với các tỉ lệ chia 8:2, 7:3, và 6:4 được giảm chiều 1/3 so với chiều của dữ liệu gốc được trình bày trong bảng dưới đây:

Tỉ lệ	Phân lớp	MAE	MSE	R^2
8:2	1	0.0025	0.00001	0.9998
7:3	1	0.0034	0.00002	0.9997
6:4	1	0.0024	0.00001	0.9998

Bảng 7.25: Kết quả trên dữ liệu PCA với các tỉ lệ chia khác nhau

Nhận xét:

Dữ liệu PCA sau khi giảm chiều được sử dụng để xây dựng mô hình hồi quy Softmax, cho kết quả rất cao với R^2 dao động trong khoảng 0.9997 – 0.9998, MSE rất thấp ($\approx 0.000008 - 0.000016$) và MAE nhỏ ($\approx 0.0024 - 0.0034$), phản ánh khả năng dự đoán gần như hoàn hảo trên tập kiểm tra. Hiệu suất mô hình ổn định với các tỉ lệ chia dữ liệu 8:2, 7:3 và 6:4, trong đó tỉ lệ 8:2 cho R^2 cao nhất và MAE thấp nhất nhờ tập huấn luyện lớn hơn. Việc giảm chiều bằng PCA vẫn giữ được thông tin quan trọng nhất, giúp mô hình Softmax dự đoán xác suất chính xác và duy trì khả năng tổng quát hóa tốt. So với dữ liệu gốc, hiệu suất hiện tại cao hơn, chứng tỏ quá trình tiền xử lý và giảm chiều giúp loại bỏ nhiễu, nâng cao khả năng học của mô hình. Tuy nhiên, kết quả quá hoàn hảo với MSE và MAE gần bằng 0 cũng gợi ý khả năng bị overfitting, bởi mô hình có thể học gần như tất cả các đặc trưng trong tập huấn luyện, dẫn đến độ chính xác trên tập test cao hơn thực tế.

Nhận xét chung cho 2 hướng dữ liệu

Hai hướng tiếp cận (dữ liệu gốc và dữ liệu giảm chiều bằng PCA) cho thấy sự khác biệt rõ rệt về hiệu suất mô hình. Dữ liệu gốc đạt R^2 trong khoảng 0.8824 – 0.9228, với MSE (0.0079 – 0.0146) và MAE (0.0746 – 0.1024), phản ánh hiệu suất ổn định và khả năng dự đoán tốt. Ngược lại, dữ liệu giảm chiều bằng PCA chỉ còn 10 thành phần chính (còn 1/3 so với chiều của dữ liệu gốc), cho kết quả quá cao với $R^2 \approx 0.9997 - 0.9998$, MSE và MAE gần bằng 0, cho thấy mô hình

dự đoán gần như hoàn hảo trên tập kiểm tra. Sự khác biệt này phản ánh rằng việc giảm chiều giúp loại bỏ nhiễu và giữ lại các đặc trưng quan trọng nhất, làm mô hình học tốt hơn, nhưng đồng thời cũng tạo điều kiện cho overfitting: mô hình có thể học gần như tất cả các đặc trưng trong tập huấn luyện, dẫn đến độ chính xác trên tập test cao hơn thực tế. Tỷ lệ chia 8:2 thường cho hiệu suất tốt nhất nhờ tập huấn luyện lớn hơn, cả với dữ liệu gốc và dữ liệu giảm chiều PCA. Nhìn chung, dữ liệu gốc mang lại hiệu suất ổn định và an toàn hơn, trong khi dữ liệu PCA cần cân nhắc thêm về số lượng thành phần hoặc các phương pháp điều chỉnh để tránh overfitting và duy trì khả năng tổng quát hóa của mô hình.

7.10 Chuyển bài toán về dạng hồi quy với SVM

7.10.1 Lý thuyết

SVM (Support Vector Machine)

SVM là một mô hình phân loại mạnh mẽ, thường sử dụng các hàm kernel để xử lý dữ liệu không tuyến tính. Trong bài này, SVM được cấu hình với `kernel='rbf'` (Radial Basis Function), cho phép mô hình phân tách dữ liệu trong không gian cao hơn thông qua ánh xạ phi tuyến. Tham số `C=1.0` kiểm soát mức độ phạt sai phân loại, `gamma='scale'` tự động điều chỉnh độ rộng của kernel dựa trên dữ liệu, `class_weight='balanced'` cân bằng trọng số giữa các lớp để xử lý dữ liệu không cân bằng, và `probability=True` cho phép dự đoán xác suất. Xác suất của lớp k được ước lượng dựa trên đầu ra của SVM với hàm quyết định được chuẩn hóa.

Chuyển thành bài toán hồi quy

Thay vì chỉ dự đoán nhãn, xác suất đầu ra từ SVM (được kích hoạt bởi `probability=True`) được sử dụng làm mục tiêu cho mô hình hồi quy tuyến tính:

$$\hat{y} = w^T x + b,$$

sai số được tối ưu bằng MSE. Với bài toán nhị phân (0 và 1), xác suất $P(y = 1|x)$ được huấn luyện riêng biệt làm mục tiêu hồi quy.

Phần giảm chiều dữ liệu và các chỉ số đánh giá mô hình có thể tham khảo thêm trong phần 5.9.1.

7.10.2 Các bước triển khai mô hình

Phần này trình bày chi tiết các bước triển khai mô hình chuyển đổi SVM thành bài toán hồi quy trên tập dữ liệu khách hàng hàng không đã được xử lý. Dữ liệu đã được giảm chiều (gốc, PCA) và chia thành các tập train-test theo các tỉ lệ 8:2, 7:3, và 6:4.

Bước 1: Chuẩn bị dữ liệu đã xử lý

- **Dữ liệu sẵn có:** Dữ liệu đã được xử lý, bao gồm dữ liệu gốc và dữ liệu giảm chiều bằng PCA.
- **Chia tập dữ liệu:** Dữ liệu đã được chia sẵn thành tập huấn luyện và tập kiểm tra với các tỉ lệ 8:2, 7:3, và 6:4.
- **Kiểm tra dữ liệu:** Đảm bảo dữ liệu đã được chuẩn hóa và sẵn sàng để huấn luyện.

Bước 2: Huấn luyện SVM

- **Khởi tạo mô hình:** Sử dụng SVC từ scikit-learn với tham số `kernel='rbf'`, `C=1.0`, `gamma='scale'`, `class_weight='balanced'`, và `probability=True`.
- **Huấn luyện trên từng loại dữ liệu:** Huấn luyện riêng trên dữ liệu gốc và PCA, với từng tỉ lệ chia 8:2, 7:3, và 6:4.
- **Lấy xác suất:** Sử dụng `predict_proba` để lấy xác suất đầu ra cho phân lớp 1 trên tập train và test.

Bước 3: Chuyển thành bài toán hồi quy

- **Huấn luyện mô hình hồi quy:**
 - Sử dụng `LinearRegression` từ scikit-learn.
 - Huấn luyện một mô hình dự đoán xác suất lớp 1.
 - Huấn luyện trên tập train của từng loại dữ liệu (gốc, PCA).

Bước 4: Đánh giá hiệu suất

- **Dự đoán trên tập test:** Sử dụng mô hình hồi quy để dự đoán xác suất lớp 1 trên tập test.

- **Tính toán chỉ số:** Tính MSE, MAE, và R^2 cho từng loại dữ liệu và tỉ lệ chia, bằng các hàm từ scikit-learn (`mean_squared_error`, `mean_absolute_error`, `r2_score`).
- **Lưu kết quả:** Tổng hợp kết quả vào bảng để so sánh giữa các hướng dữ liệu và tỉ lệ chia.

7.10.3 Dữ liệu gốc

Kết quả trên dữ liệu gốc với các tỉ lệ chia 8:2, 7:3, và 6:4 được trình bày trong bảng dưới đây:

Tỉ lệ	Phân lớp	MAE	MSE	R^2
8:2	1	0.0110	0.0002	0.9661
7:3	1	0.0107	0.0002	0.9719
6:4	1	0.0113	0.0002	0.9633

Bảng 7.26: Kết quả trên dữ liệu gốc với các tỉ lệ chia khác nhau

Nhận xét : Nhận xét: Dữ liệu gốc đạt R^2 trong khoảng 0.9633 – 0.9719, với MSE khoảng 0.0002 và MAE khoảng (0.0107– 0.0113). Các chỉ số này cho thấy mô hình hồi quy dựa trên xác suất từ SVM có hiệu suất rất cao, với khả năng giải thích khoảng 96–97% biến thiên của dữ liệu. Tỉ lệ 7:3 đạt R^2 cao nhất (0.9719), nhờ sự cân bằng hợp lý giữa tập huấn luyện và tập kiểm tra, trong khi tỉ lệ 6:4 ($R^2 = 0.9633$) và 8:2 ($R^2 = 0.9661$) có hiệu suất thấp hơn một chút do khác biệt về kích thước tập huấn luyện. MSE và MAE ở mức rất thấp, phản ánh dự đoán chính xác gần như hoàn hảo của mô hình trên tập test.

7.10.4 Dữ liệu giảm chiều bằng PCA

Kết quả trên dữ liệu giảm chiều theo phương pháp PCA (giảm 1/3 chiều dữ liệu gốc) với các tỉ lệ chia 8:2, 7:3, và 6:4 được trình bày trong bảng dưới đây:

Tỉ lệ	Phân lớp	MAE	MSE	R^2
8:2	1	0.0478	0.0031	0.9465
7:3	1	0.0492	0.0034	0.9435
6:4	1	0.0542	0.0040	0.9305

Bảng 7.27: Kết quả trên dữ liệu PCA với các tỉ lệ chia khác nhau

Nhận xét: Dữ liệu giảm chiều bằng PCA (giảm 1/3 chiều dữ liệu gốc) cho kết quả khá cao, với R^2 dao động trong khoảng 0.9305 – 0.9465, MSE từ 0.0031 – 0.0040 và MAE từ 0.0478 – 0.0492. Các chỉ số này cho thấy mô hình hồi quy dựa trên xác suất từ SVM vẫn duy trì hiệu suất tốt, mặc dù thấp hơn đôi chút so với dữ liệu gốc ($R^2 \approx 0.9661 - 0.9719$). Tỷ lệ 8:2 đạt R^2 cao nhất (0.9465), nhờ tập huấn luyện lớn hơn, trong khi tỷ lệ 6:4 (R^2 0.9305) có hiệu suất thấp hơn do tập huấn luyện nhỏ hơn. MSE và MAE ở mức rất thấp, phản ánh dự đoán chính xác của mô hình. So với dữ liệu gốc, kết quả PCA vẫn đảm bảo khả năng tổng quát hóa tốt, đồng thời giảm nhẹ độ phức tạp nhờ giảm chiều dữ liệu.

Nhận xét chung cho hai hướng dữ liệu

Hai hướng tiếp cận (dữ liệu gốc và dữ liệu giảm chiều bằng PCA) cho thấy sự khác biệt rõ rệt về hiệu suất mô hình. Dữ liệu gốc đạt R^2 trong khoảng 0.9633 – 0.9719, với MSE khoảng 0.0002 và MAE khoảng (0.0107 – 0.0113), phản ánh khả năng giải thích rất cao (96–97% biến thiên) và dự đoán chính xác gần như hoàn hảo trên tập test. Ngược lại, dữ liệu PCA cho kết quả R^2 từ 0.9305 – 0.9465, MSE từ 0.0031 – 0.0040 và MAE từ 0.0478 – 0.0492, vẫn giữ hiệu suất tốt nhưng thấp hơn đôi chút so với dữ liệu gốc do một phần thông tin bị mất khi giảm chiều. Tỷ lệ 8:2 thường cho R^2 cao nhất nhờ tập huấn luyện lớn hơn, trong khi tỷ lệ 6:4 có hiệu suất thấp hơn.

Nhìn chung, dữ liệu gốc mang lại hiệu suất tối ưu và ổn định, trong khi dữ liệu PCA giúp giảm nhẹ độ phức tạp của mô hình mà vẫn duy trì khả năng tổng quát hóa tốt, cũng như tránh overfitting. Kết quả này cho thấy việc giảm chiều bằng PCA là khả thi nhưng cần cân nhắc lựa chọn số lượng thành phần để vừa giữ được thông tin quan trọng, vừa tránh giảm quá nhiều hiệu suất mô hình.

7.10.5 So sánh hiệu suất hai mô hình sử dụng trong bài toán hồi quy dựa trên Softmax Regression và SVM

Bảng 7.28 trình bày kết quả so sánh hiệu suất hai mô hình hồi quy dựa trên Softmax Regression và SVM (RBF), được đánh giá trên dữ liệu gốc và dữ liệu giảm chiều bằng PCA, thông qua các chỉ số R^2 , MSE và MAE.

Kết quả thực nghiệm cho thấy hai mô hình hồi quy dựa trên Softmax Regression và SVM có sự khác biệt rõ rệt về hiệu suất khi áp dụng trên dữ liệu gốc và dữ liệu giảm chiều bằng PCA.

Trên dữ liệu gốc, mô hình hồi quy dựa trên SVM cho hiệu suất vượt trội hơn so với Softmax Regression. Cụ thể, SVM đạt giá trị R^2 rất cao trong khoảng 0.9633 – 0.9719, với MSE xấp xỉ 0.0002 và MAE chỉ khoảng 0.0107 – 0.0113,

Tiêu chí	Softmax – Gốc	SVM (RBF) – Gốc	Softmax – PCA	SVM (RBF) – PCA
R^2	0.8824 – 0.9228	0.9633 – 0.9719	0.9997 – 0.9998	0.9305 – 0.9465
MSE	0.0079 – 0.0146	≈ 0.0002	≈ 0.00001 – 0.00002	0.0031 – 0.0040
MAE	0.0746 – 0.1024	0.0107 – 0.0113	0.0024 – 0.0034	0.0478 – 0.0492
Tỉ lệ tốt nhất	8:2 ($R^2 = 0.9228$)	7:3 ($R^2 = 0.9719$)	8:2 / 6:4 ($R^2 = 0.9998$)	8:2 ($R^2 = 0.9465$)
Đánh giá tổng thể	Hiệu suất tốt, ổn định	Hiệu suất rất cao	Gần như hoàn hảo, có nguy cơ overfitting	Tốt, nhưng giảm so với dữ liệu gốc
Ghi chú	Phù hợp hồi quy xác suất	Mạnh với dữ liệu đầy đủ	Nhạy cảm với PCA	Ổn định, ít overfitting

Bảng 7.28: So sánh hiệu suất giữa Softmax Regression và SVM (RBF) trên dữ liệu gốc và dữ liệu PCA

cho thấy khả năng dự đoán xác suất gần như chính xác tuyệt đối và khả năng giải thích tới 96–97% biến thiên của dữ liệu. Trong khi đó, Softmax Regression trên dữ liệu gốc đạt R^2 trong khoảng 0.8824 – 0.9228, với MSE và MAE cao hơn, phản ánh hiệu suất tốt nhưng chưa đạt mức tối ưu như SVM. Điều này cho thấy SVM khai thác hiệu quả hơn cấu trúc dữ liệu đầy đủ khi chuyển sang bài toán hồi quy.

Ngược lại, trên dữ liệu giảm chiều bằng PCA, mô hình Softmax Regression thể hiện ưu thế rõ rệt. Với chỉ khoảng 1/3 số chiều ban đầu, Softmax vẫn đạt R^2 gần bằng 1 (0.9997 – 0.9998), cùng MSE và MAE gần bằng 0, phản ánh khả năng dự đoán gần như hoàn hảo trên tập kiểm tra. Tuy nhiên, kết quả này cũng tiềm ẩn nguy cơ overfitting, khi mô hình có thể đã học quá tốt cấu trúc dữ liệu sau khi giảm chiều mạnh. Trong khi đó, SVM trên dữ liệu PCA có hiệu suất giảm rõ rệt, với R^2 chỉ đạt 0.9305 – 0.9465 và sai số lớn hơn, cho thấy mô hình này nhạy cảm hơn với việc giảm chiều và mất mát thông tin.

Xét tổng thể, Softmax Regression cho hiệu suất ổn định và đáng tin cậy trên cả hai hướng dữ liệu, đặc biệt phù hợp khi kết hợp với PCA trong bài toán hồi quy xác suất. SVM tỏ ra rất mạnh trên dữ liệu gốc, nhưng hiệu suất suy giảm khi áp dụng PCA. Do đó, trong bối cảnh dữ liệu đã được giảm chiều hoặc yêu

cầu tính ổn định cao, Softmax Regression là lựa chọn phù hợp hơn. Ngược lại, nếu sử dụng dữ liệu đầy đủ và không giảm chiều, mô hình hồi quy dựa trên SVM mang lại hiệu suất cao nhất.

TỔNG KẾT

Trong khuôn khổ bài báo cáo này, chúng em đã triển khai mô hình chuẩn đoán sớm khả năng mắc bệnh ở bệnh nhân, sử dụng bộ dữ liệu "alzheimers_disease_data" từ Kaggle. Quá trình nghiên cứu được thực hiện đầy đủ và có hệ thống từ tiền xử lý dữ liệu, huấn luyện mô hình đến đánh giá kết quả, với mục tiêu tối ưu hóa, làm chậm tiến trình bệnh và cải thiện chất lượng sống cho bệnh nhân.

Về bài toán phân loại (Classification), nhóm đã áp dụng ba phương pháp chính là SVM, KNN và Softmax Regression. Kết quả thực nghiệm cho thấy KNN trên dữ liệu gốc đạt hiệu năng cao nhất, đặc biệt ổn định trong bối cảnh dữ liệu phức tạp và biến động lớn giữa các thuộc tính. SVM với kernel RBF cũng là một lựa chọn mạnh mẽ, trong khi Softmax Regression hoạt động ổn định trên các không gian đã giảm chiều. Các yếu tố như nguy cơ mắc bệnh, đánh giá chức năng, nhận thức và chỉ số sinh học đã được mô hình nhận diện rõ ràng, minh chứng cho việc KNN và SVM phù hợp với các bài toán có đặc trưng phân lớp phức tạp.

Về bài toán phân cụm (Clustering), kết quả nghiên cứu chỉ ra rằng kỹ thuật giảm chiều UMAP mang lại hiệu quả vượt trội hơn so với PCA trong việc tách biệt các nhóm đối tượng. Cụ thể:

- Sự kết hợp giữa UMAP và K-means đạt chất lượng cao nhất trong các thử nghiệm, đặc biệt phù hợp khi cấu trúc lớp rõ ràng và cần cụm đầy đủ.
- Thuật toán DBSCAN trên không gian UMAP chứng minh được khả năng tạo ra các cụm có độ đặc rất cao, hữu ích trong việc phát hiện ngoại lai hoặc các cụm có hình dạng bất kỳ.
- Ngược lại, việc kết hợp PCA với DBSCAN cho hiệu quả thấp nhất do làm mất thông tin phi tuyến, dẫn đến chỉ số Silhouette thấp. Do đó, nếu ưu tiên độ tinh khiết của cụm, UMAP là lựa chọn tối ưu, trong khi PCA chỉ nên cân nhắc khi bắt buộc phải bảo toàn phương sai dữ liệu gốc.

Mặc dù báo cáo đã đạt được những kết quả khả quan, nhưng vẫn còn một số hạn chế. Thứ nhất, bộ dữ liệu tuy phong phú nhưng có thể chưa đủ đại diện cho mọi trường hợp thực tế. Thứ hai, các mô hình đơn giản được sử dụng (SVM, KNN, Softmax) phần nào giới hạn khả năng khai thác các mối quan hệ phi tuyến sâu hơn. Thứ ba, việc so sánh ảnh hưởng của các kỹ thuật giảm chiều

(như PCA, LDA, UMAP) lên hiệu suất mô hình vẫn cần được phân tích định lượng sâu hơn nữa.

Trong tương lai, hướng phát triển tiếp theo sẽ bao gồm việc mở rộng tập dữ liệu, thử nghiệm các mô hình học sâu (Deep Learning), áp dụng các kỹ thuật chọn đặc trưng nâng cao và triển khai mô hình trong môi trường thực tế để đánh giá tính thích nghi trong các tình huống đa dạng hơn.

Tài liệu tham khảo

- [1] Cao Van Chung, “*Machine Learning*,” lecture slides, Informatics Dept., MIM, HUS, VNU Hanoi, 2024. Available:
<https://drive.google.com/file/d/1xqvPCWeQeJlPfdexSxbcvf2WloQtCJDW6/view>.
- [2] Pham Dinh Khanh, “Phương pháp phân cụm dựa trên mật độ (DBSCAN)” *Deep AI KhanhBlog*, 2021. Available:
https://phamdinhkhanh.github.io/deepai-book/ch_ml/DBSCAN.html.
- [3] Cao Văn Chung - Support Vector Machine
<https://drive.google.com/file/d/12G7h7CSQqvovzx1lQY9FRzgFsvddm2bXc/view>
- [4] Support-Vector Networks, CORINNA CORTES, VLADIMIR VAPNIK, 1995, AT&T Bell Labs., Hohndel, NJ 07733, USA.
<https://link.springer.com/article/10.1007/BF00994018>
- [5] Multi-Class Support Vector Machine via Maximizing Multi-Class Margins, Jie Xu, Xianglong Liu, Zhouyuan Huo, Cheng Deng, Feiping Nie, Heng Huang
<https://www.ijcai.org/proceedings/2017/440>
- [6] Pham Dinh Khanh, “Các bước của thuật toán k-Means Clustering” *Deep AI KhanhBlog*, 2021. Available:
https://phamdinhkhanh.github.io/deepai-book/ch_ml/KMeans.html#cac-buoc-cua-thuat-toan-k-means-clustering.
- [7] Tran Thi Mai Anh - Clustering Algorithms Final Project Notebook (Google Colab), 2025.
<https://colab.research.google.com/drive/1KmSQFn9fnLQBvPb4KvqAacjngWluLoMu>